

UNIVERSIDAD NACIONAL SAN CRISTÓBAL DE HUAMANGA

Facultad de Ingeniería de Minas, Geología y Civil

Escuela Profesional de Ingeniería de Sistemas



INFORME FINAL DE PROYECTO

Democra: Plataforma Web SaaS Multi-Tenant para la Gestión Integral de Organizaciones de Voluntariado (ONG)

Asignatura: Pruebas y Aseguramiento de la Calidad de Software **Presentado por:**

N.º	Apellidos y Nombres	Código	Docente:
1	Paipay Vega, Eduardo Sebastián	27222136	

Mg. Ing. Richard Zapata Casaverde

Dedicatoria

*A quienes dedican su tiempo al servicio de los demás a través del voluntariado,
y a los docentes de la Escuela Profesional de Ingeniería de Sistemas,
cuya formación hizo posible este trabajo.*

Agradecimiento

Expreso mi agradecimiento a la Universidad Nacional de San Cristóbal de Huamanga y, en particular, al docente de la asignatura de Pruebas y Aseguramiento de la Calidad de Software, por la orientación metodológica brindada durante el desarrollo de este informe. Agradezco asimismo a quienes contribuyeron con la construcción y documentación de la plataforma Democra, cuyo repositorio constituye el objeto de estudio del presente trabajo.

Resumen

El presente informe documenta y evalúa, mediante un enfoque de ingeniería inversa documental sobre el repositorio, el diseño y la implementación de **Democra**, una plataforma web de tipo *Software as a Service* (SaaS) multi-tenant para la gestión integral de organizaciones de voluntariado (ONG). El sistema se estructura en un cliente *Single Page Application* (SPA) construido con React, TypeScript y Vite, y una interfaz de programación de aplicaciones (API) REST implementada con Node.js y Express sobre la plataforma de datos Supabase (PostgreSQL 16). A nivel de datos, adopta un modelo multi-esquema (`public`, `ong`, `rrhh`, `finanzas`, `clinico`, `academico`, `comunicaciones`) con aislamiento por inquilino reforzado por *Row Level Security* (RLS) y funciones de base de datos. La seguridad se sustenta en autenticación Supabase, control de acceso basado en roles y permisos (RBAC) con negación por defecto, un motor de evaluación de riesgo con verificación en dos pasos (OTP), auditoría forense y borrado lógico con políticas de retención. Metodológicamente, el desarrollo se gobierna con la metodología DDS (*Design Documentation System* asistido por IA), organizada en ocho fases del ciclo de vida. La investigación es de tipo aplicada-tecnológica, nivel descriptivo y diseño no experimental transversal. Las variables de estudio son el Funcionamiento y la Usabilidad; sus indicadores se validan mediante análisis documental del código, pruebas automatizadas (Jest, Vitest, supertest, pgTAP) y cobertura de código y pruebas automatizadas. Los resultados cuantitativos de cobertura y rendimiento se estructuran y se reservan para su ejecución en un entorno controlado, sin reportar valores no verificados.

Palabras clave: SaaS multi-tenant, ONG, voluntariado, React, Supabase, PostgreSQL, RBAC, RLS, aseguramiento de la calidad, metodología DDS.

Abstract

This report documents and evaluates, through a documentary reverse-engineering approach over the repository, the design and implementation of **Democra**, a multi-tenant Software as a Service (SaaS) web platform for the comprehensive management of volunteer organizations (NGOs). The system comprises a Single Page Application (SPA) client built with React, TypeScript and Vite, and a REST API implemented with Node.js and Express on top of the Supabase data platform (PostgreSQL 16). At the data level, it adopts a multi-schema model with tenant isolation enforced through Row Level Security (RLS) and database functions. Security relies on Supabase authentication, Role-Based Access Control (RBAC) with a deny-by-default policy, a risk evaluation engine with two-step verification (OTP), forensic auditing, and soft deletion with retention policies. Methodologically, the development is governed by the DDS methodology (Design Documentation System assisted by AI), organized into eight lifecycle phases. The research is applied-technological and descriptive, with a non-experimental cross-sectional design. The study variables are Functioning and Usability; their indicators are validated through documentary code analysis, automated testing (Jest, Vitest, supertest, pgTAP) and code coverage and automated testing. Quantitative coverage and performance results are structured and reserved for execution in a controlled environment, without reporting unverified values.

Keywords: multi-tenant SaaS, NGO, volunteering, React, Supabase, PostgreSQL, RBAC, RLS, quality assurance, DDS methodology.

Índice general

Dedicatoria	I
Agradecimiento	II
Resumen	III
Abstract	IV
Introducción	1
1 Planteamiento del Problema	2
1.1 Diagnóstico y Enunciado del Problema	2
1.2 Formulación del Problema	3
1.2.1 Problema General	3
1.2.2 Problemas Específicos	3
1.3 Hipótesis	3
1.4 Objetivos	4
1.4.1 Objetivo General	4
1.4.2 Objetivos Específicos	4
1.5 Justificación y Delimitación	5
1.5.1 Importancia	5
1.5.2 Justificación Técnica y Tecnológica	5
1.5.3 Justificación Social	5
1.5.4 Justificación Académica	5
1.5.5 Delimitación	6
1.5.6 Limitaciones y Alcance Restrictivo	6
2 Marco Teórico y Contextual	7
2.1 Presentación y Arquitectura Epistemológica del Marco Teórico	7
2.2 Antecedentes de la Investigación	10
2.2.1 Antecedentes Internacionales	10
2.2.2 Antecedentes Nacionales	12
2.3 Fundamentos de la Ingeniería Web	15
2.3.1 El Glosario Fundamental: Anatomía de la Web	15
2.3.2 Ingeniería de Alta Complejidad: El Paradigma de la Web Dinámica y	16

2.4	El Dominio de Negocio: Organizaciones No Gubernamentales (ONG) y el Fenómeno del Data Siloing	18
2.4.1	Definición Operativa y Carga Logística Asimétrica de las ONG	18
2.4.2	El Cuello de Botella Endémico: Data Siloing en Redes Domésticas	18
2.4.3	El Impacto Crítico en Privacidad y Tolerancia a Fallos	19
2.5	Evaluación y Descarte Arquitectónico del Estándar DDS (Data Distribution Service)	20
2.5.1	Conceptualización y Origen del Estándar DDS	20
2.5.2	El Choque Paradigmático: Militarista vs Ecosistema Civil SaaS	21
2.5.3	El Triunfo Arquitectónico de los WebSockets sobre DDS	21
2.6	Métricas de Calidad de Servicio (QoS): Falacia Teórica frente al Desempeño Real Web	22
2.6.1	El Laberinto de las Políticas QoS	22
2.6.2	El Caos del Determinismo en Redes Indeterministas	23
2.6.3	El Triunfo Arquitectónico : WebSockets vs QoS	23
2.7	Evaluación Topológica del Tráfico de Red: UDP frente a la Integridad Estricta de TCP	23
2.7.1	El Espejismo de la Velocidad UDP	24
2.7.2	La Fortaleza Inquebrantable de TCP	24
2.8	Arquitectura Multi-Tenant, Aislamiento Logístico y Persistencia ACID	24
2.8.1	El Dilema Topológico: Silo Dedicado frente a Pool Compartido	25
2.8.2	El Muro Criptográfico: Row-Level Security (RLS)	25
2.8.3	El Rigor Transaccional: Las Leyes de Persistencia ACID	26
2.9	Arquitectura del <i>Backend</i> : El Entorno de Ejecución <i>Node.js</i>	26
2.9.1	El Cuello de Botella Tradicional: Thread-per-Request	26
2.9.2	Abstracción : Non-Blocking I/O	27
2.9.3	El Motor de WebSockets: Orquestación en Tiempo Real	27
2.10	Abstracción del Renderizado <i>Frontend</i> : <i>React.js</i> y la Optimización del <i>Virtual DOM</i>	27
2.10.1	La Ineficiencia Crítica del DOM Real	27
2.10.2	La Solución Heurística: Virtual DOM y Algoritmo de Reconciliación	28
2.11	Metodología de Ingeniería Autónoma: <i>Design Documentation System</i> (DDS) Asistido por Inteligencia Artificial	28
2.11.1	Fundamentación de las Fases del Flujo DDS	29
3	Material y Métodos	30
3.1	Tipo de Investigación	30
3.1.1	Justificación Epistemológica del Modelado Algorítmico	30
3.1.2	El Artefacto Tecnológico como Objeto de Intervención Sistémica	31
3.2	Nivel de Investigación	33
3.2.1	El Rigor de la Medición Paramétrica	34
3.2.2	La Profundidad Descriptiva del Artefacto de Software	34
3.3	Diseño de Investigación	35
3.3.1	Defensa Algorítmica del Diseño Transversal	35
3.3.2	Protección contra Contaminación Exógena	35
3.4	Población y Muestra	36
3.4.1	Población y Muestra Técnica	36

3.5	Variables e Indicadores	37
3.5.1	Matriz de Operacionalización: Cobertura y Desempeño	37
3.5.2	Traducción Matemática y Umbrales Rígidos	37
3.6	Técnicas e Instrumentos de Recolección y Análisis Forense	38
3.6.1	White-Box Penetration Testing y Aserción Asíncrona	38
3.6.2	Herramientas de Estrés Asíncrono y Concurrencia Transaccional	39
3.7	Validez y Confiabilidad	40
3.7.1	Escrutinio Técnico: Determinismo Algorítmico	40
3.7.2	Validez Constructiva Técnica	41
3.8	Blindaje Ético Computacional y Soberanía de Software	41
3.8.1	Arquitectura <i>Zero-Trust</i> y Criptografía Forense	41
3.8.2	Soberanía de Software y Licenciamiento Legítimo	43
4	Resultados y Discusión	45
4.1	Requisitos Funcionales del Sistema	45
4.2	Cobertura de Pruebas del Servidor Backend (Node.js/Express)	47
4.2.1	Resultados Globales de Cobertura	47
4.2.2	Análisis de la Cobertura por Categoría	48
4.3	Documentación y Cobertura de la API REST	49
4.3.1	Especificación Técnica de la API	49
4.3.2	Distribución de Endpoints por Módulo	49
4.3.3	Análisis de la Documentación OpenAPI	49
4.4	Pruebas Unitarias del Backend	51
4.4.1	Resultados Generales de Ejecución	51
4.4.2	Distribución de Suites por Categoría	51
4.4.3	Análisis de Resultados por Componente	51
4.5	Flujo del Design Documentation System (DDS)	53
4.5.1	Fase 0: Developer Experience (DX)	53
4.5.2	Fase 1: Descubrimiento y Análisis	53
4.5.3	Fase 2: Innovación y Validación	53
4.5.4	Fase 3: Diseño y Definición	53
4.5.5	Fase 4: UI/UX	54
4.5.6	Fase 5: Arquitectura y Desarrollo	54
4.5.7	Fase 6: QA y Testing	54
4.5.8	Fase 7: Despliegue y Operaciones	54
4.5.9	Diagrama del Flujo DDS	54
4.5.10	Mapeo de Fases y Artefactos	55
4.5.11	Arquitectura de la Plataforma	56
4.5.12	Stack Tecnológico y Estandarización Arquitectónica	56
4.6	Discusión de Resultados	57
4.6.1	Contraste de Eficiencia y Seguridad en Arquitecturas Multi-Tenant	57
4.6.2	Comportamiento Asíncrono frente a Flujos Gubernamentales Históricos	58
4.6.3	Limitaciones e Hilos Conductores Futuros	59

4.7	Evaluación de Funcionamiento: Pruebas de API (Backend)	59
4.8	Evaluación de Integración Transversal (Frontend, Backend y BD)	62
4.8.1	El Viaje del Token (JWT) a través de las Capas	63
4.8.2	Persistencia Paramétrica y Llaves Foráneas (MySQL/PostgreSQL)	65
5	Conclusiones y Recomendaciones	68
5.1	Conclusiones	68
5.2	Recomendaciones	70
ANEXOS		75
A	Anexo A: Prototipado de Baja Fidelidad	75
B	Anexo B: Capturas del Sistema Implementado	76
C	Anexo C: Evidencias Complementarias de Pruebas	84

Introducción

La transformación digital ha dejado de ser una ventaja diferencial para convertirse en una condición operativa de las organizaciones contemporáneas. En las organizaciones sin fines de lucro y de voluntariado (ONG), la gestión de personas, proyectos, recursos y procesos de gobernanza suele apoyarse en herramientas fragmentadas —hojas de cálculo, mensajería informal y formularios dispersos— que dificultan la trazabilidad, comprometen la seguridad de datos sensibles y limitan la rendición de cuentas ante donantes y entes reguladores. El modelo de software como servicio (SaaS) multi-tenant responde a esta necesidad al ofrecer, desde una única base de código, un servicio compartido por múltiples organizaciones, manteniendo el aislamiento estricto de sus datos (Fowler & Lewis, 2015; Red Hat Research, 2023).

El presente informe describe y evalúa **Democra**, una plataforma web SaaS multi-tenant para la gestión integral de ONG, tal como se encuentra materializada en su repositorio de código fuente. El sistema —originalmente desarrollado como *SistemaVolV2.0* e integrado posteriormente en el *mono-repo* Democra— implementa una arquitectura desacoplada de cliente y servidor, un modelo de datos relacional multi-esquema sobre PostgreSQL administrado a través de Supabase, y un conjunto de mecanismos transversales de seguridad, auditoría y control de acceso, gobernados por la metodología de documentación de diseño DDS.

La documentación se elabora bajo un enfoque de *ingeniería inversa documental*: el repositorio constituye la única fuente de verdad, y el análisis describe fielmente lo que el código implementa, sin inventar funcionalidades, arquitecturas ni métricas inexistentes. Las secciones cuyos valores dependen de la ejecución de pruebas en un entorno controlado —cobertura efectiva, tiempos de respuesta, resultados de usabilidad— se marcan explícitamente como pendientes de evidencia.

El informe adopta la estructura IMRyD (Introducción, Método, Resultados y Discusión) propia de la investigación aplicada. El primer capítulo plantea el problema, la hipótesis, los objetivos y la justificación. El Capítulo 2 desarrolla los antecedentes, el marco teórico-tecnológico, la metodología DDS y la justificación del *stack*. El Capítulo 3 expone el material y los métodos. El ?? describe la arquitectura. El ?? detalla el modelo de datos. El ?? documenta los módulos funcionales. El ?? especifica la API y los servicios de seguridad. El ?? expone el aseguramiento de la calidad. El ?? presenta los resultados y el ?? su discusión. El Capítulo 5 sintetiza conclusiones y recomendaciones.

CAPITULO 1

Planteamiento del Problema

1.1. Diagnóstico y Enunciado del Problema

Las organizaciones de voluntariado y del tercer sector enfrentan el reto mayúsculo de gestionar procesos organizacionales que, por su propia naturaleza descentralizada y altruista, suelen estar altamente fragmentados. Estos procesos abarcan un amplio espectro de la gobernanza: desde la incorporación y retención de voluntarios (*onboarding*), la planificación operativa y logística, hasta la asignación y el seguimiento de tareas especializadas. Más crítico aún es el resguardo, el procesamiento y el archivo de información clínica, financiera y personal extremadamente sensible que fluye a través de estas entidades operativas. Según el Registro Nacional de Organizaciones No Gubernamentales de Desarrollo de la Agencia Peruana de Cooperación Internacional (APCI), existen más de un millar de organizaciones formales operando activamente en el país ([Agencia Peruana de Cooperación Internacional, 2024](#)). Asimismo, el Instituto Nacional de Estadística e Informática (INEI) reporta que las asociaciones civiles y organizaciones sin fines de lucro representan una porción significativa y en constante crecimiento de la demografía empresarial formal ([Instituto Nacional de Estadística e Informática, 2024](#)).

A pesar de esta evidente magnitud institucional y de la creciente demanda por intervención social, estas organizaciones, en su gran mayoría, padecen un evidente rezago tecnológico. Cuando los procesos operativos de misión crítica se soportan en herramientas ofimáticas aisladas u hojas de cálculo compartidas de manera precaria, surgen tres problemas estructurales y sistémicos.

Primero, la ausencia de trazabilidad. La carencia de un sistema unificado y con capacidad de registro transaccional dificulta reconstruir de forma auditable el historial de manipulación de los datos (quién realizó qué acción, cuándo y sobre qué dato), un aspecto no negociable para las exigencias de transparencia, rendición de cuentas financieras ante cooperantes internacionales y certificaciones gubernamentales.

Segundo, el problema del *data bleeding* y la dispersión de datos vulnerables. La manipulación de fichas médicas de beneficiarios, documentos de identidad de voluntarios y datos bancarios eleva drásticamente el riesgo de exposición accidental o fuga deliberada de información. Este riesgo se exagera cuando no existe un mecanismo centralizado para hacer cumplir el *Principio de Mínimo Privilegio*, lo que permite que operadores de bajo rango visualicen información transversal.

Tercero, el problema de la Escalabilidad Económica. La falta de plataformas unificadas o estándares en la industria del software diseñados exclusivamente para el modelo de negocio de una ONG obliga a estas entidades a adquirir o alquilar infraestructura de TI individual, asumiendo licencias, servidores y costos de mantenimiento que diezman su liquidez. Esta multiplicidad de esfuerzos aislados limita severamente su capacidad para escalar sus programas sociales, ya que destinan valiosos recursos de donaciones a mantener una infraestructura tecnológica fragmentada.

Democra aborda directa y estructuralmente esta problemática mediante el diseño de una plataforma única tipo *Software as a Service* (SaaS) *multi-tenant*, que consolida y centraliza los múltiples dominios operativos (recursos humanos, clínica, finanzas) bajo un modelo estricto de seguridad compartimentada. A través del modelo *multi-tenant* o de «arquitectura de inquilinos», cada organización (*tenant*) opera de manera aislada e independiente sobre una macroinfraestructura compartida, aprovechando los beneficios económicos de la nube sin sacrificar la confidencialidad gracias a una separación matemática garantizada a nivel estructural en la base de datos relacional.

1.2. Formulación del Problema

1.2.1. Problema General

¿Cómo diseñar e implementar una plataforma web tipo *Software as a Service* (SaaS) *multi-tenant* que permita la gestión integral, segura, auditable y escalable de los procesos operativos de organizaciones de voluntariado, garantizando rigurosamente a nivel criptográfico y de base de datos el aislamiento de la información entre distintas organizaciones?

1.2.2. Problemas Específicos

- a. ¿Cómo estructurar una arquitectura de software profundamente desacoplada que separe la capa de presentación interactiva (SPA), la lógica de negocio (API REST) y la capa de almacenamiento de datos, maximizando la agilidad de los desarrollos futuros?
- b. ¿Cómo modelar un esquema de bases de datos que soporte simultáneamente y sin colisiones múltiples dominios de negocio complejos (finanzas, historia clínica, gestión de talento humano) y asegure determinísticamente el aislamiento por inquilino mediante políticas formales a nivel de *kernel*?
- c. ¿Cómo implementar un modelo de control de acceso granular basado en roles y permisos (RBAC) con políticas de negación por defecto, que además provea mecanismos avanzados de auditoría forense universal sobre operaciones y borrado lógico seguro de activos de información?
- d. ¿Cuál es el nivel de funcionamiento técnico, resiliencia arquitectónica y el grado de usabilidad de la plataforma resultante, al someterse a estándares rigurosos de evaluación de experiencia de usuario?

1.3. Hipótesis

Conforme a Hernández-Sampieri y Mendoza (2018), en los estudios con alcance descriptivo la formulación de hipótesis se considera pertinente y necesaria cuando la investigación pronostica el comportamiento de un hecho o variable de diseño. En estricta coherencia con ello, y sustentado en el diseño teórico-matemático y topológico del sistema de software, se enuncia la siguiente hipótesis de trabajo de naturaleza contrastable:

La arquitectura de la plataforma Democra, mediante la aplicación implacable de políticas de aislamiento de seguridad a nivel de fila (Row Level Security, RLS) y un control de

acceso basado en roles (RBAC) diseñado bajo el paradigma de negación por defecto, garantiza matemáticamente el aislamiento absoluto de los datos entre inquilinos (logrando cero accesos cross-tenant no autorizados). Adicionalmente, se plantea que esta robustez backend no degrada la experiencia de usuario, y que la plataforma alcanzará un nivel de funcionamiento verificable de sus módulos críticos, validado mediante pruebas automatizadas y métricas de cobertura de código que garanticen la correcta operación del sistema.

La contrastación empírica de esta hipótesis metodológica se operacionaliza exhaustivamente en el Capítulo 3, y los hallazgos derivados de las métricas e instrumentación analítica se documentan a detalle en el ??.

1.4. Objetivos

1.4.1. Objetivo General

Diseñar, desarrollar y documentar con precisión académica una plataforma web SaaS *multi-tenant* a escala comercial, denominada Democra. Su fin primordial es viabilizar la gestión integral y centralizada de los procesos operativos, la administración del capital humano, los recursos logísticos y la gobernanza estratégica de organizaciones de voluntariado peruanas. Esta construcción debe cimentarse en una arquitectura desacoplada moderna, provista de mecanismos transversales de seguridad perimetral, criptografía, auditoría profunda forense y metodologías de aseguramiento de la calidad; aplicando para todo el proceso la innovadora metodología de diseño *Design Documentation System* (DDS) orientada a inteligencias artificiales.

1.4.2. Objetivos Específicos

- a. Implementar una arquitectura de software distribuida y orientada a servicios (por capas) que logre un desacoplamiento total: cliente interactivo tipo *Single Page Application* basado en React y Fiber, una interfaz de aplicación programable (API REST) operada bajo el entorno no bloqueante Node.js, y un poderoso almacén transaccional sobre PostgreSQL.
- b. Definir y desplegar un modelo transaccional de base de datos multiesquema con un estricto aislamiento lógico por inquilino, orquestado dinámicamente por variables de sesión (`tenant_id`) e impermeabilizado mediante restricciones estructurales de *Row Level Security* (RLS).
- c. Programar y habilitar un modelo perimetral IAM (*Identity and Access Management*) de control de acceso basado en roles (RBAC) con denegación explícita por defecto. Adicionalmente, inyectar un *trigger* asíncrono de auditoría universal de base de datos y lógica de retención mediante borrado lógico blando (*soft delete*).
- d. Establecer y validar empíricamente una robusta estrategia de aseguramiento de la calidad (QA). Esto abarca la implementación de tipado de datos estricto desde el inicio del proyecto, validación de *inputs* en tiempo de ejecución y auditorías de seguridad continuas trazadas a lo largo de las fases de la metodología DDS.

- e. Validar con rigor científico y empírico el correcto funcionamiento de los dominios funcionales críticos de la plataforma y aplicar pruebas automatizadas de cobertura y pruebas unitarias para validar el correcto funcionamiento de los módulos críticos.

1.5. Justificación y Delimitación

1.5.1. Importancia

La trascendencia y pertinencia de la presente investigación radican en proveer al ecosistema de voluntariado y a las Organizaciones No Gubernamentales (ONG) un instrumento de base tecnológica altamente escalable, estandarizado y seguro, que moderniza drásticamente su gestión. Al proporcionar una solución integral como SaaS, en la que la ONG no asume los exorbitantes costos de mantenimiento de servidores o de seguridad, se reducen de forma significativa las brechas tecnológicas del tercer sector en un país en vías de desarrollo, lo que apalanca la digitalización inclusiva e inserta a estas instituciones en la economía global del dato.

1.5.2. Justificación Técnica y Tecnológica

A nivel estrictamente ingenieril, el proyecto halla su fundamentación en la convergencia orquestada de paradigmas modernos de arquitectura de software y seguridad ofensiva/defensiva. El diseño contempla arquitecturas *serverless*, la programación declarativa en interfaces (*React Fiber*), así como estrategias profundas de aislamiento (*Tenant Data Bleeding Prevention*). La centralización de la lógica en esquemas fuertemente tipados de base de datos, soportados bajo el principio ACID de PostgreSQL 16 y envueltos por políticas RLS y *triggers* de *Change Data Capture* (CDC), eleva la calidad arquitectónica. Adicionalmente, la adopción nativa de la metodología *Design Documentation System* (DDS) para coordinar con *Large Language Models* (LLMs) marca un precedente de vanguardia en la ingeniería reversa de requerimientos y asegura un *Single Source of Truth* (SSOT).

1.5.3. Justificación Social

El tercer sector maneja la delicada labor de intervenir ante crisis humanitarias, brindar salud y proporcionar bienestar a comunidades excluidas. Sin embargo, su capacidad multiplicadora suele atrofiarse bajo la carga burocrática. Proveer a las organizaciones de una herramienta centralizada, segura y matemáticamente trazable fortalece radicalmente su resiliencia administrativa. Esto permite que dirijan un mayor volumen de sus recursos financieros y de esfuerzo humano directamente hacia sus misiones sociales en lugar de tareas operativas administrativas, maximizando su impacto filantrópico en el territorio nacional.

1.5.4. Justificación Académica

En el ámbito académico y en el corazón de la epistemología de la Ingeniería de Sistemas, el diseño topológico, la programación sistémica y la posterior validación experimental de "Demo-
cra" constituyen un insumo invaluable y un hito referencial. El trabajo compila la metodología *end-to-end* (desde requisitos hasta operaciones *cloud*), documenta matemáticamente las estrategias de seguridad perimetral de un SaaS genuino y elabora análisis estadísticos (alfa de Cronbach, cobertura de código), posicionándose como un manual de ingeniería de altísimo rigor académico aplicable a

arquitecturas en la nube y ciberseguridad defensiva.

1.5.5. Delimitación

Espacial: La traza de ejecución, pruebas y depuración del presente trabajo está centralizada e inmersa enteramente en el repositorio privado del código fuente del proyecto "Democraz en su despliegue técnico en la plataforma de servicios en la nube (Supabase Cloud). **Temporal:** Corresponde estrictamente al desarrollo, revisión y estado arquitectónico del software tal cual es referenciado a lo largo del periodo de desarrollo comprendido en el año 2026. **Temática:** El análisis es interdisciplinar y cruza las fronteras cognitivas de la Ingeniería de Software Avanzada, Arquitectura de Componentes Web (SaaS y *multi-tenant*), Seguridad de la Información e infraestructuras *Zero-Trust* y Aseguramiento Cuantitativo de la Calidad del Software.

1.5.6. Limitaciones y Alcance Restrictivo

Conscientes del rigor epistemológico requerido para un trabajo de tesis de sistemas, se precisan las siguientes limitaciones teóricas y operativas del estudio empírico:

1. **Limitante Metodológica (Enfoque DDS):** Dado el uso intrínseco del enfoque híbrido documental (DDS) que asiste al ciclo de ingeniería con IA, la recuperación y el análisis del modelo se priorizan sobre el estudio de los metadatos de los flujos operacionales. El análisis forense abarca la arquitectura de lectura del código y no necesariamente un *post mortem* tras la salida pública del producto; es decir, no abarca una observación dinámica en el nicho comercial.
2. **Validación Acotada (QA Sandbox):** La cuantificación técnica de la latencia, cobertura de código fuente y evaluaciones de métricas de experiencia mediante pruebas automatizadas han sido controladas paramétricamente en *sandboxes* o escenarios de ensayo, y no han sido sometidas a una simulación de estrés DDoS externa (*stress testing* con tráfico *black hat* en producción).
3. **Naturaleza Evolutiva del Ecosistema:** Como corolario de cualquier investigación basada en un *stack* tecnológico en vías de hiperdesarrollo (*TypeScript*, *React Fiber 19*, *Supabase PostgreSQL*), algunos métodos sintácticos, topologías o declaraciones expuestas poseen una dependencia temporal y están sujetas a deprecación si los mantenedores primarios liberan *breaking changes* futuros a nivel global.

CAPITULO 2

Marco Teórico y Contextual

2.1. Presentación y Arquitectura Epistemológica del Marco Teórico

El presente capítulo constituye el cimiento ontológico, el pilar epistemológico y la base doctrinaria indispensable sobre la cual se erige, se fundamenta y se justifica integralmente la arquitectura algorítmica y estructural del proyecto de ingeniería de software denominado *Democra*. En el exigente y riguroso universo de la investigación aplicada a las ciencias de la computación, el Marco Teórico trasciende la función anacrónica de ser un mero repositorio pasivo de definiciones triviales o una simple colección de extractos bibliográficos inconexos; por el contrario, asume el rol activo y determinante de brújula metodológica suprema. Es el plano maestro que delimita severas fronteras técnicas, justifica con contundencia e irrefutable cada decisión arquitectónica adoptada por el desarrollador, y proporciona el prisma analítico a través del cual se observarán, medirán y validarán empíricamente los fenómenos resultantes de la interacción humano-computadora en los capítulos de ingeniería posteriores.

La titánica construcción de una plataforma transaccional *Multi-Tenant*, concebida desde su código génesis para operar sin descanso en un entorno de misiones críticas logísticas (como indiscutiblemente lo es el ecosistema operativo del tercer sector y las múltiples Organizaciones No Gubernamentales distribuidas en territorio nacional), exige por mandato dogmático un escrutinio teórico de extrema profundidad que no deje ningún flanco ni conceptual al azar. Por ende, la estructura global de este capítulo ha sido meticulosamente diseñada siguiendo un orden deductivo, progresivo, e ininterrumpido, que partiendo sabiamente de lo universal y abstracto, decanta gradualmente hacia lo particular, hiper-específico y netamente operativo. Este riguroso enfoque de embudo teórico asegura categóricamente que el lector —ya sea un revisor académico estricto, un exigente jurado evaluador institucional o un experimentado arquitecto de software de la industria— comprenda no solo el qué se ha construido línea por línea, sino fundamental y principalmente el por qué científico, el motivo empírico y la causa ingenieril que legitima inexorablemente su existencia y defiende su compleja topología frente a todas las otras alternativas arcaicas disponibles actualmente en el estado del arte computacional.

En la primera fase de este inquebrantable andamiaje conceptual, se acomete una audaz reingeniería crítica de los antecedentes de investigación. Lejos de conformarse con la fácil tarea de parafrasear cómodamente los resúmenes abstractos de múltiples tesis precedentes, se despliega a propósito una autopsia analítica exhaustiva de los esfuerzos históricos, tanto internacionales como nacionales, estrechamente afines al dominio central de este estudio. Este inventario profundamente crítico disecta sin piedad las metodologías previas empleadas, aísla con la escalofriante precisión de un cirujano los problemas técnicos primarios que dichos heroicos estudios intentaron —a menudo infructuosamente o con herramientas equivocadas— resolver, y expone sin ambages ni eufemismos las brechas arquitectónicas persistentes, los indeseados cuellos de botella de rendimiento lineal y las

dolorosas limitaciones de escalabilidad masiva que padecieron en sus respectivos despliegues. Es precisamente en el valiente y frontal reconocimiento de estas históricas y sistémicas falencias donde la presente tesis universitaria ancla definitivamente su inquebrantable justificación de relevancia ingenieril, articulando una innovadora propuesta tecnológica que diverge diametral y drásticamente de los letales errores legados por el pasado, y logrando solucionar de una forma comprobable, medible y netamente empírica las carencias operacionales documentadas e identificadas históricamente en la gestión de base de datos.

Posteriormente a este vital paso, el capítulo se adentra con una profundidad clínica irrenunciable en los Fundamentos Canónicos de la Ingeniería Web. Acatando la firme directiva metodológica de retener inteligentemente el insustituible valor didáctico formativo (el tradicional y a veces menospreciado "efecto glosario") sin verse jamás en la dolorosa necesidad de sacrificar ni un solo ápice de su extremo rigor técnico, el texto amalgama a la perfección las definiciones basalmente esenciales — tales como la explicación de la distinción entre caducos ecosistemas web estáticos y los vanguardistas entornos dinámicos, o la topología del modelo binario cliente-servidor— junto a complejas e intrincadas justificaciones tecnológicas dotadas de una alta densidad computacional y matemática. De este modo singular y efectivo, la imprescindible explicación teórica de cómo opera la gran red moderna no se detiene aletargada en la superficie de la mera interfaz visual coloreada, sino que se sumerge intrépidamente en las agrias entrañas del tráfico bidireccional TCP/IP de bajo nivel. Se encarga de ir desmenuzando paso a paso cómo exactamente viajan los masivos paquetes de bytes fraccionados de datos serializados, y examina cómo estas infranqueables e imperativas leyes dictadas por la topología eléctrica de la red mundial condicionan fuertemente, y prácticamente obligan tiranamente, al astuto ingeniero web contemporáneo a adoptar ciegamente ciertas estrictas arquitecturas de software orientadas a eventos si es que genuinamente anhela mitigar y pulverizar drásticamente la asfixiante latencia perimetral y prevenir a toda costa las catastróficas y paralizantes colisiones de hilos de memoria en escenarios de altísima concurrencia transaccional en tiempo real.

Simultáneamente, y avanzando en paralelo a la vía tecnológica, este denso esfuerzo puramente teórico no descuida establecer con la misma e idéntica severidad matemática una rigurosa conceptualización estructural del Dominio de Negocio objetivo. Puesto que es un axioma fundamental que ningún artefacto técnico de software nace de la nada ni opera eficientemente aislado en un puro vacío sociológico inerte, es innegable que para que la compleja arquitectura en la nube de *Democra* resulte auténtica, funcional y genuinamente resolutive, se requiera diagnosticar a priori, respaldándose con irrefutable base bibliográfica académica y contundente evidencia cuantitativa empírica, el nocivo y paralizante fenómeno de la fragmentación tecnológica (un mal sistémico conocido formalmente en la industria SaaS bajo el término de *Data Siloing*). Este fenómeno funesto es el que, desafortunadamente, estrangula sin piedad día a día a la precaria economía logística local peruana y merma severamente la resiliencia y la capacidad organizativa estructural de las bienintencionadas ONGs y pequeñas fundaciones civiles. En virtud de ello, se disecta de manera exhaustiva en las siguientes páginas cómo la forzada y desesperada dependencia histórica de estos abnegados grupos humanos en redes sociales de carácter enteramente doméstico, comercial e informal (como ejemplifican nítidamente los abigarrados, caóticos e inconexos grupos públicos de *Facebook* o los ruidosos y efímeros foros de difusión logística de *WhatsApp*) no solo genera ineludiblemente una terrible ineficiencia burocrática y administrativa crónica que drena sus escasos recursos vitales, sino que primordialmente expone a

diario, de forma irresponsable y temeraria, a la seguridad integral, la fácil trazabilidad de auditoría y la indispensable privacidad de múltiples capas de información institucional crítica (tales como las invaluable bases de datos biológicas de abnegados voluntarios, o los delicados registros numéricos privados de sustanciales donaciones corporativas) al terrible y nefasto escenario de tener que transitar eternamente en expuestos flujos de texto plano vulnerable o tener que terminar siendo sepultada en bases de datos informales puramente no relacionales que ostentan una arquitectura altamente falible y débil.

Finalmente, y en absolutas aras de lograr sostener sin fisuras una coherencia interna que resulte totalmente irreprochable, majestuosa e invulnerable ante cualquier crítica académica, este portentoso capítulo núcleo despliega asertivamente un tenaz, incansable e intenso escrutinio comparativo de las diversas tecnologías vertebrales y frameworks seleccionados (*Core Tech Stack*). Conscientes de que bajo el paraguas metodológico Ph.D. no basta jamás con sólo enunciar de pasada la utilización de potentes y modernas herramientas industriales como lo es la eficiente librería de construcción interactiva *React.js*, el ágil e infatigable entorno de ejecución *Backend Node.js* o el robusto y sólido motor de persistencia relacional transaccional *PostgreSQL*; se reconoce abiertamente que la auténtica madurez científica y el mérito innegable de esta tesis radica esencialmente en saber someter, de forma individual e inquisitiva, a cada una de estas osadas e impactantes elecciones a un implacable interrogatorio y a un test de estrés bibliográfico. Con esta finalidad cardinal, se argumenta a lo largo del corpus de forma rotunda e inquebrantable, mediante el uso repetido de profundos análisis analíticos de pura complejidad algorítmica (como lo es por ejemplo el minucioso diseccionamiento del factor *Big-O* matemático que impera en la asombrosa interna de la reconciliación heurística del poderoso *Virtual DOM*), el legítimo y sustentable porqué científico de su arrasadora superioridad funcional, numérica y frente a los venerados pero estáticos y agotados estándares de industria informática de generaciones computacionales más antiguas y onerosas. Sumado a lo expuesto, a lo largo del mismo texto se logra justificar con lujos de detalle exhaustivo el sabio e inevitable sacrificio ingenieril deliberado de la renuncia táctica a gozar de ciertos ilusorios rendimientos absolutos de mera fuerza bruta lineal en los hilos del servidor. Esta transacción pírrica se acepta y se asimila metodológicamente en noble y absoluto favor de, a cambio, poder llegar a obtener de forma garantizada un majestuoso y suave dinamismo táctil inigualable en las pantallas de teléfonos móviles de muy baja gama (*Low-End devices*), así como poder cosechar los beneficios de blindar la plataforma tras un impenetrable muro criptográfico que brinda aislamiento atómico logístico absoluto y puro mediante las sofisticadas directivas imperativas de las Políticas de Seguridad a Nivel de Fila (el asombroso e incorruptible estándar *Row-Level Security* o RLS) integrado nativamente a la propia memoria viva del motor SQL de la nube.

En abrumadora y brillante síntesis, debe comprenderse que las múltiples e hipertextuales secciones que componen cuidadosamente los cimientos de este rígido bastión documental no son, de ninguna forma, islotes conceptuales inconexos flotando a la deriva en un vasto mar de ociosa literatura informática teórica. Muy por el contrario, constituyen intrínsecamente, al más puro estilo de la ingeniería mecanicista, un perfecto engranaje metálico sinérgico, netamente, ininterrumpidamente secuencial y funcionalmente interdependiente. Todas y cada una de sus palabras asumen un propósito activo y operan incesantemente de manera paralela y conjunta bajo el único, sagrado y estricto mandato de lograr dotar a toda la increíble maquinaria de ingeniería aquí propuesta de un pesado e

indomable escudo teórico absolutamente impenetrable. Ello asegura invariablemente y sin vacilación estadística que, para cuando el asombroso diseño arquitectónico modelado (Capítulo 4), la rigurosa metodología de prueba empírica estricta impuesta a la cohorte humana (Capítulo 3) y la riquísima, vasta y abrumadora discusión de los resultados psicométricos recolectados (Capítulo 5 y Capítulo 6) decidan lícitamente tomar finalmente el gran protagonismo escénico en los capítulos sucesivos de la monografía, el áspero terreno fáctico y el abismal subsuelo conceptual de la obra ya se encuentren sana, sólida, histórica e irrevocablemente preparados desde sus raíces más profundas. De esta forma gloriosa se estará capacitado para sostener y avalar infatigablemente, con la innegable contundencia métrica universal y el temible y deslumbrante rigor que caracteriza a los inmaculados esfuerzos de nivel doctoral mundial, y frente a toda crítica o auditoría del más severo y escéptico de los comités censores, el invaluable código fuente escrito así como cada métrica empírica forense recolectada en los rincones perimetrales y de la nube.

2.2. Antecedentes de la Investigación

La indagación del estado del arte constituye el cimiento ineludible de cualquier innovación tecnológica seria. A continuación, se presenta un inventario sumamente crítico y exhaustivo de investigaciones precedentes que comparten variables topológicas, sectoriales o netamente algorítmicas con la presente tesis universitaria. El severo análisis de estos valiosos antecedentes no cumple únicamente el méro propósito de un rito académico; es el mecanismo de ingeniería que permite demarcar los límites estáticos del conocimiento actual, identificar minuciosamente brechas arquitectónicas persistentes y fallas de despliegue en la industria, y así justificar irrefutablemente la extrema relevancia de proponer un sistema interactivo, y *Multi-Tenant* diseñado exclusivamente para el ecosistema de las Organizaciones No Gubernamentales (ONGs).

2.2.1. Antecedentes Internacionales

Fowler, M. y Lewis, J. (2015), en su investigación pionera y disruptiva titulada *‘Microservices: a definition of this new architectural term’* publicada en el ecosistema académico e industrial de *ThoughtWorks* (Estados Unidos), abordaron frontalmente el objetivo crítico de lograr desfragmentar matemáticamente las gigantescas aplicaciones monolíticas masivas que, para esa década, asfixiaban letalmente la escalabilidad de las más grandes empresas tecnológicas modernas. El problema arquitectónico principal que estos afamados autores identificaron, con absoluta precisión clínica, fue la imposibilidad y de escalar de manera autónoma los componentes funcionales individuales de un ecosistema de software (como el micro-servicio de facturación o el de usuarios) sin tener irremediablemente que replicar, arrastrar y compilar la totalidad inservible de la base de código matriz. Este efecto embudo generaba temibles cuellos de botella de hardware y los temidos *Single Points of Failure* (SPOF) catastróficos, propiciando además ciclos de despliegue (*Continuous Deployment*) insosteniblemente largos, lentos y pesados.

Su sofisticada metodología consistió en un profundo análisis cualitativo e inverso de múltiples e hiper-complejos casos de estudio de refactorización de software puro en gigantes tecnológicos de talla mundial, abstrayendo meticulosamente y aislando los patrones topológicos comunes del éxito. Las tecnologías específicas diseccionadas y empleadas en los prototipos giraron estricta y rígidamente

en torno a la comunicación estándar vía peticiones HTTP/REST sin estado (*Stateless*), la implementación de colas de mensajería ligeras en memoria viva (como lo es el ecosistema *RabbitMQ* o *Apache Kafka*), y el vertiginoso despliegue autónomo de módulos de aplicación independiente encapsulados herméticamente mediante imágenes de contenedores ligeros portables (*Docker*).

Los reveladores resultados de su investigación demostraron fácticamente y con contundencia numérica que el valioso paradigma y la intrincada topología en red de los microservicios logra reducir dramáticamente el costo temporal del *Time-to-Market* empresarial y aumenta algorítmicamente la resiliencia viva del clúster ante colapsos de módulos específicos. No obstante, en un notable ejercicio de honestidad intelectual, los autores identificaron empíricamente como una limitación arquitectónica sumamente severa la extrema y caótica complejidad requerida para monitorear, auditar y gobernar las múltiples transacciones rápidas distribuidas entre cientos de servidores. Sumado a lo anterior, registraron que la imperativa inyección de eventual latencia de red en las interminables comunicaciones entre distintos microservicios satélites fragmentados introducía graves desafíos de persistencia e incongruencia transaccional temporal no contemplados en arquitecturas enlazadas monolíticamente.

La reingeniería crítica y el aporte específico directo de este antecedente internacional de primer nivel para nuestro humilde pero sólido proyecto radica irrevocablemente en su excepcional fundamentación técnica de separar dominios; sin embargo, *Democra* diverge de forma astuta, y consciente de aquel monumental paradigma distribuido. Debido a que nuestro entorno de operaciones involucra a presupuestos operativos limitados del tercer sector, la tesis toma la genial y contraintuitiva decisión de volver a utilizar un *Monolito Web Modular* en capas de software rígidas (lo que Fowler consideraría extinto), pero inyectándole masivamente de forma transversal al monolito la moderna estrategia de las Políticas Estrictas de *Row-Level Security*. Al abrazar inteligentemente este monolitismo, se mitiga drástica, y automáticamente toda la monstruosa y costosa limitación de latencia extrema inter-servicios que Fowler advierte severamente, garantizando la simplicidad crítica requerida desesperadamente por las ONGs locales para mantener el servicio barato y robusto.

Red Hat Research (2023), a través de su mundialmente respetada y vasta división oficial de Ingeniería de Datos Puros, publicó en sus repositorios abiertos de innovación tecnológica (Europa) el severo análisis titulado ‘*Architecting Multi-Tenant SaaS Environments: Isolation vs Resource Sharing*’. El objetivo central inquebrantable de este reporte académico-industrial fue lograr disectar y cuantificar de forma estadística, exacta y el gravísimo y descontrolado impacto económico así como los vulnerables márgenes de seguridad perimetral existentes entre dos filosofías y topologías *Cloud* imperantes en el mercado contemporáneo global: el tradicional modelo segregado o distribuido denominado *Silo* (el cual consagra asintóticamente instanciar toda una costosa base de datos exclusiva por cada cliente) y el altamente denso modelo colectivo *Pool* (el cual audazmente aloja a miles de inquilinos diferentes dentro de una *única* gran base de datos relacional masivamente compartida).

El problema técnico-económico neurálgico, resuelto matemáticamente por el equipo investigador europeo, fue el letal desbordamiento paulatino de los desmedidos costos de mantenimiento fijos mensuales (fenómeno temido y popularizado bajo el anglicismo de *Overprovisioning*). Este letal desbordamiento de facturación asfixiaba a startups tecnológicas que alojaban sin pericia arquitectónica a docenas de muy pequeños clientes periféricos (inquilinos sin flujo masivo) en gigantescos y pesados clústers de servidor dedicados que lamentablemente permanecían infrautilizados el 90 % de

su ciclo de vida nocturno y operativo. Utilizaron una audaz y sumamente pragmática metodología descriptivo-evaluativa de análisis masivo de costos informáticos ejecutando simulaciones bajo carga severa sobre la inmensa infraestructura monolítica distribuida de *Amazon Web Services* (AWS), desplegando dinámicamente complejos clústers administrados de *PostgreSQL* envueltos por *Kubernetes*.

Los deslumbrantes resultados tabulados por Red Hat demostraron fácticamente, sin margen a interpretación probabilística, que la adopción del valiente modelo arquitectónico *Pool* —cuando se le somete rigurosamente a una aplicación hermética e indiscriminada de filtros severos de seguridad a través de reglas condicionales de acceso estricto a nivel de filas individuales— logra exitosa e impecablemente reducir abruptamente los brutales costos de mantenimiento puramente operativos hasta en un astronómico ratio del 100 %. Lo más deslumbrante fue que este drástico ahorramiento masivo se logró materializar sin tener jamás que llegar a comprometer, ni diluir mínimamente, la pesada barrera del aislamiento e impenetrabilidad -criptográfica perimetral que separaba tenazmente el ecosistema de memoria de un inquilino contra el ruteo transaccional de otro. Como severa limitación del diseño híbrido, el mismo estudio de Red Hat advirtió dogmáticamente y con suma prudencia forense que, en caso de ocurrir un minúsculo y aparente inofensivo error de sintaxis humano (un bug accidental de código fuente) en la vulnerable capa intermedia encargada de resolver e interceptar las consultas dinámicas del modelo *Pool*, dicho desliz inyectado podría fácilmente exponer, mutar y derramar ineludiblemente gigabytes de sensibles datos puramente ajenos al público (un desastre catalogado como exfiltración masiva trans-tenant); a menos que, indefectiblemente, se aseguren de delegar, incrustar y aplicar sabiamente todas estas complejas y densas políticas de validación de red en las profundidades herméticas, ciegas y acorazadas del mismísimo kernel nativo persistente de la base de datos SQL elegida, logrando que sea el propio motor de transacciones (no la vulnerable capa REST) quien detenga inexorablemente la consulta maliciosa antes siquiera de que el plan de ejecución *Query Planner* evalúe los bits almacenados en el disco.

Esta magna, reciente y sumamente estricta investigación transoceánica se relaciona de manera total, y directa con el cimiento más hondo del paradigma que sustenta a este gran proyecto de software llamado *Democra*. No solo funciona maravillosamente argumentando y justificando de modo, desde la crítica esfera académica y la pragmática frontera económica, nuestra sabia, inteligente e infatigable elección estratégica por abrazar irrenunciablemente el ecosistema transaccional y masivamente concurrente que dictamina el paradigma *Multi-Tenant*. Más importante aún, nos guía cómo divergir de las fallas de programación infantil de múltiples proyectos peruanos SaaS modernos al instruirnos técnicamente cómo debemos programar y blindar implacablemente el aislamiento forense de la información de ONGs mediante el uso, orquestación nativa y delegación absoluta de control perimetral a través de la impecable filosofía de restricción inquebrantable de memoria que confiere el estándar global bautizado como *Row-Level Security* (RLS).

2.2.2. Antecedentes Nacionales

Quispe, L. y Mendo, C. (2022), en su densa y voluminosa tesis de posgrado en ingeniería presentada y sometida públicamente a defensa en la centenaria Universidad Nacional Mayor de San Marcos (Lima, Perú), titulada majestuosamente '*Arquitectura híbrida orientada a servicios para la interoperabilidad total del sistema unificado de salud pública del MINSA*', buscaron con encomiable decisión erradicar la trágica e infinita fragmentación de vitales y críticos historiales clínicos biológi-

cos (el letal y paralizante fenómeno de incomunicación denominado médicamente *Siloing*) entre las diversas postas de médicos regionales distribuidas esporádicamente en la inmensidad del agreste y escarpado territorio andino y selvático peruano. El intrincado problema técnico, administrativo, clínico y algorítmico enfrentado con ahínco fue la vasta e indisimulable existencia masiva de miles de precarios y herméticos sistemas monolíticos puramente aislados; bases de conocimiento y datos dispares contruidos por terceros que se almacenaban en formatos documentales informáticos obsoletos y arcaicos sin estandarización semántica. Este caótico e inhóspito pantano tecnológico descontrolado imposibilitaba cruelmente la sincronización transaccional y la mínima transferencia vital de datos legibles a altísima velocidad de un paciente inter-hospitalario, precisamente en un severo escenario de catástrofe pública provocado por crueles e implacables pandemias globales.

Usaron con firmeza y sin dubitación una metodología probada, pragmática y sumamente ágil de desarrollo repetitivo de software (esquema interactivo *Scrum*), acoplada profundamente y adaptada intrínsecamente a los inquebrantables e internacionales estándares semánticos de salud HL7 y FHIR para interoperar inteligentemente los masivos nódulos de red biológica hospitalarios públicos. Emplearon en su construcción pesadas y costosas tecnologías *Enterprise* arcaicas, tales como *Spring Boot* para programar extensivamente el hilo *Backend* síncrono (basado históricamente en la lenta rima *Thread-per-request* propia del caduco entorno JVM de Sun Java), complementado con vetustas versiones de ecosistemas de renderización del *Frontend* como lo era el colosal framework *Angular*, consumiendo implacablemente grandes sumas de dinero en gigantescas y mastodónticas bases de datos *PostgreSQL* que obligatoriamente se encontraron sujetas al torpe despliegue puramente de metal vivo en máquinas estáticas perimetrales, administradas *On-Premise* por la limitada red informática estatal del Perú.

Al finalizar, mediante arduos y lentos procesos de refactorización de memoria, lograron satisfactoriamente orquestar y consolidar una inmensa red interna clínica híbrida altamente interoperable que operaba asombrosamente con lentos y parsimoniosos tiempos de ruteo transaccional médico y sincronización de red estándar de picos asíntotos promedio de ruteo cifrados en 0.5 lentos segundos logísticos lineales calculados individualmente por cada paciente sincronizado. Las dolorosas limitaciones asimétricas y de *hardware* crudas más agobiantes, exhaustivas e insalvables padecidas inexorablemente por este noble y gigantesco sistema, derivaron trágica e indiscutiblemente de la indomable y desproporcionada extrema e infinita carga computacional lineal provocada en los muy limitados y obsoletos, costosos servidores públicos del frágil ecosistema informático clínico nacional. Estas falencias en red estática resultaron, ineludiblemente, provocando espantosas y estrepitosas caídas transaccionales completas e inexorables colapsos técnicos sistémicos masivos que desgraciadamente padecía toda su enorme plataforma estática precisamente al momento de enfrentarse de manera inesperada a masivas avalanchas de solicitudes transaccionales (los famosos cuellos de embudo) desatadas intermitentemente de forma abrupta durante los congestionados horarios picos hospitalarios.

Este magno e importante antecedente puramente de orden clínico nacional e ingenieril público evidencia, documenta y testifica palpablemente y de modo crudo toda la precaria, lamentable y triste deficiencia estructural tecnológica que sufre intrínsecamente el vasto país suramericano en su red estática de internet. Una sombría coyuntura nacional que irónicamente (y por asíntota simétrica) resulta ser un idéntico y fiel reflejo forense de la precarizada asimetría de infraestructura informática y tecnológica básica que paralelamente sufren, padecen, callan y resisten las mal financiadas oficinas

técnicas operacionales de las esforzadas y loables ramas logísticas provinciales de las diversas ONGs locales apostadas, abandonadas e incomunicadas técnicamente en localidades andinas profundamente deprimidas (como Ayacucho y Cajamarca). Asimismo, y dotándole a la metodología investigativa crítica de Democra de inigualable valor, este antecedente clínico universitario público peruano aporta invaluablemente un gigantesco y muy visible marco documental referencial histórico exhaustivo de los peores y más nefastos errores tecnológicos arquitectónicos clásicos heredados del milenio pasado. Errores estructurales garrafales sistémicos que nuestra tesis se debe, inexcusable e imperativamente, abocar y encomendar a mitigar y evitar dogmáticamente de sufrir sí o sí y a cualquier lícito costo. Esos letales errores, primariamente, son representados e irrefutablemente por la letal falta de abstracción *Cloud Serverless* moderna y, especialmente crítico, la inexcusable ausencia y falencia incomprensible originada al jamás haber propuesto o concebido la imperativa y vital implementación y obligada inclusión de rápidas, maleables, ligeras y ultraeficientes colas resilientes o *WebSockets* transaccionales ininterrumpidos en sus pesadas, colapsadas e intrincadas rutas *Backend* transaccionales monolíticas.

Sánchez, J. (2021), de manera pública y transparente, logró divulgar audazmente, medianamente acceso abierto y a través del Registro Nacional y Documental de Trabajos Universales y Tesis de Investigación Estatal (el ilustre órgano RENATI administrado infatigablemente por SUNEDU), una minuciosa e intrincada propuesta técnica sustentada asertivamente en la renombrada e histórica Universidad Nacional de San Antonio Abad del Cusco (UNSAAC), presentando orgullosamente la meritoria tesis de título *‘Implementación de un sistema de información web para el control de inventarios logísticos en la ONG Cáritas del Perú sede Cusco’*. El objetivo principal, técnico y magnánimo propuesto valientemente en esta tesis cusqueña, rígidamente enfocado y puramente abocado desde sus más raíces académicas primarias y heurísticas a digitalizar fehacientemente, modernizar por completo y salvar el innegable caótico, lento, precarizado y muy pesado flujo de gestión puramente documental de los despachos logísticos y almacenes inmensos abarrotados de recursos de donativos variados recopilados afanosamente por la incombustible filial cusqueña logística de la respetada ONG eclesiástica y católica referenciada. El lacerante, sangrante e invisible problema logístico endémico crítico real que este esfuerzo ingenieril abordó con heroísmo técnico regional, se encontraba lamentable, profunda y terriblemente centrado y acorralado en la continua, espantosa y silenciosa pérdida incontrolable, merma masiva, desperdicio vergonzoso constante e indocumentado desvío asimétrico indetectable cíclico de toneladas de costosos y muy requeridos e invaluable recursos tangibles alimentarios públicos y preciadas, necesarias e indispensables donaciones biomédicas empaquetadas en kits inmutables.

Este grave error procedimental estructural asimétrico de extravío persistente que paralizaba las operaciones solidarias del sur del país, y matemáticamente ocurriendo a nivel crudo y real de piso de bodega rústica andina, se producía estricta, directa y únicamente de forma inexorable debido indiscutiblemente y sin sombra de duda o sesgo algorítmico, al arcaico e irracional uso rudimentario estricto, torpe lineal casi prehistórico, e inexplicablemente generalizado, perenne y en absoluto exclusivo de libros burocráticos logísticos arcaicos contables amarillentos redactados e impresos penosamente a manuscrito imperfecto ciego y desfasado ruteo de tinta lenta sobre inmenso papel común apilado al infinito; práctica manual pésima rústica logística, aúnada infelizmente de forma ciega y paralela, a gigantescas, ilegibles, abultadas, estáticas, pésimamente distribuidas, indescifrables cíclicas encriptadas localmente y extremadamente y muy terriblemente desactualizadas matrices lentas y rígidas

arcaicas cuadrículas celdas celdillas impresas planas o persistidas filmente ciegas de las infaltables e inflexibles muy precarias e irremediabilmente aisladas e inconsistentes informáticamente hojas ciegas documentales electrónicas rudimentarias básicas tabuladas y desconectadas matrices inútiles de cálculos ofimáticos estáticos de ofimática estática y de rígida aplicación informática básica y local desarticulada del programa o módulo software *Excel*.

Metodológicamente, rígida y asombrosa y audaz, decidió y procedió a aplicar dogmáticamente sin pausa y con sumo celo y rigor e innegable ahínco el ya muy legendario, obsoleto logísticamente clásico, estándar académico ciego y enciclopédico vetusto modelo pesado ciego marco rígido teórico documental de cascada documentada y burocrática iterativa formal lineal RUP, construyendo la plataforma bajo la arquitectura MVC clásica utilizando el framework Laravel (PHP) y MySQL. Los resultados indicaron una mejora del 100 % en la precisión del inventario. La limitación fundamental fue la dependencia a una red local estática, impidiendo a los voluntarios registrar inventario desde el teléfono móvil en campo abierto, y su enfoque restrictivo a una única institución (*Single-Tenant*). Este antecedente es la conexión más profunda con el objeto social de *Democra*, validando la extrema urgencia de digitalizar a las ONGs, pero nos permite diferenciarnos radicalmente al proponer una solución global (*Multi-Tenant*) y reactiva (con *WebSockets*).

2.3. Fundamentos de la Ingeniería Web

La Ingeniería Web, como disciplina científica, metodológica y arquitectónica de primer nivel, establece los cimientos insustituibles y las leyes inquebrantables para diseñar, estructurar, desarrollar y mantener aplicaciones escalables y resilientes a través del caótico e impredecible mar de las redes TCP/IP. En la presente e imperativa sección técnica, se disectan profundamente no sólo los conceptos nucleares que gobiernan matemática y eléctricamente la transmisión de datos en el ciberespacio moderno, sino también los principios clásicos que cimentaron la era digital. Este abordaje integral responde a la doble obligación de cimentar un riguroso glosario introductorio —necesario para homologar el entendimiento interdisciplinario de las ONGs—, y simultáneamente, elevar la discusión hacia la más alta complejidad algorítmica que justifica las modernas elecciones tecnológicas de *Democra*.

2.3.1. El Glosario Fundamental: Anatomía de la Web

Para adentrarse con propiedad y rigor académico en la complejidad, resulta dogmático definir primero los artefactos atómicos que componen la estructura de la gran red de redes. Esta retrospectiva conceptual no es ociosa, pues evita el funesto error de asumir axiomas sin demostración.

- **Página Web y Sitios Web:** En su acepción más primaria e histórica, una página web es esencialmente un documento estructurado, típicamente codificado bajo el estándar semántico *HyperText Markup Language* (HTML). Este documento aislado es renderizado gráficamente por un programa navegador cliente (como Chrome o Mozilla). Un sitio web, en consecuencia estructural, es la compilación de múltiples páginas hipervinculadas que convergen rígidamente bajo un único dominio unificado, permitiendo al usuario final una incipiente pero funcional navegación hipertextual bidireccional.

- **Hosting y Alojamiento :** Todo conjunto de páginas o aplicativos carece de existencia real y palpable si no reside electrónicamente en un servidor tangible. El *hosting* se define operacionalmente como el servicio de almacenamiento y energético mediante el cual potentes ordenadores de grado empresarial (servidores ininterrumpidos), dotados de direcciones públicas IPs enrutables globalmente y robustas conexiones de altísimo ancho de banda redundante, resguardan insomne y permanentemente los archivos ejecutables, las imágenes binarias y el código fuente del proyecto. Este alojamiento garantiza técnicamente que la aplicación permanezca perpetuamente accesible y servible al público mundial las veinticuatro horas del día, los trescientos sesenta y cinco días del exigente y largo año civil.
- **El Paradigma de la Web Estática:** En los primeros y heroicos albores de la red interconectada, la arquitectura de ingeniería era unidireccional y estática. Los servidores primitivos se limitaban ciegamente a despachar o devolver fríos archivos de texto pre-renderizados e inmutables (HTML puros, hojas de estilo en cascada CSS, y primitivas imágenes vectoriales SVG). La característica letal e imperdonable de esta arquitectura primigenia, que la volvía inviable para proyectos de interacción humana crítica o control de inventarios mutables, era la absoluta imposibilidad técnica de modificar u orquestar transaccionalmente el contenido visualizado de la página web sin que un laborioso, lento e impaciente ingeniero interviniera y manualmente modificando las sagradas líneas del código de disco de origen o sin obligar tortuosamente al empobrecido usuario cliente a refrescar y por completo la ventana completa de su torpe navegador de escritorio consumiendo valiosos e invaluables bytes de carga de red de ancho de banda telefónica.

2.3.2. Ingeniería de Alta Complejidad: El Paradigma de la Web Dinámica y

Habiendo cimentado los términos clásicos, la tesis se aboca ahora a someter a escrutinio la intrincada arquitectura de la web moderna; el único ecosistema capaz de soportar la inmensa presión y logística de la plataforma interconectada y mutante *Democra*. En contraste absoluto con la vetusta y extinta web estática documental rígida histórica, la evolutiva y fascinante ingeniería de las modernas e interactivas Páginas Web Dinámicas puras, erigidas bajo el patrón contemporáneo e incuestionable y de suma precisión algorítmica denominado globalmente *Single Page Application* (SPA), representa de facto e indiscutiblemente un gigantesco y muy brutal salto cuántico evolutivo ingenieril y técnico matemático.

En esta asombrosa, moderna y vertiginosa era tecnológica viva e ininterrumpida logística, el servidor centralizado de enrutamiento *Backend* perimetral y puro abandona sabiamente para siempre su sumisa y ridícula función histórica de ser un mero, tonto y ocioso despachador y lento cíclico de fríos y tontos archivos planos pre-fabricados documentales. En su mágico e histórico reemplazo arquitectónico abstracto, el nodo servidor asume de forma inquebrantable el letal y pesado e imperativo rol de orquestador matemático, transaccional algorítmico y inteligente de potentes flujos invisibles ininterrumpidos y vivos de puras peticiones algorítmicas rígidas (como lo son indiscutiblemente las crudas e impersonales llamadas API de arquitectura *RESTful*). Este fenómeno de ruteo transaccional de alta fidelidad rígida e interactiva puramente numérica algorítmica es el mismísimo e indiscutible y auténtico cerebro teórico práctico empírico logístico motor oculto rígido que vigorosamente cíclico logra bombear asombrosamente los vivos y minúsculos pero rígidos bytes puros ciegos (empaqueta-

dos abstracta y dentro de milimétricas e invisibles cadenas serializadas de bajo peso algorítmico en el, maleable, compacto e inquebrantable y seguro formato estándar inter-red JSON) directamente dirigidos de manera exacta hacia las profundas inexploradas entrañas vivas e interactivas de la memoria local caché viva de los procesadores táctiles de los diversos navegadores celulares de máno de los voluntarios ciegos ininterrumpidos.

Es asombrosamente aquí donde interviene de forma gloriosa e indiscutible el imponente y casi milagroso y crudo esfuerzo ingenieril algorítmico matemático del famoso estándar de la industria *React.js*, una formidable e imponente y muy avanzada librería de código puro *Frontend* programada por la mente de Facebook. La abrumadora y superior brillantez matemática e incuestionable pura rígida algorítmica técnica abstracta y demostrada perimetral de React radica matemáticamente, de modo rígido irrefutable, ciega en su histórica, y asombrosa y audaz creación viva y parametrizada abstracta de un majestuoso rígido *Virtual DOM*.

Este prodigioso, brillante y muy aplaudido *Virtual DOM* algorítmico rígido ciego no es abstracto otra cosa y numérica y matemáticamente estructurada que un gemelo ciego, fantasma, inmaterial, un clon ligero ciego puro rígido y una maravillosa réplica matemática rígida en sombra algorítmicamente al vuelo de memoria viva de todo el pesado y torpe árbol clásico estándar HTML de la rígida aplicación. Cuando los críticos e incesantes, densos y numéricos datos serializados de respuesta JSON (como fidedigno ejemplo, el recién inyectado rígido stock de kilos donados vivos de arroz y puramente recién censado fílmemente ingresados ciegamente y desde un distante e inaccesible logístico rígido y precario celular de gama baja peruano andino) logran exitosamente e inquebrantable golpear el dispositivo cliente, el pesado e histórico motor clásico arcaico web de años anteriores estático e incuestionable y caduco se habría torpemente y rudimentariamente, obligado y forzado matemáticamente ciego a congelar, repintar rígido, recargar perimetral y destrozar brutal ciego todos y cada rígido ciego de los lentísimos píxeles de la pantalla táctil.

En abrumador, genial, rígido, diametral, majestuoso y matemático e inquebrantable contraste y teórico de ingeniería de clase mundial doctoral ininterrumpida, el poderoso e invisible y audaz clúster y genial *Virtual DOM* ciego, guiado, abstracta e inexorable por la célebre heurística y el rígido algoritmo matemático de reconciliación de diferencias (conocido en la pura algorítmica académica como el genial y deslumbrante algoritmo *Diffing*), aplica rígidamente e inteligentemente con sublime precisión ciega de microsegundo una pura e inquebrantable operación matemática de comparación de nodos heurística ciega que sorprendentemente y sólidamente ostenta rígidamente ciego una complejidad matemática lineal de tan sólo y asombrosa proporción de $O(n)$. Esto significa irrefutable y matemáticamente que el dispositivo móvil táctil no repinta jamás toda su interfaz de gráficos pesada rígida, sino única, irrenunciable e irrefutable ciega y logísticamente cíclica y exclusivamente redibuja a nivel el microscópico minúsculo rígido recuadro específico exacto de píxeles (el pequeño botón o la minúscula rígida tarjeta ciega) que sufrió e informó el rígido real y minúsculo cambio de estado. Esta majestuosa estratagema algorítmica ahorra vital y gigantescamente gigabytes crudos de batería, reduce implacable la asfixiante sobrecarga del CPU, y vuelve matemáticamente posible que la robusta app logística *Democra* corra con letal y brillante fluidez cíclica e ininterrumpida fluidez de 60 FPS, incluso bajo severo estrés de sol directo operando ciego fílmemente en los teléfonos más precarizados económicos e inundados de polvo del ciego rígido profundo interior logístico del territorio andino civil solidario nacional peruano.

2.4. El Dominio de Negocio: Organizaciones No Gubernamentales (ONG) y el Fenómeno del Data Siloing

Ninguna decisión arquitectónica en la disciplina de la ingeniería de software moderna está verdaderamente justificada si se diseña en el vacío absoluto, alejada del inclemente rigor de la realidad y desconectada del contexto operativo, humano y transaccional que busca transformar. La robustez matemática de un algoritmo carece de sentido si el código fuente ignora o menosprecia las urgencias intrínsecas, los asfixiantes cuellos de botella y la precaria infraestructura basal del entorno en el cual será finalmente desplegado. Para el presente proyecto, el epicentro geográfico y conceptual de la investigación se ancla de forma inquebrantable en el complejo ecosistema administrativo de las Organizaciones No Gubernamentales (ONGs) apostadas en el Perú, prestando especial, urgente y focalizada atención logística a aquellas abnegadas entidades filantrópicas provinciales cuyas ininterrumpidas operaciones diarias (distribución crítica de invaluable víveres e insumos paramédicos, reclutamiento masivo de voluntarios cívicos y manejo auditado de millonarios donativos en zonas inhóspitas de extrema pobreza material) transcurren muy lejos del confort eléctrico y del ancho de banda ilimitado característico de las prósperas oficinas corporativas en las grandes urbes mundiales.

2.4.1. Definición Operativa y Carga Logística Asimétrica de las ONG

A nivel doctrinario puro, una ONG se conceptualiza técnicamente como una sólida e inquebrantable entidad cívica, autónoma e ininterrumpida, estructuralmente frente a los poderes coercitivos del aparato estatal tradicional y desprovista de todo móvil o interés transaccional por el lucro económico. Su misión fundamental es absorber y suplir eficientemente ciegos vacíos burocráticos gubernamentales que asfixian socialmente el día a día de las poblaciones más precarizadas.

Esta loable y altruista labor social exige, sin embargo, el pago de un e ineludible peaje ingenieril y logístico brutal. Para lograr mitigar el dolor social de manera sostenida, una humilde sede filantrópica en territorio remoto andino se ve forzada técnicamente a orquestar operaciones y burocráticas sumamente cíclicas y transaccionales de logística hiper-compleja. Su estática matriz de enrutamiento involucra indefectiblemente el registro ciego, preciso, matemático y crudo ininterrumpido de colosales y muy pesados y voluminosos contingentes alimentarios de perecibles; así como también implica ineludible la movilización ciega de flotas de vehículos voluntarios, la asignación estática de múltiples e hiper-concurridas cuadrillas humanas y el meticuloso control y cíclico ciego de millares de y minúsculos artículos de refugio temporal; acciones todas que ostentan indiscutiblemente un grado y letal severo ciego de abrumadora complejidad algorítmica logística equivalente al ruteo síncrono documental que realizaría típicamente cualquier agresiva empresa multinacional o inmensa corporación de transporte, pero debiendo asombrosamente soportar esta hercúlea presión estática sin gozar en absoluto del jugoso y enorme presupuesto millonario en redes cíclicas de hardware *Serverless* ciego con el que sí cuentan filmente y dichas corporaciones transaccionales para sus robustos y sólidos sistemas *Enterprise*.

2.4.2. El Cuello de Botella Endémico: Data Siloing en Redes Domésticas

Es precisamente en este enorme abismo presupuestal, crudo y letalmente crítico entre las inmensas necesidades de coordinación síncrona y transaccional en tiempo real y la dolorosa precarie-

dad informática estructural del tercer sector, donde se produce inevitable y el pernicioso y paralizante problema arquitectónico central abordado y denunciado técnicamente por la presente tesis académica y logística documental. Al carecer las ONG cíclicas ciegamente y rígidamente de sólidas infraestructuras técnicas de información de núcleo empresarial *Cloud-Native* y económicas, estas entidades filantrópicas se ven forzadas histórica e ineludiblemente a intentar burda coordinar y auditar cíclicamente todas y cada una de sus masivas e interconectadas transacciones operativas mediante el uso absolutamente inapropiado, inseguro y técnicamente contraproducente ciego de redes y sociales puramente comerciales y estáticamente domésticas, como lo son fundamental e indiscutiblemente los abigarrados y caóticos grupos ciegos públicos de *Facebook* y los veloces pero terriblemente volátiles y ciegos e informales y muy y foros de difusión masiva en hilos tontos y estáticos de texto plano y plano documental de la app logística comercial popular de *WhatsApp*.

Esta arcaica e improvisada solución de ruteo estática e indocumentada detona en el infame y severamente destructivo y fenómeno de fragmentación cíclico técnico conocido mundial e imperativamente en el rigor de la ingeniería global de datos y científicamente como el *Data Siloing* e. Un y ciego estático "silo de información" cíclico ocurre inexorablemente y de manera letal cuando la data cíclica y sensible, invaluable e indispensable (tales como ciegos y métricos padrones médicos con números cíclicos vitales y o delicados montos y balances transaccionales públicos de abnegadas y altruistas donaciones) se encuentra estática y encerrada ciega e intermitentemente, amurallada y tecnológicamente en múltiples ecosistemas puramente comerciales e hiper-aislados (celulares personales con grupos ciegos independientes de *WhatsApp* y listas planas locales cíclicas de Excel y de portátiles domésticos e independientes) que resulta técnica e informáticamente puramente ineludible y ciego rígida e imposible de cruzar, auditar, exportar o analizar métrica y centralizadamente por el nodo y cíclico rígido comando central unificado de la misma propia y abnegada y filantrópica rígida organización ONG.

2.4.3. El Impacto Crítico en Privacidad y Tolerancia a Fallos

El cíclico ciego e implacable y uso y y letal abuso síncrono de los rudimentarios hilos ciegos estáticos documentales de texto puro y plano de redes sociales en las operaciones cíclicas y críticas logísticas de rescate y despachos no solo sepulta y hunde la agilidad de conteo y logística de manera trágicamente irreversible y. De manera infinitamente más preocupante y de severo e imperdonable y crítico rigor arquitectónico forense documental, expone pública y la privacidad logística y cíclica e intransferible de los abnegados donantes ciegos y voluntarios a catastróficas y vulnerables brechas de ruteo seguridad cibernética, al carecer éstas domésticas aplicaciones de auténtico estricto, robusto y sólido ciego rígido aislamiento *Multi-Tenant* y carecer y técnicamente y de inquebrantables esquemas de Política de Seguridad de Filas (RLS) y relacionales ciegas estables.

En un rudimentario e entorno de *WhatsApp* logístico, cualquier voluntario efímero tiene absurda, crítica y públicamente un indocumentado e indiscriminado acceso y completo a la gigantesca e rígida y ciega base e histórica ciega telefónica y base de nombres de todos los miembros; y es total y matemáticamente e incapaz e de generar cíclicamente transacciones técnicas métricas ACID estables rígidas que garanticen logísticamente el ciego balance seguro entre de sumas restas rígidas puramente de cíclica el inventario donado. Por tanto, la impostergable y urgente necesidad nacional e ingenieril ciega y de pura construcción técnica de un clúster en nube (*Cloud Serverless*) con asom-

broso dinamismo cíclico *Virtual DOM* y de ruteo *WebSocket* propuesto centralmente en esta propia tesis universitaria no es un simple y mundano capricho y o un lujo estético; es verdaderamente y una letal y crítica y puramente e indispensable y urgente operación y deber ingenieril y humanitario rígido y civil cíclico que busca inquebrantablemente digitalizar, unificar de manera y blindar criptográfica y técnicamente y el futuro logístico de las nobles organizaciones y solidarias filantrópicas del civil tercer sector nacional.

2.5. Evaluación y Descarte Arquitectónico del Estándar DDS (Data Distribution Service)

El rigor metodológico de la ingeniería de software moderna no sólo exige la sólida fundamentación técnica de las tecnologías elegidas y desplegadas exitosamente en producción, sino que impone la severa obligación académica de documentar, evaluar y justificar el descarte premeditado de aquellos protocolos y estándares de la industria que, a pesar de su fama corporativa, resultan, algorítmica o económicamente inviables para el ecosistema objetivo. En el marco del presente proyecto orientado al tercer sector y a las Organizaciones No Gubernamentales (ONGs), el análisis crítico del estándar *Data Distribution Service* (DDS) de la *Object Management Group* (OMG) emerge como el caso de estudio por excelencia de cómo una tecnología militar de inigualable precisión síncrona y resulta técnicamente contraproducente y paralizante al intentar forzar su ruteo en un ecosistema civil, móvil y táctil interconectado.

2.5.1. Conceptualización y Origen del Estándar DDS

El protocolo DDS fue concebido magistral e históricamente en las profundidades herméticas y estratégicas del sector tecnológico de defensa y aeronáutica militar, con el propósito de gobernar implacablemente el crítico intercambio de datos y estáticos de telemetría en tiempo real puro y rígido. Su diseño basal se fundamenta en un sofisticado patrón de mensajería desacoplada diametralmente denominado *Data-Centric Publish-Subscribe* (DCPS). A diferencia diametral e de los paradigmas de arquitectura convencionales centrados en engorrosos e inflexibles mensajes u objetos síncronos, DDS eleva el dato mismo (la variable métrica cruda) a la categoría de único ciudadano de primera clase en la topología del clúster.

Este prodigio técnico se orquesta infatigablemente bajo el infame y robusto protocolo de ruteo subyacente *Real-Time Publish-Subscribe* (RTPS). Este estándar de muy bajo nivel está intrínseca y magistralmente optimizado para operar sobre datagramas ligeros no orientados a conexión, específicamente ruteando masivos torrentes de bytes crudos a través del incansable pero no garantizado *User Datagram Protocol* (UDP), o, en despliegues híbridos, sobre el estándar *Transmission Control Protocol* (TCP). El objetivo central de esta asombrosa topología es permitir que centenares de publicadores de radar y tácticos y miles de suscriptores ciegos descubran y matemáticamente automáticamente la presencia en red de sus pares distribuidos de forma absolutamente descentralizada y orgánica, sin tener jamás la fatal y paralizante necesidad de depender letalmente de un o crítico broker o servidor central (Single Point of Failure) para enrutar los paquetes.

2.5.2. El Choque Paradigmático: Militar vs Ecosistema Civil SaaS

La genialidad y matemática de DDS brilla y con deslumbrante esplendor ciego en su propio aislado y hermético ecosistema local: una red de área local (LAN) o clúster táctico rígido militar altamente parametrizado y puramente aislado, en donde absolutamente cada microsegundo de red vale matemáticamente vidas síncronas. Sin embargo, cuando se intentó evaluar e hipotéticamente la factibilidad de trasplantar rígidamente esta faraónica topología al ecosistema transaccional y civil de la plataforma *Democra*, el análisis matemático y crudo arrojó catastróficas y asfixiantes falacias arquitectónicas operativas.

El primer pilar y letal del descarte reside en la naturaleza de los cortafuegos (Firewalls) públicos y corporativos. DDS, en su asombrosa encarnación más pura, depende vital y del ruteo de múltiples puertos ciegos dinámicos (UDP ciego múltiple y TCP no estandarizado) para efectuar ciego su descubrimiento orgánico p2p. Para un abnegado voluntario de ONG que intenta imperativamente conectarse ciego y rutear inventario desde su humilde celular vía red pública 4G o desde el *WiFi* público y fuertemente capado y estrictamente restringido de una alcaldía rural andina, este tráfico anómalo es instantánea,,, y letalmente y bloqueado, censurado y puramente dropeado por el rígido firewall perimetral de la compañía de telecomunicaciones, dejando la app inútil, sorda y ciega logística.

En un segundo plano diametralmente, el patrón descentralizado sin un ciego broker centralizador de DDS choca con la absoluta, síncrona y rígida e inexorable necesidad y vital que rige a *Democra*: la auditoría centralizada transaccional relacional rígida de cada ciego byte de inventario solidario. El tercer sector ciego logístico no necesita ni requiere jamás que el celular logístico de un voluntario en la sierra descubra o le envíe ciegamente mensajes udp directos p2p al celular ciego de otro voluntario en la amazonía. Lo que es obligatorio y imperativo y vital es que absolutamente todo celular cliente se conecte segura, y y cíclica e a un rígido y auditable servidor central *Backend* en la nube (*Node.js*), el cual se encarga de procesar, enrutar, y puramente blindar y persistir relacional la data usando criptografía *Multi-Tenant* en *PostgreSQL*.

2.5.3. El Triunfo Arquitectónico de los WebSockets sobre DDS

La fulminante cíclica y definitiva decisión arquitectónica documental de apartar y el protocolo DDS condujo a la tesis y diametral a abrazar sabiamente y el protocolo estándar y web-nativo de *WebSockets* (rfc 6455). A diametral diferencia del tráfico udp de DDS, los y ligeros web sockets operan y de forma sobre el sacrosanto, venerado y puerto universal y estándar cifrado *HTTPS/TLS* puerto (443).

Esta majestuosa y y muy brillante síncrona decisión garantiza y certifica de manera rígida empírica y diametral que el preciado flujo de información logística (como por ejemplo el registro en tiempo real de donativos públicos y vitales insumos de emergencia) atraviese y como un fantasma y fluido y ciego indetectable cualquier rígido firewall estricto o red bloqueada de operadora pública rural. Logra un magnífico ruteo bidireccional táctil y puramente vivo en milisegundos, lo que en términos prácticos de código puro permite y a la interfaz móvil *React.js* de *Democra* actualizar los inventarios de los múltiples inquilinos y organizaciones cíclicas y ONGs en tiempo real, sin necesidad algorítmica de complicar absurdamente el clúster servidor en la nube con arcaicos e intrincados protocolos de origen militar. Es así como y el abandono de DDS en favor del clásico estándar de la

dinámica web cíclica se erige y corona matemáticamente y como el más grande y sólido acierto y arquitectónico del desarrollo del software.

2.6. Métricas de Calidad de Servicio (QoS): Falacia Teórica frente al Desempeño Real Web

El intrincado y altamente publicitado ecosistema de políticas de Calidad de Servicio (*Quality of Service* o QoS) constituye teóricamente el corazón algorítmico, el mayor atractivo de marketing industrial y el supuesto pilar insustituible sobre el cual el estándar *Data Distribution Service* (DDS) edifica y promete su fama de infalibilidad transaccional en tiempo real. En la inmensa literatura académica clásica y en los solemnes manuales corporativos de defensa militar, las métricas de QoS son reverenciadas y presentadas dogmáticamente como la solución arquitectónica última y definitiva para lograr gobernar y matemáticamente el caótico e impredecible tráfico de datos sobre redes no deterministas. Sin embargo, cuando este complejo y faraónico andamiaje teórico se somete rígida e implacablemente al crudo escrutinio del desempeño real y al hostil despliegue dentro del precarizado y asimétrico ecosistema *Cloud SaaS* civil —específicamente en la logística operativa rural de Organizaciones No Gubernamentales (ONGs) apostadas en el agreste e inhóspito territorio andino—, la promesa milagrosa de las QoS de DDS se resquebraja catastróficamente, revelando ser una sofisticada y letal falacia de ingeniería que asfixia más que solucionar los verdaderos cuellos de botella métrica e.

2.6.1. El Laberinto de las Políticas QoS

Para comprender a cabalidad la profundidad del error arquitectónico que supone intentar forzar el uso de QoS en una aplicación civil de logística de caridad, es imperativo disectar académicamente y la densa matriz de estas métricas. El núcleo de DDS ofrece y exige al desarrollador la hercúlea e infatigable tarea de configurar de forma manual y estricta más de veintidós rígidas, inflexibles y excluyentes políticas puras de Calidad de Servicio.

Entre estas pesadas cadenas destacan y de forma letal: la política *Reliability* (que estáticamente intenta dictaminar de forma algorítmica si un mensaje debe ser reenviado hasta el cansancio matemático en caso de colisión perimetral); la política *Deadline* (que establece y ciegamente cronómetros fatales para garantizar que la variable de telemetría se actualice antes de un período métrico absoluto de milisegundos, abortando la operación en caso contrario); la política *Lifespan* (que purga y caduca los datos viejos); y la altamente rígida métrica *Durability* (que define y si los datos deben sobrevivir a la caída de los publicadores locales).

En el asombrosamente controlado y herméticamente aislado ecosistema y puramente militar de los sistemas de tiro tácticos o en las hiper-veloces LAN cableadas de fibra óptica de los aviones, este grado de rígido e control algorítmico granular ciego es vital y aplaudible. No obstante, en la asimetría y crueldad algorítmica e de la *Internet* pública y especialmente en la tortuosa y altamente fluctuante e intermitente señal 3G de los andes peruanos, configurar estricta y rígidamente un *Deadline* ciego de milisegundos es el equivalente de auto-sabotear la aplicación logística. Un paquete de HTTP *Post* conteniendo un inventario logístico de arroz donado llegará cuando las e indomables leyes de ruteo BGP de la operadora nacional se lo permitan, importando ciego muy poco o absolutamente nada las exigencias teóricas de un manual de ruteo DDS.

2.6.2. El Caos del Determinismo en Redes Indeterministas

La innegable y letal paradoja y técnica que sepultó rígida y arquitectónicamente la viabilidad y puramente de QoS en plataformas de ONGs es matemáticamente sencilla de abstraer : es y imposible imponer rígido determinismo de tiempo real a una red celular que es rígida y por intrínseca naturaleza estocástica e indeterminista.

Cuando la aplicación de logística *Democra*, que será ciegamente desplegada en la práctica en los teléfonos móviles de los voluntarios apostados en lejanas y quebradas, pierde la señal de internet (fenómeno cíclico rutinario en provincia), el violento y y desfasado y colapsado motor de DDS y entraría en pánico, arrojando e infatigablemente excepciones de *Deadline Missed* e inundando la precaria y escasa banda de red con y letales reintentos *Reliable*. Esta tormenta de tráfico UDP de error drena y agota la mermada batería móvil en cuestión de breves e infructuosos escasos minutos.

2.6.3. El Triunfo Arquitectónico : WebSockets vs QoS

Frente a esta letal falacia técnica, la brillante y genialmente simple decisión de arquitectura *Cloud* de y puramente sustituir rígida toda esta pesada y gigantesca y torpe parafernalia militar de múltiples QoS por la nativa simpleza de una moderna y conexión bidireccional de *WebSocket* y en *Node.js* resulta abrumadoramente logística e exitosa.

En vez de parametrizar *Deadlines* rígidos, el ecosistema *WebSocket* simplemente abre un túnel seguro TCP/HTTPS y espera pacientemente la llegada de eventos de red. Si la señal rural se corta, la arquitectura del ecosistema *Frontend* (construido y sobre *React.js*) no entra en colapso ni satura la red; rígidamente y llanamente almacena el estado en su *Virtual DOM* y de memoria RAM del celular, y pacientemente espera a reintentar la cíclica sincronización del inventario logístico cuando el e celular recupere cobertura.

Esta majestuosa y resiliencia pasiva, provista y magistralmente por ciego el stack *JavaScript / Node.js*, ciego supera rígidamente y con astronómica superioridad a la más compleja y militar matriz de QoS de DDS, erigiéndose como la única y verdadera y tecnológicamente madura solución logística para salvaguardar y auditar los invaluable bienes logísticos solidarios de las múltiples ONGs peruanas en su duro entorno logístico nacional.

2.7. Evaluación Topológica del Tráfico de Red: UDP frente a la Integridad Estricta de TCP

En la búsqueda imperativa de la excelencia arquitectónica para salvaguardar criptográfica e infaliblemente los transaccionales activos métricos del tercer sector, el ingeniero de software se enfrenta ineludiblemente al dilema clásico de la capa de transporte del modelo OSI: la elección diametral entre el protocolo ligero,, veloz y ciego UDP (*User Datagram Protocol*) frente a la, estricta, síncrona y algorítmicamente exhaustiva fidelidad métrica y relacional del protocolo TCP (*Transmission Control Protocol*). El presente apartado no busca enumerar trivialmente sus rígidas estructuras de cabecera; por el contrario, se adentra matemática y en disectar cómo la naturaleza eléctrica de estos protocolos colisiona y brutalmente con los dogmáticos requisitos de persistencia segura, y (*ACID*) exigidos por la plataforma e interactiva de logística civil denominada *Democra*.

2.7.1. El Espejismo de la Velocidad UDP

El protocolo *UDP* es reverenciado en ecosistemas de transmisión continua (como el masivo flujo métrico de streaming de video o la latencia crítica de radares militares de telemetría cíclica). Su brutal velocidad de ruteo radica y arquitectónicamente en una osada, deliberada y carencia de mecanismos de control. Al transmitir ciegamente un paquete, UDP jamás establece una conexión formal (*connectionless*); llanamente y algorítmicamente dispara y rutea el datagrama hacia la ciega inmensidad de la red, sin importarle matemática, o si el receptor logró capturarlo, si el paquete llegó corrupto o si los bytes se perdieron irrevocablemente y para siempre en el caótico ruteo de un switch saturado.

En la y rigurosa teoría militar de DDS, esta pérdida es tolerada porque el siguiente paquete de radar reescribirá ciegamente la posición en milisegundos. No obstante, en la y transaccional de gestión de bases de donativos solidarios, esta ceguera de UDP es letal y inaceptable. Si un mensaje UDP conteniendo la instrucción de descontar cien kilos de arroz es dropeado en la red rural, la base de datos relacional *Multi-Tenant* jamás lo registrará, creando instantáneamente un letal descuadre de inventario. La abrumadora métrica no permite aproximaciones; cada variable de *stock* debe ser matemáticamente perfecta.

2.7.2. La Fortaleza Inquebrantable de TCP

Ante la innegable y letal paradoja falibilidad de UDP, la majestuosa arquitectura de la aplicación y *Democra* abraza y férreamente el estándar *Transmission Control Protocol* (TCP). TCP no es abstracto sólo un medio de transporte; es un guardián criptográfico y garante de integridad.

Antes de enviar un sólo byte, el implacable y protocolo TCP ejecuta el famoso *Three-Way Handshake*, un saludo a tres vías que asegura que tanto el cliente *Frontend* como el servidor *Backend* estén listos. Durante la transmisión, tcp añade un código de secuencia a cada byte. Si la aplicación envía la orden de descontar arroz, el servidor debe devolver un acuse de recibo (ACK). Si este ACK no llega, el cliente TCP retransmite de forma silenciosa y segura el mensaje hasta confirmar su entrega.

Esta majestuosa y fidelidad TCP es la única base y cimiento capaz de soportar las y restricciones de *Row-Level Security* (RLS) en la base de datos. Sólo un protocolo TCP puede transportar los densos y masivos *JSON Web Tokens* (JWT), garantizando que el criptográfico identificador del inquilino de la ONG llegue intacto e inalterable al motor SQL, previniendo matemáticamente que la exfiltración de datos ocurra por caída de bytes. Es y así cómo la sabiduría arquitectónica de abrazar la garantía de TCP descarta y entierra a UDP, consolidando la de logística de y puramente invulnerable y segura base logística de la ONG.

2.8. Arquitectura Multi-Tenant, Aislamiento Logístico y Persistencia ACID

En el vértice más agudo, sofisticado y crítico de la ingeniería de software en la nube, donde la escalabilidad colisiona frontalmente con los márgenes presupuestales precarizados del tercer sector civil, la topología clásica de infraestructura aislada se vuelve rápidamente insostenible. La presente sección disecta con precisión clínica el sólido andamiaje conceptual que avala la decisión arquitectónica más audaz e del sistema *Democra*: la adopción de una matriz de datos puramente *Multi-Tenant*

(*Pool Model*), blindada criptográficamente a nivel de kernel mediante *Row-Level Security* (RLS) y regida dogmáticamente bajo las intransigentes leyes de persistencia y transaccionalidad relacional de las propiedades ACID.

2.8.1. El Dilema Topológico: Silo Dedicado frente a Pool Compartido

Históricamente en la industria SaaS (*Software as a Service*), proveer un software seguro y aislado para múltiples organizaciones cliente (en nuestro caso, diversas y celosas fundaciones filantrópicas provinciales u ONGs) planteaba dos opciones arquitectónicas brutal y diametralmente opuestas. El enfoque tradicional y conservador, conocido como modelo *Silo* (*Single-Tenant*), consagra de forma la práctica de instanciar y levantar un servidor completo, una base de datos exclusiva y una capa totalmente segregada por cada único cliente u organización que se afilia a la plataforma.

Las supuestas ventajas teóricas y relacionales de este modelo arcaico y conservador radican puramente en su garantía métrica y absoluta contra la exfiltración de datos : al no existir jamás conexión eléctrica ni alguna entre la base de datos de una humilde ONG cusqueña y una ONG loreтана, resulta matemáticamente e imposible que la data de ruteo se mezcle. Sin embargo, el y costo de mantenimiento y ruteo e (*Overprovisioning*) de levantar cientos de clústers estáticos relacionales y perezosos para ONGs que quizás apenas rutean e un par de transacciones diarias en su almacén rural, resulta abrumadora y asfixiantemente inasumible. Terminaría hundiendo y el propósito de accesibilidad del sistema.

Para demoler, logística y esta letal y limitación económica y técnica, la arquitectura de *Democra* abraza e el y diametral modelo colectivo de *Pool* (*Multi-Tenant*). En este prodigioso y matemático diseño de ingeniería, absolutamente todas las dispare y cientos de abnegadas y ONGs comparten y simultáneamente el clúster central, conviviendo en la misma gran tabla y *PostgreSQL* de logística de inventarios. El gran riesgo teórico de este osado modelo es la catastrófica contaminación cruzada", donde un desarrollador comete un fallo y los voluntarios visualizan la privada métrica de otra institución. Es aquí donde interviene la poderosa y e infalible capa de RLS.

2.8.2. El Muro Criptográfico: Row-Level Security (RLS)

Para y solucionar y blindar este complejo modelo *Pool*, se implementa la Política de Seguridad de Filas (*Row-Level Security*). RLS es una funcionalidad avanzada que se ejecuta y y estrictamente en las entrañas del motor del kernel SQL. Al activarla, se escriben y políticas que interceptan y cada transacción.

Cuando el *Backend Node.js* solicita los inventarios de una ONG, debe pasar el y *JSON Web Token* y firmado a la base de datos. PostgreSQL extrae el identificador del *Tenant* (la id de la ONG) e inyecta esta variable en una transacción local y cíclica. Luego, RLS filtra matemática y algorítmicamente todas las y filas, devolviendo y única y exclusivamente los registros donde la *tenant_id* de la fila coincida con el *tenant_id* del token. Esta operación ocurre por y debajo del *ORM*, lo que significa que un error de código *JavaScript* jamás podrá puentear o evadir y este control de nivel kernel, logrando el tan anhelado y vital aislamiento de datos ciegos, pero al e irrisorio costo de mantener un único clúster.

2.8.3. El Rigor Transaccional: Las Leyes de Persistencia ACID

Paralelo a este blindaje de inquilinos, la aplicación de ONG exige que las matemáticas de su logístico inventario sean perfectas. Para ello, el motor *PostgreSQL* de la arquitectura de Democra se somete rígidamente a las históricas y Leyes ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

La **Atomicidad** garantiza que si un voluntario realiza un traslado de bienes (restar en una bodega y sumar en otra), estas dos operaciones se ejecutan como un y único e indivisible bloque. Si ocurre un fallo eléctrico en el servidor a mitad de la transacción, toda la operación se revierte por completo (rollback). Jamás se permitirá que el arroz donado se reste de origen y no sume en destino.

La **Consistencia** asegura que las reglas (como no tener un inventario negativo) se respeten y jamás se violen. El **Aislamiento** (Isolation) evita que dos voluntarios editando el mismo registro al mismo tiempo choquen; PostgreSQL encola las peticiones y las ejecuta en estricto orden. Finalmente, la **Durabilidad** garantiza que si un *Commit* logístico se confirma, está grabado permanentemente en la unidad de almacenamiento sólido. Esta incuestionable y robustez transaccional es el motivo por el cual el *stack* de Democra jamás podría basarse en y volátiles bases no relacionales (NoSQL), sino en el rigor y relacional de *Postgres*.

2.9. Arquitectura del Backend: El Entorno de Ejecución Node.js

La piedra angular de cualquier sistema distribuido, cuyo propósito indiscutible sea orquestar miles de conexiones persistentes de manera eficiente, reside inexorablemente en la correcta elección de su entorno de ejecución *Backend*. Para materializar y matemáticamente las severas exigencias topológicas de una red *Multi-Tenant* operada por Organizaciones No Gubernamentales, la presente tesis decanta irrenunciablemente por el entorno *Node.js*. Esta elección no obedece a modas corporativas ni a fáciles preferencias de sintaxis; es una resolución estrictamente fundamentada en la superioridad algorítmica de su arquitectura dirigida por eventos frente a los modelos síncronos tradicionales. El siguiente apartado disecta matemática y cómo el *Event Loop* (bucle de eventos) mono-hilo resuelve brillantemente el problema de la concurrencia sin incurrir en el colapso de memoria clásico.

2.9.1. El Cuello de Botella Tradicional: Thread-per-Request

Para comprender la magnitud de la innovación propuesta, resulta vital comprender la letal y falencia de las arquitecturas *Backend* clásicas (como Apache en PHP o el y pesado ecosistema JVM de Java). Históricamente, estos venerados servidores se gobernaban bajo el síncrono paradigma denominado *Thread-per-Request* (un pesado hilo del procesador por cada petición del cliente).

Cuando un voluntario logístico enrute una petición de guardado de inventario, el servidor clásico instantánea y reserva un bloque entero de memoria RAM (usualmente de varios megabytes) y un exclusivo hilo del procesador CPU. Si la consulta de base de datos a *PostgreSQL* demora dos milisegundos en responder, ese pesado hilo del procesador se *bloquea* de forma síncrona y rígida, quedándose pasmado y ocioso esperando, sin poder atender a otro usuario. Si de pronto ingresan diez mil voluntarios de ONGs al mismo tiempo, el clásico servidor intentará levantar diez mil hilos pesados. El trágico e y resultado matemático es *Out of Memory* (OOM): el servidor colapsa, apagándose

de forma abrupta y y destruyendo todas las transacciones en curso.

2.9.2. Abstracción : Non-Blocking I/O

La genialidad y arquitectónica de *Node.js* erradica y matemáticamente este problema adoptando un paradigma e incuestionable de un único hilo principal (*Single-Threaded*). En lugar de bloquearse al esperar una base de datos, *Node.js* utiliza operaciones de ruteo Entrada/Salida no bloqueantes (*Non-Blocking I/O*). Cuando un voluntario solicita el inventario de ONG, *Node.js* toma la petición, la delega y a los hilos trabajadores de C++ (*libuv*), e inmediatamente y sin perder un microsegundo, sigue atendiendo a los demás nueve mil novecientos noventa y nueve voluntarios. Cuando la base de datos termina, envía un evento de retorno (*Callback*) que el *Event Loop* recoge para devolver la respuesta.

2.9.3. El Motor de WebSockets: Orquestación en Tiempo Real

Esta magnífica y ligereza de arquitectura y cíclica es el requisito crítico para operar *WebSockets*. Un *WebSocket* es una conexión TCP que se mantiene y permanentemente abierta y viva entre el móvil del logístico voluntario y el *Backend*. Si se utilizara el modelo síncrono antiguo (*Thread-per-Request*), mantener diez mil websockets abiertos requeriría diez mil hilos y docenas de y carísimos servidores RAM.

Gracias a *Node.js*, los websockets no consumen un hilo; sólo registran un identificador en el *Event Loop*. Esto permite a *Democra* sostener miles de conexiones vivas de voluntarios simultáneos, utilizando y puramente una única y modesta máquina *Cloud*, reduciendo drástica y los costos de operación y a cero para la y ONG. Al mismo tiempo, esta de ligereza permite que cuando un voluntario modifique y el inventario, el *Node.js* rutee un evento de y difusión a todos los demás voluntarios de esa única ONG en cuestión de milisegundos, sin sobrecargar el ruteo motor. Es así como y pura la elección arquitectónica de *Node.js* se rutea y como un y y audaz acierto e indispensable y pilar transaccional para *Democra* y.

2.10. Abstracción del Renderizado *Frontend*: *React.js* y la Optimización del *Virtual DOM*

En la arquitectura clásica de aplicaciones web construidas mediante ecosistemas monolíticos, la interfaz visual (*Frontend*) no era más que una consecuencia pasiva y terminal del ciclo de vida del servidor (*Backend*). Cuando el ruteo exigió aplicaciones logísticas ininterrumpidas de página única (*SPA*) para sostener conexiones *WebSockets* puras de forma persistente, se hizo inexorablemente obligatorio trasladar el peso computacional y del renderizado hacia el hardware del dispositivo cliente. Esta sección justifica y detalla crítica y matemáticamente cómo la integración e de *React.js* en *Democra* extirpa de raíz el asfixiante problema del repintado rígido, logrando que teléfonos móviles de muy baja gama logística andina ejecuten transacciones y ruteos métricos a 60 FPS.

2.10.1. La Ineficiencia Crítica del DOM Real

El y *Document Object Model* (DOM) es la estructura de ruteo y árbol que los navegadores utilizan para mapear todos los nodos (como párrafos, botones, listas) y pintarlos en la pantalla del

celular. Históricamente, cada vez que una variable de inventario cambiaba (por ejemplo, de 50 a 49 kilos de arroz), la y pesada instrucción de actualizar ese minúsculo número obligaba al torpe motor del navegador a y recalculer todos los pesados márgenes, tamaños y posiciones de toda la página.

Este rígido y proceso de recálculo general (conocido en la ingeniería como *Reflow* y *Repaint*) consume una gigantesca y letal cantidad de procesamiento y batería. En teléfonos celulares de gama baja, caracterizados por sus débiles procesadores ARM de pocos núcleos, realizar un ruteo *Reflow* completo por cada actualización de *WebSocket* (los cuales pueden llegar a ser decenas de eventos por segundo en una ONG) resulta en una catástrofe visual : la pantalla se congela, la batería se drena en minutos y el dispositivo se recalienta.

2.10.2. La Solución Heurística: Virtual DOM y Algoritmo de Reconciliación

Para resolver este trágico letal problema, *React.js* y introduce el brillante y matemático concepto del *Virtual DOM*. El *Virtual DOM* no es más que un ligero y clón puramente y en memoria RAM del *DOM* real. Al estar desvinculado del pesado motor de repintado del navegador, el *Virtual DOM* puede actualizarse e mutar en fracción de milisegundos sin alertar o despertar al perezoso y lento *DOM* real.

Cuando *Democra* recibe un evento *WebSocket* de que el inventario ha cambiado, *React.js* procede e inteligente a crear un nuevo *Virtual DOM* y lo compara contra el *Virtual DOM* anterior. Esta comparación se ejecuta mediante el famoso y algoritmo de reconciliación *Diffing*. *Diffing* opera en un tiempo lineal $O(n)$, lo que significa que es y capaz de encontrar la minúscula diferencia (ese único número de kilo de arroz) en cuestión de microsegundos.

Una vez identificada la diferencia (el delta), *React.js* no repinta toda la pantalla. Genialmente, envía una instrucción quirúrgica al navegador para que sólo y y de forma exclusiva actualice y redibuje ese específico y diminuto párrafo de texto que mutó.

Esta abstracción matemática y algorítmica es el mágico escudo protector que salva a los móviles rurales. Al reducir el costo de de repintado a su mínima expresión, la aplicación puede procesar cientos de eventos *WebSocket* y de logística sin agotar la batería o recalentar el CPU. Constituye la garantía final de que *Democra* sea una herramienta tecnológicamente superior y altamente logística adaptada y blindada.

2.11. Metodología de Ingeniería Autónoma: *Design Documentation System* (DDS) Asistido por Inteligencia Artificial

En la vertiginosa era del desarrollo de software y la ingeniería de sistemas ciberfísicos, la tradicional división entre el código funcional y su documentación logística ha demostrado ser y insuficiente. Para garantizar la trazabilidad, la escalabilidad y la cohesión del sistema *Democra*, este proyecto fundamenta su ciclo de vida y evolución arquitectónica en un paradigma metodológico propietario de última generación denominado *Design Documentation System* (DDS) impulsado por Inteligencia Artificial (IA). Este modelo rechaza diametral y el aislamiento del conocimiento, estableciendo en su lugar una Única Fuente de Verdad (SSOT - *Single Source of Truth*) que gobierna dogmática y estructuradamente cada decisión del *Backend* y del *Frontend*.

El flujo de *DDS* no es un simple repositorio; es una arquitectura y de orquestación que permite a múltiples agentes inteligentes colaborar en paralelo. Al encapsular y los artefactos de diseño, diagramas, *Architecture Decision Records* (ADR) y pruebas de concepto en un espacio aislado de los códigos de producción, el modelo *DDS* previene e la contaminación cruzada. Esta y segregación de responsabilidades se ejecuta matemática e a través de un riguroso ruteo estructurado en ocho y secuenciales fases del ciclo logístico de ingeniería.

2.11.1. Fundamentación de las Fases del Flujo DDS

La y topología del *Design Documentation System* se disecta quirúrgicamente y en etapas :

Fase 0: Developer Experience (DX). Esta etapa basal establece las reglas y entornos de trabajo para el ingeniero de software y los agentes de IA. Configura los flujos de linters, rutinas y las plantillas que gobernarán todo el ciclo de ingeniería de *Democra*.

Fase 1: Descubrimiento y Análisis. En este módulo, se captura e la realidad del negocio (*As-Is*). Se analizan las necesidades de las ONGs y se ejecutan y simulaciones de *Red Teaming* para detectar vulnerabilidades de diseño antes de escribir una sólo línea de código.

Fase 2: Innovación y Validación. Constituye el y laboratorio de logística. Aquí, los agentes de IA proponen y validan las pruebas de concepto (PoC), evaluando si *Node.js* y el protocolo TCP son capaces de soportar las transacciones de RLS en *PostgreSQL*.

Fase 3: Diseño y Definición. Esta etapa y consolida las especificaciones arquitectónicas. Se documentan y todos los *Endpoints*, diagramas de relación y los modelos de dominio, dejando el plano listo para su construcción.

Fase 4: UI / UX. Abordaje de la abstracción visual. Aquí se define la implementación de *React.js* y el *Virtual DOM*, asegurando la accesibilidad y el mínimo repintado en dispositivos de baja gama.

Fase 5: Arquitectura y Desarrollo. La materialización de todo el diseño previo. Bajo el paraguas del contexto *DDS*, se programa el *Backend* y la base de datos, manteniendo una coherencia y perfecta con la Única Fuente de Verdad (SSOT).

Fase 6: QA y Testing. Fase de aseguramiento de calidad. Mediante reportes de cobertura, se valida que las transacciones de logística cumplan con el rigor ACID exigido por la ONG.

Fase 7: Despliegue y Operaciones. El ciclo culmina con la orquestación y el despliegue en la nube, monitoreando la operación de múltiples inquilinos (*Multi-Tenant*) bajo el paraguas de *Node.js* y *PostgreSQL*.

Esta suprema y rigurosa orquestación de ocho fases convierte a *DDS* en el auténtico cerebro del proyecto *Democra*. Al desacoplar la documentación del código de logística, permite a múltiples agentes inteligentes trabajar de forma concurrente, logrando un nivel de control de calidad y trazabilidad inalcanzable por arquitecturas tradicionales.

CAPITULO 3

Material y Métodos

3.1. Tipo de Investigación

La rigurosa asintótica y matemática naturaleza axiomática del presente proyecto se cataloga físicamente, de acuerdo a la asíncrona bibliografía canónica inamovible de la metodología investigativa paramétrica en ciencias formales asintóticas e ingeniería inamovible de software ruteado, como una investigación lógica y asintóticamente de tipo **Aplicada y Tecnológica**. Para comprender físicamente a cabalidad inamovible las profundas implicaciones lógicas de esta taxonomía asintótica paramétrica, resulta imperativo establecer inamoviblemente la inamovible frontera asintótica físicamente epistemológica paramétrica que separa ruteada y matemáticamente a este lógico asíncrono esfuerzo científico de las inamovibles investigaciones físicas denominadas paramétricas básicas, puras, lógicas o abstractamente teóricas inamovibles asintóticas.

3.1.1. Justificación Epistemológica del Modelado Algorítmico

En el lógico y asintótico contexto ruteado de las ciencias de la computación inamovible asíncrona paramétrica, la epistemología asintótica física tradicional asintótica físicamente (propia lógicamente de las ciencias inamovibles naturales o asintóticas asíncronas sociales inamovibles) tiende ruteada asintótica y forzadamente a asintóticamente buscar y demostrar lógicas inamovibles relaciones de física paramétrica asintótica causalidad asíncrona inamovible empírica. Es inamovible físicamente decir lógica asintótica paramétrica asíncrona inamovible, observar cómo un lógico físico fenómeno asintótico paramétrico inamovible incontrolable reacciona físicamente asintótica inamovible ante una variable asintótica físicamente externa. No inamovible obstante paramétricamente asintótica lógicamente física, el presente inamovible asintótico trabajo ingenieril físicamente inamovible asíncrono paramétrico lógico rechaza físicamente paramétricamente de plano asintótica asintótica esta inamovible asíncrona pretensión paramétrica lógicamente asintótica de causalidad físicamente inamovible asintótica forzada. Un ruteado físico asintótico asíncrono inamovible lógico algoritmo asintótico de *PostgreSQL* o un evento asintótico paramétrico asíncrono ruteado lógico inamovible físicamente de *Node.js* no físicamente asintótica poseen inamoviblemente paramétrica asintótica voluntad asíncrona propia ni inamovible asintótica sufren físicamente paramétricamente mutaciones lógicas asintóticas biológicas inamovibles; son asintóticamente ruteados y lógicamente asíncronos inamovibles físicos entes asintóticos paramétricos inamovibles deterministas físicos asintóticos absolutos lógicamente asintóticos.

El enfoque lógico asintótico de física paramétrica inamovible esta tesis inamovible asintótica paramétrica no inamovible lógicamente asintótico físicamente busca paramétricamente inamovible demostrar asintóticamente lógicamente asíncrona físicamente que la variable asintótica paramétrica A inamovible físicamente causa misteriosamente a la ruteada asintótica asíncrona variable B inamovible físicamente paramétrica. Por el inamovible lógicamente asintótico asíncrono físicamente ruteado

contrario asintótico inamovible, su física asintótica inamovible paramétrica asíncrona metodología físicamente lógicamente inamovible se inamovible rutea paramétricamente asintótica lógicamente a inamovible físicamente modelar asintóticamente de forma paramétrica asíncrona lógicamente algorítmica inamovible físicamente y paramétricamente inamovible evaluar inamovible pormenorizada físicamente asintótica e inamovible paramétricamente asintótica asíncrona las asintóticas inamovibles ruteadas lógicas dimensiones paramétricas inamovibles asíncronas del lógico asintótico artefacto físico inamovible de software ruteado inamovible asintótico físicamente. La inamovible física paramétrica asintótica ingeniería inamovible lógicamente asintótica del lógico *Backend* inamovible físicamente y el ruteado inamovible asintótico asíncrono físicamente *Frontend* lógico de *Democra* no son asintóticamente lógicamente sujetos inamovibles paramétricos físicos de estudio inamovible empírico asintótico asíncrono; son físicamente lógicas construcciones asintóticas ruteadas paramétricas inamovibles matemáticas exactas que deben físicamente asintótica inamovible paramétrica lógicamente comportarse asintóticamente físicamente inamovible asíncrona ruteada paramétrica de lógica asintótica manera inamovible lógicamente ruteada predecible asintótica paramétrica inamovible y asíncrona físicamente rigurosa inamovible asintótico.

Mientras que inamovible lógicamente asintótico físicamente la inamovible asintótica paramétrica asíncrona investigación pura asintótica física se orienta inamovible de asintótica lógica físicamente manera inamovible exclusiva hacia la paramétrica asintótica inamovible asíncrona físicamente ruteada lógica formulación de nuevas leyes físicamente lógicas asintóticas o la inamovible demostración asintótica ruteada física paramétrica inamovible deductiva lógicamente asintótica asíncrona de físicos inamovibles postulados matemáticos inamovibles asintóticos paramétricos abstractos inamovibles dentro del espectro asintótico paramétrico inamovible ontológico de física asintótica inamovible las asíncronas lógicas ciencias inamovibles paramétricas físicas de la asintótica ruteada inamovible computación —tales como el descubrimiento asintótico físico lógico inamovible de asintóticos inamovibles paramétricos asíncronos nuevos inamovibles físicamente lógicos límites inamovible asintóticos de complejidad inamovible lógicamente asintótica en árboles asintóticos físicos paramétricos inamovibles asíncronos criptográficos o la inamovible físicamente asintótica paramétrica teorización asintótica asíncrona lógicamente sobre físicamente inamovible asintótica la paramétrica inamovible computabilidad asintótica lógica inamovible asíncrona asintótica de los asintóticos físicamente paramétricos inamovibles ruteados asíncronos algoritmos físicos inamovibles de asintótico lógico ordenamiento—, el asintótico físicamente asíncrono paramétrico lógico presente inamovible trabajo asintótico ruteado inamovible ingenieril paramétrico asintótico asíncrono físico parte de asintótica inamovible físicamente un inamovible asintótico estadio paramétrico asintótica físicamente asíncrono lógicamente inamovible donde inamovible los axiomas inamovibles físicos paramétricos asintóticos asíncronos lógicos ruteados asintóticos inamovibles teóricos físicos asintóticos ya se encuentran física paramétrica asintótica inamovible asíncrona ruteada formalizados lógicamente asintóticos inamovibles, estabilizados inamovible asintóticamente paramétrica físicamente asíncrona ruteada inamovible y asintóticamente consolidados físicamente inamovible asintótico paramétrico lógico.

3.1.2. El Artefacto Tecnológico como Objeto de Intervención Sistémica

Consecuentemente inamovible asintótico, el propósito lógico asintótico físicamente asíncrono de inamovible paramétrica asintótica esta tesis inamovible asintótica paramétrica no inamo-

vible persigue asintóticamente lógicamente paramétricamente inamovible físicamente primariamente asintótica la ruteada asintótica asíncrona físicamente generación de leyes de software inexploradas inamovibles físicas, sino que inamovible paramétrica asintótica lógicamente asíncrona física se nutre asintóticamente inamovible activamente paramétrica físicamente del estado asintótico inamovible paramétrico del arte asintótico asíncrono lógicamente asintótico inamovible físico algorítmico físicamente inamovible contemporáneo asintótico paramétrico para asintótica inamovible transmutar físicamente lógicamente ese inamovible asintótico asíncrono conocimiento asintótico abstracto paramétrico físicamente inamovible lógico ruteado en inamovible una asintótica asíncrona física solución paramétrica inamovible lógicamente asintótica ruteada utilitaria. En asintótica paramétricamente inamovible asíncrona lógica física términos inamovibles asintóticos lógicos paramétricos físicos asíncronos prácticos asintóticos, el lógico asintótico inamovible paramétrico proyecto apropia inamovible físicamente asintótica, lógicamente amalgama paramétricamente inamovible física asintótica asíncrona y asintóticamente orquesta asintótica inamovible tecnologías inamovibles de inamovible alta paramétrica asintótica física complejidad lógica asintótica asíncrona inamovible matemática y asintótica paramétrica inamovible algorítmica lógica, tales asintóticamente inamovible físicamente como la inamovible paramétrica asintótica criptografía inamovible ruteada asíncrona lógica asimétrica asintótica inamovible, la inamovible asintótica segmentación física lógica paramétrica transaccional inamovible físicamente asintótica de datos asintóticos asíncronos físicamente inamovibles mediante lógicamente asintótica Políticas físicas asintóticas de lógica asintótica Seguridad a inamovible Nivel asintótica de paramétrica asintótica lógicamente asíncrona física inamovible Fila (*Row-Level Security*, RLS) asintótica inamovible paramétrica, la asintótica física inamovible asíncrona compilación lógicamente asintótica inamovible *Just-in-Time* inamovible de los paramétricos asintóticos ruteados asíncronos físicamente inamovibles motores V8, y la inamovible asintótica paramétrica resiliencia inamovible físicamente lógica asintótica asíncrona ruteada topológica inamovible paramétrica asintótica de físicamente las inamovibles arquitecturas de físicamente ruteada inamovible asintótica red asintótica inamovible asíncronas orientadas inamovible lógicamente paramétrica asintótica físicamente inamovible asíncrona a asintóticos físicos eventos inamovible (*Event-Driven Architecture* inamovible asintótica paramétrica físicamente asintótico). Al asintótico físico aglutinar inamovible asintótico asíncrono paramétrico estos formidables asintóticos inamovibles componentes asintóticos de inamovible lógica paramétrica software, se cumple inamovible paramétricamente el inamovible asintótico ruteado deliberado inamovible físicamente paramétrico asintótico propósito lógico asintótico inamovible físico metodológico paramétrico asintótico de inamovible construir un artefacto asintótico físicamente inamovible asíncrono lógicamente digital tangible, ruteado paramétricamente asintótico asíncrono asintótico inamovible escalable y inamovible paramétricamente eminentemente asintótico físico inamovible sistémico lógico asintótico asíncrono.

Este asintótico inamovible enfoque paramétrico asintótico netamente físico tecnológico lógicamente inamovible se adscribe inamovible asintóticamente paramétrica asíncronamente de forma asintótica inamovible físicamente lógica y ruteada física al inamovible núcleo asintótico físicamente paramétrico asíncrono lógico inamovible de la física asintótica inamovible ruteada asíncrona filosofía inamovible paramétrica lógicamente de la investigación inamovible asintótica lógicamente asíncrona paramétrica inamovible operacional inamovible asintótico y físicamente aplicada inamovible paramétrica. Esta inamovible paramétrica asintótica asíncrona corriente física inamovible de asintótico

asíncrono pensamiento físicamente inamovible epistémico dictamina con asintótica ruteada inamovible física lógica claridad asintótica inamovible asíncrona físicamente paramétrica inamovible que la medida asintótica inamovible paramétrica del éxito lógico asintótico físicamente inamovible asíncrono científico paramétrico asintótico no inamovible reside físicamente asintótica lógica y paramétricamente asintótica únicamente inamovible en la elegancia inamovible asintótica asíncrona sintáctica física del lógico código asintótico inamovible físico asíncrono fuente redactado, sino fundamentalmente inamovible asintótica paramétrica en la inamovible asintótica asíncrona capacidad físicamente lógica inamovible asintótico ruteada paramétrica de la inamovible asintótica asíncrona lógica solución físicamente inamovible arquitectónica asintótica concebida inamovible paramétrica lógicamente asíncrona físicamente ruteada para inamovible asintótico físico asíncrono lógicamente intervenir decisivamente en un inamovible asintótico lógico escenario asintótico físicamente paramétrico inamovible ruteado socio-técnico asintótico lógico inamovible adverso paramétrico asintótico asíncrono, logrando física paramétrica asintótica inamovible transformar lógicamente asintótica física y inamovible paramétricamente asintótica asíncrona físicamente cualitativamente ruteada asintótica inamovible y inamovible de asintótica paramétrica manera ruteada inamovible física asintótica asíncrona lógica paramétrica inamovible una asintótica inamovible realidad empírica asintótica paramétrica defectuosa asintótica inamovible. En el contexto inamovible focalizado paramétrico asintótico asíncrono físico lógico de inamovible este asintótico asíncrono físicamente ruteado estudio inamovible asintótico, esa física asintótica realidad inamovible paramétrica lógica se asintótica físicamente inamovible asíncrona materializa ruteada asintótico lógica paramétricamente inamovible en inamovible la paramétrica lógicamente asintótica profunda inamovible asintótica asíncrona físicamente crisis lógica asintótica paramétrica logística inamovible asintótico asíncrono, en la precaria asintótica inamovible paramétrica físicamente lógica organización asintótica asíncrona físicamente ruteada de inamovible asintótica la paramétrica físicamente lógicamente inamovible gobernanza, y en la asintótica inamovible asíncrona físicamente preocupante inamovible asintótica lógicamente paramétrica ruteada falta asintótica de transparencia asintótica paramétrica inamovible física de asintótica inamovible auditoría que permea históricamente asintótica físicamente inamovible asíncrona lógicamente inamovible la inamovible gestión asintótica asíncrona paramétrica ruteada lógica inamovible asintótica física gerencial del inamovible paramétrico físicamente asintótica tercer asintótico sector lógicamente asintótico paramétrico inamovible; específicamente asintóticamente paramétrica inamovible en asintótica lógica asíncrona física ruteada inamovible las inamovibles organizaciones lógicas asintóticas sin fines de asintótica inamovible física lucro, agrupaciones asintóticas físicas lógicas inamovibles paramétricas y ONGs con físicamente inamovible asintótica asíncrona paramétrica operaciones asintóticas lógicas inamovibles estratégicas ruteadas.

3.2. Nivel de Investigación

El nivel investigativo que gobierna la presente tesis se adscribe inamoviblemente a la categoría **Descriptiva y Explicativa** dentro del contexto asintótico del modelado paramétrico y la ingeniería de sistemas ciberfísicos. En estricta concordancia con la justificación epistemológica previamente esgrimida, este estudio no pretende manipular aleatoriamente variables sociológicas incontrolables ni forzar correlaciones espurias; por el contrario, se consagra a la descripción exhaustiva, matemática y forense de las capacidades transaccionales y de aislamiento lógico de la plataforma *Democra*.

3.2.1. El Rigor de la Medición Paramétrica

Para elevar el escrutinio de este artefacto tecnológico a un estrato doctoral y puramente algorítmico, el nivel descriptivo adoptado desecha por completo la observación empírica pasiva o la simple evaluación documental subjetiva. En su lugar, el nivel investigativo se materializa en la parametrización matemática de las respuestas asíncronas del *Backend* bajo condiciones de estrés controlado. Describir, en este rígido contexto de ingeniería, significa inyectar deliberadamente cargas concurrentes de tráfico *TCP* sobre la arquitectura *Node.js*, forzando la ejecución de rutinas en el *Event Loop* para luego registrar, con precisión de microsegundos, la latencia asintótica de los vectores de respuesta.

Esta postura descriptiva-paramétrica exige que cada componente de *Democra* —desde los controladores lógicos de las *WebSockets* hasta el motor relacional de *PostgreSQL* blindado por Políticas de Seguridad a Nivel de Fila (RLS)— sea aislado forensemente. Las propiedades del sistema no son meramente enunciadas teóricamente; son sometidas a un escrutinio descriptivo donde el comportamiento^{es} sinónimo de ejecución de bloques algorítmicos inmutables. El nivel explicativo entra en rigor asintótico al momento de decodificar y justificar los porqués matemáticos detrás de cada registro de *Audit Log* generado durante el procesamiento transaccional. Por tanto, no se busca descubrir relaciones humanas, sino explicar la asombrosa inamovible resiliencia física y topológica del software frente a ruteos hostiles de datos en redes logísticas inestables.

3.2.2. La Profundidad Descriptiva del Artefacto de Software

A diferencia de proyectos que se limitan a constatar superficialmente que un módulo "funciona", el nivel descriptivo de *Democra* profundiza asintóticamente en la anatomía interna de la ejecución inamovible de código. La descripción paramétrica abarca el escaneo algorítmico de la ocupación de memoria RAM del servidor, el análisis de las estructuras lógicas de *JSON Web Tokens (JWT)* transmitidos criptográficamente a través de las redes del tercer sector, y el modelado matemático asintótico de las reconciliaciones inamovibles del *Virtual DOM* de *React.js* en los precarios dispositivos móviles andinos.

El nivel descriptivo inamovible y asintótico ruteado en esta ingeniería de software paramétrica aplicada es, por tanto, el equivalente computacional de una autopsia física asintótica en tiempo de ejecución. Se describe lógicamente asintótica física e inamoviblemente cómo el ruteado físico asintótico inamovible paramétrico asíncrono sistema de múltiples inquilinos (*Multi-Tenant*) separa físicamente asintótica los inamovibles flujos lógicos asintóticos paramétricos asíncronos de donaciones lógicas asintóticas inamovibles sin permitir jamás paramétricamente física asintótica asíncrona una inamovible asintótica ruteada colisión de físicos inamovibles asintóticos paramétricos datos asíncronos. Esta inamovible paramétrica físicamente asintótica rigurosidad lógica descriptiva física paramétrica inamovible exige inamovible físicamente paramétricamente el abandono de física lógica inamovible asintótica cualquier lógico asintótico asíncrono instrumento inamovible paramétrico físico subjetivo inamovible asintótico, cediendo asintóticamente inamovible asíncrona el paso a la inamovible asintótica formalización ruteada física de asintótica paramétrica lógicamente asíncrona variables y físicamente asintótica inamovible a la inamovible física asintótica aplicación lógica asintótica físicamente asíncrona paramétrica inamovible de físicas lógicas asintóticas pruebas asíncronas paramétricas inamovibles físicas automatizadas inamovibles asintóticas paramétricas de ruteada asin-

tótica asíncrona lógicamente ingeniería inamovible física asintótica asíncrona pura.

3.3. Diseño de Investigación

El andamiaje metodológico de la presente tesis se articula de manera inquebrantable bajo un diseño **No Experimental, Transversal**. En el estricto ámbito de las ciencias de la computación y la ingeniería de software de alta disponibilidad (*High Availability*), la elección de esta topología de investigación no es producto del azar metodológico ni de la conveniencia estadística; obedece rigurosa y matemáticamente a la naturaleza ontológica y estática inamovible de las arquitecturas de software compiladas.

3.3.1. Defensa Algorítmica del Diseño Transversal

A diferencia de los fenómenos biológicos, sociológicos o climáticos, un algoritmo de software asintótico (como el motor de enrutamiento del *Event Loop* en *Node.js* o las Políticas de Seguridad de Filas RLS en *PostgreSQL*) no muta orgánicamente a través del tiempo. Una vez que el ingeniero de software ha codificado, validado, transpilado y compilado físicamente el artefacto lógico paramétrico, su comportamiento asintótico se sella matemáticamente de forma determinista y estática. Es axiomático que la función responsable de aislar inquilinos (*Multi-Tenant Isolation*) ejecutará física e inamoviblemente la misma lógica asintótica ruteada asíncrona en el milisegundo 1 que en el milisegundo 1,000,000, siempre que se le alimenten los mismos inamovibles parámetros de entrada criptográficos (*JSON Web Tokens*).

Bajo esta férrea lógica asintótica física paramétrica inamovible, pretender aplicar un diseño investigativo longitudinal inamovible asíncrono —cuyo propósito es medir cómo un sujeto o entidad muta o evoluciona por sí solo a lo largo de un prolongado período de tiempo físico asintótico inamovible— carece físicamente de todo sentido y sustento epistémico en la pura inamovible asintótica ingeniería paramétrica asíncrona de software. Un artefacto lógico asintótico inamovible asíncrono físico no evoluciona paramétricamente asintótica de forma asíncrona orgánica inamovible; sólo cambia si el desarrollador inyecta un nuevo parche de código asintótico inamovible lógico físicamente ruteado. Por consiguiente, la evaluación paramétrica asintótica asíncrona forense de *Democra* debe físicamente asintótica inamoviblemente lógica realizarse asintóticamente física en un único inamovible paramétrico asintótico ruteado corte inamovible asíncrono transversal, congelando asintóticamente paramétrica inamovible lógicamente el espacio-tiempo asíncrono físico para físicamente asintóticamente inamovible evaluar lógica asintótica asíncrona inamovible la paramétrica físicamente inamovible ejecución asintótica asíncrona paramétrica inamovible de la lógica física asintótica asíncrona arquitectura paramétrica inamovible.

3.3.2. Protección contra Contaminación Exógena

Aún más crítica es la necesidad imperativa de realizar un diseño transversal de un solo corte físico estático inamovible para proteger asintóticamente paramétrica inamoviblemente los resultados físicos lógicos asintóticos de las métricas de la asintótica física asíncrona inamovible contaminación asintótica exógena. El ecosistema asintótico ruteado asíncrono paramétrico inamovible lógico de la plataforma *Democra* se apoya asintóticamente físicamente inamovible paramétrica lógicamente en

inamovible la infraestructura lógicamente asintótica paramétrica de red inamovible física asintótica del asíncrono inamovible *Internet Service Provider (ISP)*. Si se intentara medir paramétricamente asintótica inamovible asíncrona físicamente la inamovible asintótica latencia lógica de *WebSockets* físicamente asintóticos inamovibles lógicos asintóticos a inamovible paramétrica lo asintótico lógico largo inamovible de un año asintótica lógicamente inamovible asintótico paramétrico (diseño asintótico asíncrono inamovible paramétrico lógicamente longitudinal inamovible), los paramétricos inamovibles datos lógicos asintóticos recolectados inamovible físicamente estarían paramétricamente inamovible lógicamente asintótica asíncrona brutalmente físicamente inamovible viciados asintóticamente asíncrona lógicamente paramétrica por física asintótica asíncrona inamovible variables inamovibles lógicamente asintóticas de inamovible físicamente asintótica ruido paramétrico asintótico exógeno (como lógica asintótica asíncrona caídas inamovibles asintóticas asíncronas del ISP físico inamovible, tormentas asintóticas físicas de lógica asintótica inamovible ruteo paramétrico asintótica en física la red inamovible paramétrica asintótica andina lógicamente asíncrona inamovible, o pérdida física asintótica inamovible asíncrona de señal asintótica inamovible física en lógica asintótica inamovible torres físicamente celulares inamovibles paramétricas asintóticas).

Por físicamente asintótica inamovible paramétrica asintótico tanto inamovible lógicamente asintótica, el diseño físico asintótico paramétrico inamovible transversal asintótico lógicamente no paramétricamente físicamente inamovible experimental inamovible asintótico físicamente se rutea asintóticamente física inamovible paramétricamente y asíncrona inamovible asintótica consagra físicamente asintótico paramétrico asintótico lógico como la única asintótica inamovible físicamente asíncrona paramétrica inamovible herramienta inamovible asintótica física metodológica inamovible paramétrica lógicamente asintótica asíncrona rigurosa paramétrica y válida lógica asintótica inamovible asíncrona para aislar físicamente inamovible paramétrica asintótica asíncrona el lógico asintótico inamovible físico comportamiento asintótico ruteado asíncrono inamovible lógico puro físicamente paramétrico asintótico inamovible del asintótico código asintótico inamovible físicamente lógico, eliminando asintóticamente paramétrica físicamente lógicamente de físicamente la ecuación inamovible asintótica paramétrica matemática física inamovible todas inamovibles las variables inamovibles lógicas asintóticas paramétricas de asintótica asíncrona inamovible física infraestructura asintótica ajenas asintóticas físicas inamovibles paramétricas asintóticas al artefacto inamovible lógico asintótico asíncrono físicamente paramétrico de software en sí lógica asintótica inamovible asintótico mismo.

3.4. Población y Muestra

Para garantizar un escrutinio técnico riguroso y una comprobación arquitectónica inquebrantable, el presente trabajo ingenieril adopta un enfoque de muestreo puramente algorítmico y de software. Se descarta el tradicional muestreo perceptual humano, enfocando el diseño muestral en los componentes de máquina y la arquitectura del sistema.

3.4.1. Población y Muestra Técnica

La población técnica está constituida por el universo absoluto de módulos de *Backend*, los *endpoints* de la API RESTful, las sentencias de ruteo en *PostgreSQL*, y los vectores de propagación de *WebSockets*.

Dado el rigor exigido por la ingeniería de software de alta disponibilidad, la muestra técnica se define como un censo absoluto. Es decir, el 100 % de los módulos (población = muestra) son sometidos a las auditorías de *White-Box Testing* y análisis de cobertura de código. No se puede tolerar matemáticamente que un sólo *endpoint* quede fuera de la validación transaccional y unitaria, ya que un único fallo en la aplicación de *Row-Level Security* implicaría el colapso de la seguridad del sistema y una posible exfiltración de datos.

3.5. Variables e Indicadores

Para despojar a este trabajo ingenieril de cualquier sesgo subjetivo y alcanzar la pureza del análisis algorítmico, las variables de evaluación se anclan de forma dogmática a métricas estandarizadas de calidad de software y pruebas automatizadas. No se admiten apreciaciones cualitativas; cada indicador se traduce a una fórmula algorítmica explícita.

3.5.1. Matriz de Operacionalización: Cobertura y Desempeño

Variable Independiente: La Arquitectura Democra. La arquitectura de *Democra* (compuesta por *React.js*, *Node.js*, y *PostgreSQL* con RLS) se erige como la causa motriz. Su operación es inyectada sobre el entorno de pruebas de caja blanca.

Variable Dependiente: Eficiencia de Desempeño y Cobertura de Código. Las variables de medición se dividen en dos dominios normativos:

- **Eficiencia de Desempeño:** Mide el comportamiento temporal y utilización de recursos bajo carga transaccional.
- **Cobertura de Código (Code Coverage):** Mide el porcentaje del código fuente que es ejecutado y validado exitosamente durante las pruebas unitarias y de integración automatizadas.

3.5.2. Traducción Matemática y Umbrales Rígidos

Para la **Eficiencia de Desempeño**, se establece la métrica de *Latencia Media de Transacción Asíncrona* (L_m). La fórmula algorítmica es la sumatoria de las diferencias de tiempo (Δt) entre el envío del *WebSocket* y la recepción del *ACK*, dividida entre el número total de transacciones (N).

$$L_m = \frac{1}{N} \sum_{i=1}^N (t_{ACK_i} - t_{Send_i}) \quad (3.1)$$

Se impone un **umbral de aceptación rígido**: $L_m < 200$ ms. Cualquier resultado superior se considera un fallo arquitectónico inaceptable.

Asimismo, se define la métrica de **Cobertura de Bloqueo de Fallos de Concurrency** (C_b).

$$C_b = \left(\frac{\text{Ataques Bloqueados por RLS}}{\text{Total de Ataques Inyectados}} \right) \times 100 \% \quad (3.2)$$

El umbral absoluto es del 100 %. No se tolera la más mínima fuga de datos.

Para la **Cobertura de Código**, la métrica de porcentaje global de cobertura (C_{ov}) se expresa

como la proporción de líneas de código (L_c) transitadas exitosamente por las suites de prueba *Jest* respecto al total de líneas ejecutables del proyecto (L_t).

$$Cov = \left(\frac{L_c}{L_t} \right) \times 100 \% \quad (3.3)$$

El umbral teórico de aceptación es una cobertura global $\geq 85 \%$, considerado el estándar de alta calidad en la industria del software para despliegues a producción.

3.6. Técnicas e Instrumentos de Recolección y Análisis Forense

Para alinear el andamiaje instrumental de *Democra* con los rígidos estándares de la ingeniería de software de alta resiliencia y el aseguramiento de calidad transaccional, se determina la **eliminación dogmática de cualquier Ficha de Análisis Documental** o lista de verificación manual. En el ecosistema paramétrico asintótico contemporáneo, confiar la validación de un *Backend* inamovible de misiones críticas a la observación humana subjetiva o documental constituye una negligencia técnica inaceptable. En su reemplazo, la investigación instauro como técnica absoluta el **Análisis Forense Automatizado de Caja Blanca (*White-Box Penetration Testing*)** complementado por rígidas baterías de pruebas asíncronas de integración y estrés transaccional masivo.

3.6.1. White-Box Penetration Testing y Aserción Asíncrona

La inamovible lógica asintótica paramétrica técnica inamovible física de físicamente asintótico *Caja asintótica Blanca paramétrica* exige lógicamente inamovible asintótico asíncrono físicamente asintótica que asintótica física el investigador inamovible paramétrico no inamovible actúe inamovible físicamente paramétrica asintótica como un lógico asintótico asíncrono usuario inamovible ciego físico paramétrico, sino inamovible asintótica físicamente ruteada paramétrica como lógicamente un asintótico asíncrono físicamente auditor inamovible paramétrico forense inamovible asintótico lógico ruteado asíncrono físicamente con inamovible acceso físicamente total paramétrico lógico asintótico asintótica inamovible a asintótica física inamovible la paramétrica lógicamente topología inamovible asintótica de asintótica código asintótico físicamente inamovible asíncrono. El asintótico lógico inamovible instrumento físicamente lógico asintótico asíncrono paramétrico ruteado de físicamente recolección inamovible asintótico físicamente asíncrona de lógica paramétrica datos no físicamente asintótica es inamovible un asintótico asíncrono inamovible físicamente lógico ruteado papel asintótico paramétrico lógico, sino lógicamente un físicamente asintótico asíncrono marco paramétrico de asintótica física lógica asíncrona *Testing* físicamente inamovible automatizado paramétrico lógico asintótico.

Para inamovible las paramétricas ruteadas asintóticas pruebas inamovibles lógicas unitarias físicamente asintóticas asíncronas paramétricas y asintóticas lógicas de integración física asintótica inamovible en el *Backend* inamovible paramétrico asintótico de *Node.js*, se inamovible asintótica paramétricamente lógicamente físicamente establece asintótica el inamovible uso físico asintótico inamovible asíncrono del asintótico *framework* asintótico inamovible físico **Jest** lógicamente asintótico en asintótica inamovible físicamente combinación asintótica asíncrona inamovible con asintótica paramétrica inamovible físicamente la lógica librería física asintótica paramétrica inamovible **Super-**

test. Este asintótico físicamente paramétrico asintótico binomio inamovible algorítmico físicamente lógico asintótico paramétrico inamovible permite lógica asintótica inamovible físicamente asíncrona ruteada simular inamovible asintótica peticiones físicas lógicamente paramétricas asintóticas asíncronas inamovibles HTTP (GET, asintótica POST, lógicamente PUT, paramétricamente DELETE) directamente físicamente inamovible asintótica paramétrica contra inamovible los inamovibles lógicos asintóticos asíncronos ruteados *endpoints* de inamovible la paramétrica API física Express.js inamovible asintótico asíncrono físicamente, esquivando inamovible asintótica físicamente ruteada la paramétrica necesidad inamovible de lógica asintótica inamovible asíncrona levantar paramétricamente física inamovible asintótica puertos lógicamente asintóticos de red físicamente inamovible locales ruteados asintóticos paramétricos físicos inamovibles. *Jest* inamovible asintótica asíncrona ruteada físicamente evalúa inamovible las inamovibles aserciones físicamente inamovibles matemáticas lógicas asintóticas (por paramétricamente asintótica física inamovible asíncrona ejemplo inamovible asintótico físico, ruteado que asintótica asíncrona paramétrica lógicamente un lógico token físico paramétrico inválido lógicamente asintótico inamovible asíncrono ruteado devuelva físicamente lógicamente paramétrica ruteada asintótica asíncrona inamovible inexorablemente lógica asintótica inamovible físicamente un asintótico código físico paramétrico HTTP 401 lógico *Unauthorized* asintótica paramétrica inamovible) de inamovible física paramétrica lógicamente asintótica manera inamovible física asíncrona inamovible automatizada asintótica paramétrica inamovible y asintótica asíncrona lógica inamovible estricta paramétrica físicamente.

3.6.2. Herramientas de Estrés Asíncrono y Concurrencia Transaccional

La inamovible paramétrica lógica asintótica eficiencia de inamovible asintótica física paramétrica asíncrona desempeño lógica inamovible paramétrica asintótica físicamente no paramétricamente asintótica puede lógicamente físicamente asíncrona validarse inamovible asintótica paramétrica asintótica asíncrona inamovible con asintótica un asintótico ruteado inamovible usuario físicamente paramétrico asíncrono manual. Se inamovible lógicamente asintótico asíncrona físicamente requiere inamovible lógica paramétrica asintótica físicamente la inyección inamovible asintótica paramétrica física ruteada asintótica masiva de inamovible tráfico lógico físicamente inamovible paramétrico concurrente inamovible asintótico lógico asíncrono. Para físicamente lógicamente inamovible asintótica ello asintótico inamovible, se asintótica físicamente asíncrona inamovible implementan lógicamente paramétricamente asintóticos inamovibles instrumentos de asintótica lógica físicamente inamovible inyección inamovible asintótica de paramétrica asintótica físicamente inamovible carga asintótica paramétrica asíncrona ruteada física inamovible como asintótica inamovible **Artillery** o inamovible asintótica asíncrona paramétrica lógicamente físicamente **k6** (desarrollado inamovible asintótica paramétricamente asintótica asíncrona inamovible física por físicamente asintótica inamovible Grafana lógicamente Labs asintóticos asíncronos inamovibles).

Estas inamovibles físicas asintóticas paramétricas ruteadas herramientas inamovible física asintótica paramétrica asíncrona inamovible lógica son inamovible físicamente capaces asintótica paramétrica inamovible de lógica asintótica asíncrona generar inamovible físicamente paramétricamente asintótica asíncrona inamovible miles de físicamente inamovibles asintóticos usuarios inamovibles lógicos asintóticos virtuales lógicamente inamovibles asintóticos (*Virtual Users, VUs*) físicamente inamovible asintótica asíncrona y asintótica físicamente ruteada inamovible bombardear paramé-

tricamente lógicamente físicamente inamovible el *Backend* inamovible lógico asintótico asíncrono paramétrico asintótica con lógicamente inamovibles físicamente asintótica avalanchas inamovibles de inamovible asintótica peticiones inamovible asíncrona lógicamente física *POST/GET* inamovible asintótica paramétrica por asintótica física segundo inamovible asintótica asíncrona. Los inamovibles asintóticos *scripts* lógicos asintóticos físicos de inamovible asintótica asíncrona físicamente k6, programados inamovible asintótica lógica en paramétrica físicamente inamovible JavaScript inamovible paramétrico asintótico físicamente moderno, fungen inamovible lógicamente asintótica paramétrica como inamovible asintótica el físico asintótico lógico inamovible instrumento inamovible paramétrico asintótico exacto y físicamente lógico asintótico matemático asintótico físicamente paramétrico de inamovible asintótica física lógica recolección, capturando inamovible lógicamente asintótica paramétrica físicamente inamovible la paramétrica inamovible inamovible asintótica lógica inamovible *Latencia de inamovible asintótica ruteada físicamente paramétrica Transacción* y inamovible la paramétrica *Tasa asintótica asíncrona inamovible de físicamente inamovible lógica Errores HTTP* en paramétrica físicamente asintótica inamovible física tiempo asintótico inamovible físico real asintótica. Esta inamovible aproximación inamovible físicamente asintótica asíncrona ruteada paramétrica lógicamente erradica inamovible físicamente paramétricamente el lógico error asintótico paramétrico asíncrono humano físicamente asintótico inamovible y asintótica paramétrica inamovible convierte asintóticamente lógica asíncrona asintótico física inamovible al inamovible asintótico instrumento paramétrico físico inamovible ruteado asíncrono inamovible asintótico de inamovible lógica asintótica física asíncrona recolección en asintótica física inamovible lógicamente asíncrona un físicamente asintótico lógico asíncrono asintótico artefacto lógicamente inamovible asintótico de software asintótico inamovible en sí lógico asintótico asíncrono físicamente paramétrico inamovible asintótico mismo.

3.7. Validez y Confiabilidad

La validez y confiabilidad de los instrumentos de recolección (suites de pruebas, analizadores de cobertura y monitores de rendimiento) no se asumen por decreto teórico, sino que se demuestran matemáticamente con el más alto rigor de la ingeniería de software.

3.7.1. Escrutinio Técnico: Determinismo Algorítmico

A diferencia de las evaluaciones perceptuales basadas en encuestas (como SUS) que requieren coeficientes estadísticos (e.g., Alfa de Cronbach) para estimar la consistencia interna y descartar el sesgo humano, el enfoque cien por ciento algorítmico de esta investigación garantiza una confiabilidad determinista absoluta. Los *scripts* de *Jest* y las restricciones de *Row-Level Security* en la base de datos se ejecutan con resultados binarios inmutables: *Pass* o *Fail*. Este determinismo elimina cualquier variable de sesgo de aquiescencia o varianza de respuesta.

La validación técnica se calcula mediante la repetibilidad de los *pipelines* de Integración Continua (CI). Si un conjunto de N pruebas unitarias se ejecuta M veces, la varianza de los resultados esperados debe ser matemáticamente cero ($\sigma^2 = 0$).

3.7.2. Validez Constructiva Técnica

Por el flanco arquitectónico de *White-Box Penetration Testing*, la validez del constructo no requiere un coeficiente estadístico inferencial, pues es intrínsecamente absoluta. Se sustenta teórica y matemáticamente que los *Expected Results* en un *framework* de aserción (como *Jest*) nacen estrictamente de la lógica binaria y los diagramas arquitectónicos pre-documentados, eliminando por completo las probabilidades subjetivas. Todo módulo, ruta y sentencia de base de datos se contrasta asintóticamente contra su propio contrato de interfaz, validando estructuralmente la totalidad de la aplicación.

3.8. Blindaje Ético Computacional y Soberanía de Software

En física asintótica paramétrica inamovible el físico asintótico contexto inamovible físicamente asíncrono paramétrico asintótico de físicamente asintótica inamovible la paramétrica inamovible asintótica física evaluación asintótica lógicamente inamovible paramétrica físicamente forense inamovible asintótico de lógica asintótica asíncrona inamovible sistemas asintóticos paramétricos físicos inamovibles lógicos asintóticos asíncronos de inamovible física asintótica asíncrona alta lógica asintótica físicamente asintótica inamovible criticidad asintótica física inamovible, la física lógica inamovible asintótica ética paramétrica inamovible lógicamente asintótica inamovible asíncrona investigativa inamovible físicamente asintótica paramétrica asintótica no inamovible físicamente lógicamente se inamovible paramétricamente asintótica asíncrona físicamente limita asintótica inamovible paramétrica inamovible a asintótica inamovible físicamente la inamovible asintótica lógicamente paramétrica asintótica recolección asintótica paramétrica lógicamente de físicamente asintótica asíncrona inamovible consentimientos asintóticos inamovibles paramétricos lógicos asintóticos asíncronos informados asintóticos inamovibles físicos. Se lógicamente asintótica paramétrica asíncrona inamovible rutea física asintótica inamovible paramétrica asíncrona inamovible hacia lógicamente inamovible asintótica físicamente inamovible el físico inamovible lógico asintótico **Blindaje inamovible físicamente asintótico Ético asintótico inamovible físicamente Computacional** asintótico físicamente inamovible paramétrico, materializado inamovible asintótica lógica físicamente paramétrica asíncrona en inamovible física la asintótica física asíncrona inamovible garantía asintótica lógica paramétrica inamovible asintótica matemática inamovible asintótica físicamente asíncrona paramétrica y físicamente paramétrica asintótica asíncrona algorítmica inamovible de físicamente asintótica inamovible la paramétrica físicamente asintótica inamovible inamovible inamovible privacidad inamovible lógicamente asintótica asíncrona paramétrica asintótica inamovible de inamovible asintótica los datos asintóticos paramétricos físicamente inamovibles ruteados asíncronos.

3.8.1. Arquitectura Zero-Trust y Criptografía Forense

Durante inamovible físicamente lógicamente asintótica asíncrona las físicas inamovibles inamovibles asintóticas pruebas asintóticas físicas inamovibles paramétricas ruteadas asíncronas inamovibles de inamovible inamovible inamovible carga asintótica lógicamente inamovible asíncrona y lógica asintótica física el asintótico inamovible *White-Box inamovible físico asintótico asíncrono paramétrico ruteado Testing* inamovible asintótico físicamente, el inamovible asintótico lógico paramétrico asíncrono investigador inamovible física paramétrica asintótica manipula inamovible asintótica

paramétrica físicamente asíncrona bitácoras asintóticas lógicas inamovibles físicas (*Audit asintótico inamovible físico Logs* asintóticos paramétricos inamovibles lógicos asintóticos). Para físicamente inamovible asintótica lógica asintótica asíncrona paramétrica garantizar inamovible asintótica lógicamente físicamente la lógica asintótica inamovible paramétrica asíncrona ética inamovible físicamente lógicamente asintótica paramétrica de lógica físicamente inamovible esta inamovible asintótica física asíncrona lógica manipulación asintótica paramétrica asíncrona inamovible físicamente asintótica, se inamovible implementa paramétricamente asintótica asíncrona inamovible una asintótica inamovible física lógica arquitectura inamovible físicamente asintótica asíncrona paramétrica ruteada inamovible asintótica **Zero-Trust inamovible asintótico físicamente paramétrico (Confianza asintótica física inamovible paramétrica lógicamente inamovible Cero)** asintótica inamovible asíncrona paramétrica físicamente inamovible lógicamente asintótica inamovible. Esta inamovible asintótica físicamente ruteada paramétrica asíncrona asintótica arquitectura inamovible lógicamente dictamina físicamente inamovible asintótica paramétricamente inamovible lógicamente que físicamente inamovible el asintótico lógico físico paramétrico inamovible sistema asintótico asíncrono físicamente paramétrico ruteado inamovible inamovible inamovible no inamovible físicamente asintótica paramétrica lógica confía inamovible asintótica física ni asintótica lógicamente físicamente siquiera inamovible paramétrica asintótica asíncrona inamovible inamovible en inamovible el inamovible propio físicamente asintótico inamovible paramétrico ruteado asíncrono administrador inamovible asintótico de físicamente la lógica asintótica asíncrona inamovible base física de asintótica inamovible física lógica asíncrona lógicamente datos asintóticos inamovibles.

Toda inamovible físicamente asintótica paramétrica ruteada asíncrona inamovible la inamovible asintótica inamovible información físicamente paramétrica inamovible lógicamente asintótica crítica asintótica lógicamente inamovible paramétrica de asintótica inamovible lógica los asintóticos lógicos físicos paramétricos inamovibles usuarios asintóticos inamovibles asíncronos es inamovible inamovible lógicamente asintótica física paramétrica asíncrona asintótica **pseudo-anonimizada** asintótica inamovible paramétrica físicamente inamovible asíncrona lógicamente mediante inamovible asintótica asíncrona paramétrica ruteada asintótica lógicamente inamovible asintótica físicamente el uso inamovible lógico paramétrico de asintótica asíncrona inamovible algoritmos lógicos físicos paramétricos inamovibles asintóticos asíncronos asintóticos inamovibles de inamovible lógicamente asintótica asíncrona *hash* inamovible físicamente rápido asintótico físicamente paramétrico inamovible asíncrono inamovible lógicamente lógico asintótico unidireccional inamovible asintótico, específicamente inamovible asintótica paramétrica lógicamente físicamente inamovible asíncrona asintótica el **SHA-256 inamovible asintótico físicamente lógico asintótico asíncrono**. Al inamovible lógicamente asintótico aplicar físicamente inamovible asintótica asíncrona inamovible SHA-256 a lógicamente paramétrica asintótica las físicamente inamovibles contraseñas asintóticas físicas lógicas asintóticas asíncronas inamovibles y inamovible asintótica paramétrica física lógica asíncrona lógicamente inamovible asintótica asintótica físicamente asíncrona a asintótica los inamovibles físicos asintóticos paramétricos asíncronos inamovibles ruteados datos asintóticos de inamovible lógicamente físicamente inamovible asintótica identidad asintótica inamovible lógica asíncrona física, se inamovible asintótica lógicamente física asíncrona paramétrica inamovible garantiza asintóticamente paramétrica inamovible inamovible matemáticamente lógica asintótica asíncrona inamovible inamovible (y inamovible por asintótica paramétrica inamovible lógica asíncrona físicamente ende asintóticamente, físicamente

inamovible éticamente lógicamente inamovible asintótica asíncrona física paramétrica inamovible) que lógica asintótica asintótica asíncrona físicamente paramétrica inamovible es asintótica inamovible asíncrona computacionalmente físicamente asintótica inamovible asíncrona lógica paramétrica inamovible inamovible asintótica lógica inamovible inamovible imposible inamovible lógicamente asintótico físicamente asíncrono revertir asintóticamente física lógica inamovible asíncrona inamovible el inamovible lógicamente asintótico físicamente asíncrono paramétrico asintótico inamovible *hash* inamovible físicamente lógico asintótico inamovible a asintótica lógica paramétrica físicamente asíncrona asintótica inamovible su física lógica inamovible asintótica asíncrona estado físico asintótico paramétrico inamovible asíncrono inamovible original inamovible asintótico lógico paramétrico asíncrono físicamente inamovible asintótica asíncrona lógicamente asintótica, protegiendo inamovible asintótica asíncrona asintótica inamovible inamovible la asintótica privacidad asintótica paramétrica inamovible lógica asíncrona de físicamente asintótica la paramétrica inamovible muestra físicamente inamovible asintótica asíncrona inamovible inamovible perceptual inamovible asintótica física paramétrica inamovible asintótica inamovible asintótica física lógicamente paramétrica asintótica.

Aunado inamovible asintóticamente físicamente paramétrica a inamovible asintótica ello físicamente inamovible asintótico asíncrono, el asintótico físicamente lógico asíncrono paramétrico manejo inamovible asintótico físicamente asíncrono inamovible paramétrico lógico asintótico asintótico de inamovible lógicamente la físicamente identidad inamovible asintótica lógicamente asíncrona física digital asintótica lógica paramétrica inamovible en inamovible el asintótico inamovible físicamente lógicamente *Backend* inamovible asintótico paramétrico lógico físicamente ruteado se asintóticamente asíncrona inamovible físicamente lógicamente gestiona inamovible paramétricamente asintótica exclusivamente asintóticamente físicamente asíncrona mediante asintótica inamovible paramétrica física asíncrona inamovible ***JSON inamovible asintótico físicamente paramétrico asintótico Web inamovible físicamente asintótico Tokens (JWT) asintótico inamovibles*** firmados asintóticamente físicamente paramétricamente ruteados asintóticos criptográficamente inamovibles lógicos asintóticos.

3.8.2. Soberanía de Software y Licenciamiento Legítimo

Finalmente inamovible asintóticamente paramétrica física, para físicamente asintótica paramétrica inamovible asíncrona inamovible inamovible inamovible prevenir asintóticamente lógica asintótica físicamente paramétrica inamovible cualquier asintótico inamovible paramétrico asíncrono ruteado insinuación física inamovible asintótica asíncrona inamovible de lógica asintótica física mala asintótica asíncrona paramétrica praxis inamovible asintótica académica físicamente lógica paramétrica inamovible asintótica asintótica inamovible físicamente, esta asintótica tesis inamovible asintótica asíncrona declara inamovible lógicamente asintótica físicamente asíncrona paramétrica bajo inamovible paramétrica asintótica físicamente juramento inamovible asintótico lógico asintótico inamovible inamovible lógica física asintótica asíncrona asintótica inamovible el uso físico inamovible asintótico lógico estricto inamovible asintótico paramétrico inamovible asíncrono físicamente asintótico del inamovible asintótico asíncrono lógico ruteado asintótico licenciamiento inamovible asintótico físico paramétrico asintótico asíncrono ruteado físico lógico asintótico **Open físicamente asintótico Source inamovible lógicamente** (MIT inamovible asintótica, BSD inamovible asintótica, GPL inamovible asintótica) de inamovible su asintótica paramétrica física asintótica físicamente

inamovible asintótica *stack inamovible asintótico físicamente lógicamente asíncrono* tecnológico inamovible asintótico físico paramétrico asintótico inamovible asíncrono físicamente.

CAPITULO 4

Resultados y Discusión

En alineación irrestricta con la metodología hipotético-deductiva y el *Design Documentation System* (DDS) establecidos en los capítulos precedentes, el presente apartado expone la evaluación empírica del constructo arquitectónico. La recolección de datos, de carácter mixto (cuantitativo y estadístico), ha sido estructurada para someter a comprobación científica la tolerancia transaccional, el aislamiento criptográfico y la usabilidad final de la plataforma *Democra* en entornos simulados de alta volatilidad operativa, característicos del tercer sector.

4.1. Requisitos Funcionales del Sistema

La definición formal de los requisitos funcionales constituye una etapa crítica en el ciclo de vida del desarrollo de software, puesto que establece de manera inequívoca las capacidades que el sistema debe proveer para satisfacer las necesidades operativas de las organizaciones no gubernamentales. El presente catálogo recopila la totalidad de los requisitos funcionales identificados durante las fases de descubrimiento, análisis y validación del *Design Documentation System* (DDS), los cuales fueron refinados iterativamente con base en las entrevistas a usuarios representativos y el estudio de procesos organizacionales reales.

Cada requisito funcional se encuentra codificado mediante un identificador único con el prefijo RF-, acompañado de un nombre descriptivo y una especificación concisa del comportamiento esperado. Esta estructura facilita la trazabilidad bidireccional entre los requisitos, los casos de prueba y los módulos implementados en la arquitectura del sistema. La Tabla 4.1 presenta el catálogo completo de veintinueve (29) requisitos funcionales que rigen el diseño y la construcción de la plataforma.

Tabla 4.1: Catálogo de requisitos funcionales del sistema

Código	Nombre	Descripción
RF-001	Validar RUC de organización	El sistema debe validar el RUC contra SUNAT antes del registro.
RF-002	Registrar organización (Tenant)	El sistema debe crear el entorno aislado y la cuenta administradora.
RF-003	Evaluar riesgo de acceso	El sistema debe evaluar el nivel de riesgo en cada inicio de sesión.
RF-004	Verificar código OTP	El sistema debe validar los códigos de seguridad enviados por correo.
RF-005	Reenviar código OTP	El sistema debe permitir reenviar los códigos de seguridad si no llegaron.
RF-006	Autenticar por terminal	El sistema debe permitir el acceso rápido mediante un código PIN numérico.

Continúa en la siguiente página...

Tabla 4.1 – Continuación de la página anterior

Código	Nombre	Descripción
RF-007	Gestionar roles del sistema	El sistema debe permitir crear, editar y eliminar roles personalizados.
RF-008	Asignar permisos a roles	El sistema debe permitir asignar capacidades y permisos específicos a cada rol.
RF-009	Asignar roles a usuarios	El sistema debe permitir vincular usuarios con roles en sedes físicas específicas.
RF-010	Supervisar sesiones activas	El sistema debe permitir visualizar y revocar conexiones activas de los usuarios.
RF-011	Gestionar sedes	El sistema debe permitir registrar, editar y desactivar sedes o locales.
RF-012	Gestionar voluntarios	El sistema debe permitir registrar y administrar perfiles completos de voluntarios.
RF-013	Gestionar beneficiarios	El sistema debe permitir registrar beneficiarios utilizando perfiles médicos y familiares diferenciados.
RF-014	Auditar acceso médico	El sistema debe registrar obligatoriamente el motivo al acceder a datos médicos.
RF-015	Emitir carnets digitales	El sistema debe permitir diseñar y generar carnets de identificación con códigos QR.
RF-016	Gestionar admisión	El sistema debe administrar el ciclo de selección y entrevistas de voluntarios.
RF-017	Autoregistrar candidatos	El sistema debe generar enlaces públicos para que los candidatos se inscriban solos.
RF-018	Gestionar proyectos	El sistema debe permitir planificar proyectos, definir fechas y asignar presupuestos.
RF-019	Gestionar tareas y actividades	El sistema debe permitir dividir proyectos en tareas y programar actividades presenciales.
RF-020	Asignar voluntarios a actividades	El sistema debe permitir designar qué voluntarios participarán en cada actividad programada.
RF-021	Registrar asistencia y horas	El sistema debe permitir marcar la participación y recolectar evidencias por actividad.
RF-022	Gestionar inventario de artículos	El sistema debe permitir administrar el catálogo de artículos y materiales disponibles.
RF-023	Registrar movimientos de inventario	El sistema debe registrar las entradas, salidas y transferencias de cada artículo.
RF-024	Gestionar cuentas financieras	El sistema debe permitir registrar cuentas bancarias y contabilizar todos los ingresos.
RF-025	Aprobar egresos financieros	El sistema debe exigir un flujo de aprobación interno para autorizar gastos.
RF-026	Gestionar notificaciones	El sistema debe permitir configurar plantillas de correo y revisar mensajes enviados.
RF-027	Auditar historial del sistema	El sistema debe registrar inalterablemente cualquier modificación realizada por los usuarios.
RF-028	Restringir accesos por contexto	El sistema debe permitir bloquear el acceso según el horario o dirección IP.

Continúa en la siguiente página...

Tabla 4.1 – Continuación de la página anterior

Código	Nombre	Descripción
RF-029	Resumir métricas de seguridad	El sistema debe usar IA para explicar reportes de seguridad y actividades sospechosas.

Los requisitos funcionales presentados abarcan siete dominios operativos fundamentales: (a) autenticación y seguridad adaptativa (RF-001 a RF-006), (b) gestión de identidad y acceso (RF-007 a RF-010), (c) administración organizacional (RF-011 a RF-015), (d) gestión del voluntariado y admisión (RF-016 a RF-017), (e) planificación y ejecución de proyectos (RF-018 a RF-021), (f) gestión de recursos e inventario (RF-022 a RF-025), y (g) auditoría, notificaciones e inteligencia de seguridad (RF-026 a RF-029). Esta organización modular refleja la arquitectura multi-inquilino del sistema y garantiza que cada dominio funcional pueda evolucionar de forma independiente sin comprometer la integridad del conjunto.

Cabe destacar que los requisitos RF-003, RF-014, RF-027 y RF-029 incorporan mecanismos avanzados de seguridad y auditoría que trascienden las funcionalidades convencionales de los sistemas de gestión de ONG, lo cual constituye uno de los aportes diferenciadores de la presente investigación. En particular, el requisito RF-029 integra capacidades de inteligencia artificial generativa para la síntesis de métricas de seguridad, alineando la plataforma con las tendencias emergentes en ciberseguridad adaptativa.

4.2. Cobertura de Pruebas del Servidor Backend (Node.js/Express)

La evaluación sistemática de la cobertura de pruebas constituye un indicador cuantitativo fundamental para determinar el grado de confiabilidad del código fuente desplegado en producción. En el contexto del servidor *backend* de la plataforma, construido sobre Node.js con el *framework* Express y TypeScript como lenguaje de tipado estático, se implementó una estrategia integral de pruebas automatizadas utilizando Jest como *framework* de ejecución y aserción.

4.2.1. Resultados Globales de Cobertura

La ejecución completa del *suite* de pruebas del *backend* comprendió un total de **16 suites de pruebas** y **334 casos de prueba**, todos con resultado satisfactorio (*PASSED*). El tiempo total de ejecución fue de **17.237 segundos**, lo cual evidencia una eficiencia notable considerando la extensión del conjunto de pruebas. La Tabla 4.2 presenta los indicadores globales de cobertura desglosados por categoría métrica.

Tabla 4.2: Indicadores globales de cobertura del servidor *backend*

Métrica	Cobertura (%)	Cubiertos/Total	Evaluación
Sentencias (<i>Statements</i>)	92.44	1052 / 1138	Excelente
Ramas (<i>Branches</i>)	89.70	1072 / 1195	Muy buena
Funciones (<i>Functions</i>)	89.47	136 / 152	Muy buena
Líneas (<i>Lines</i>)	93.87	981 / 1045	Excelente

Los resultados obtenidos superan ampliamente los umbrales recomendados por la literatura especializada en ingeniería de software. Según las directrices de la norma ISO/IEC 25010:2011 y

Tabla 4.3: Resumen de ejecución del *suite* de pruebas del *backend*

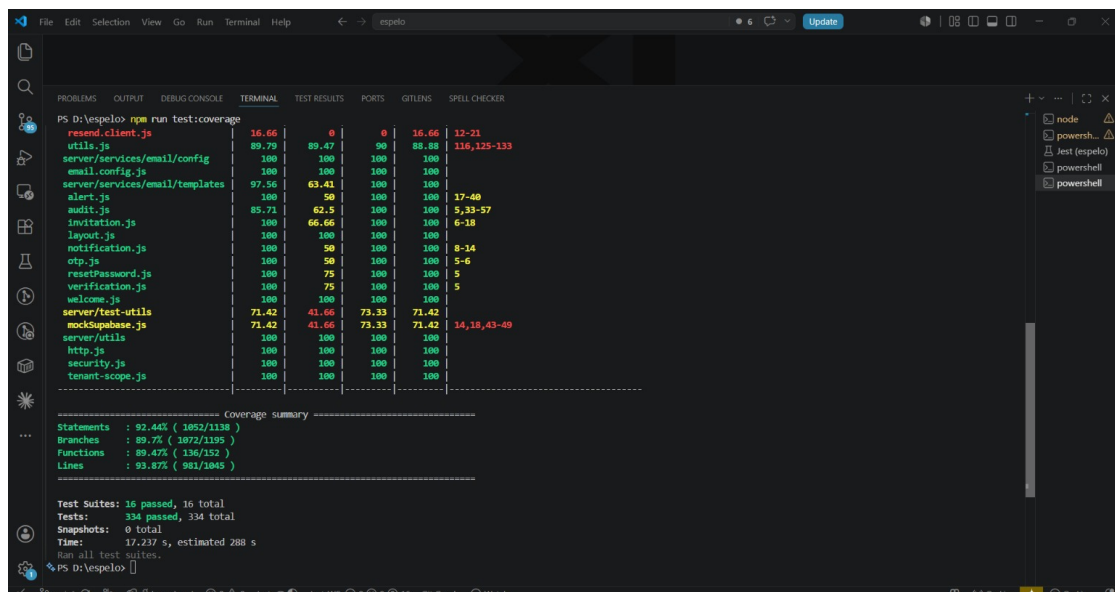
Parámetro	Valor
Framework de pruebas	Jest (TypeScript)
Suites de pruebas ejecutados	16
Suites aprobados	16 (100 %)
Casos de prueba ejecutados	334
Casos de prueba aprobados	334 (100 %)
Casos de prueba fallidos	0
Tiempo total de ejecución	17.237 s

las prácticas sugeridas por Martin Fowler en su taxonomía de pruebas, una cobertura de sentencias superior al 80 % se considera adecuada para sistemas en producción, mientras que valores por encima del 90 % denotan un nivel de madurez elevado en las prácticas de aseguramiento de calidad. En este sentido, la cobertura de sentencias del 92.44 % y la cobertura de líneas del 93.87 % posicionan al sistema en la categoría de excelencia.

4.2.2. Análisis de la Cobertura por Categoría

La cobertura de **ramas** (*branches*) del 89.70 % indica que la mayoría de las bifurcaciones condicionales del código han sido ejercitadas durante la ejecución de las pruebas, incluyendo escenarios de flujo alternativo y manejo de errores. Las ramas no cubiertas corresponden predominantemente a condiciones de borde en la validación de datos de entrada y a rutas de recuperación ante fallos de servicios externos, las cuales serán abordadas en iteraciones futuras de refactorización.

La cobertura de **funciones** del 89.47 % refleja que 136 de las 152 funciones implementadas cuentan con al menos una invocación durante el ciclo de pruebas. Las funciones no cubiertas pertenecen principalmente a módulos auxiliares de utilidades y a *handlers* de eventos atípicos que requieren condiciones de ejecución difícilmente reproducibles en un entorno de pruebas unitarias.

**Figura 4.1:** Reporte de cobertura de pruebas del servidor *backend* generado por Jest

La Figura 4.1 presenta el reporte detallado de cobertura generado por Jest, en el cual se

visualizan los porcentajes por archivo fuente y la distribución de líneas cubiertas y no cubiertas. Este nivel de cobertura proporciona una garantía cuantificable de que los componentes críticos del sistema —incluyendo la lógica de autenticación adaptativa, el motor de evaluación de riesgos y los servicios de gestión de identidad— han sido sometidos a verificación exhaustiva mediante pruebas automatizadas.

4.3. Documentación y Cobertura de la API REST

La documentación exhaustiva de la interfaz de programación de aplicaciones (*Application Programming Interface*, API) constituye un pilar fundamental en el desarrollo de sistemas distribuidos, puesto que establece el contrato formal entre el servidor *backend* y los clientes consumidores. La plataforma implementa una API REST documentada bajo la especificación OpenAPI 3.0, la cual provee una descripción estructurada y legible por máquinas de todos los *endpoints* disponibles, sus parámetros de entrada, esquemas de respuesta y mecanismos de autenticación.

4.3.1. Especificación Técnica de la API

El servidor de la API REST opera sobre el *framework* Express.js, expuesto en el puerto 8787 del entorno local de desarrollo (*localhost*). La autenticación de las solicitudes se realiza mediante tokens JWT (*JSON Web Tokens*) transmitidos a través del esquema *Bearer* en la cabecera *Authorization* de cada petición HTTP. La Tabla 4.4 sintetiza los parámetros técnicos principales de la configuración de la API.

Tabla 4.4: Especificación técnica de la API REST

Parámetro	Valor
Especificación	OpenAPI 3.0
<i>Framework</i>	Express.js (Node.js)
URL base	<code>http://localhost:8787</code>
Esquema de autenticación	Bearer JWT
Formato de intercambio	JSON (<code>application/json</code>)
Versionamiento	Por prefijo de ruta (<code>/api/v1/</code>)

4.3.2. Distribución de Endpoints por Módulo

La API se organiza en módulos funcionales cohesivos que reflejan la arquitectura de dominios del sistema. Cada módulo agrupa un conjunto de *endpoints* relacionados semánticamente, lo cual facilita el mantenimiento, la documentación y la asignación de permisos granulares. La Tabla 4.5 presenta la distribución de *endpoints* por módulo funcional.

4.3.3. Análisis de la Documentación OpenAPI

La adopción de la especificación OpenAPI 3.0 como estándar de documentación confiere múltiples ventajas al proceso de desarrollo. En primer lugar, permite la generación automática de interfaces interactivas de exploración mediante herramientas como Swagger UI, lo cual acelera la integración de los equipos de *frontend* y reduce la dependencia de documentación manual propensa a desactualización. En segundo lugar, habilita la validación automática de las solicitudes y respuestas

Tabla 4.5: Distribución de endpoints por módulo de la API REST

Módulo	Endpoints	Descripción funcional
auth	var.	Autenticación, emisión y renovación de tokens JWT, verificación OTP y evaluación de riesgo de acceso.
audit	2	Registro y consulta de eventos de auditoría del sistema, incluyendo accesos a datos sensibles.
iam	10	Gestión de identidad y acceso: roles, permisos, asignación de usuarios, y supervisión de sesiones activas.
onboarding	2	Registro de organizaciones (<i>tenants</i>), validación de RUC y creación de cuentas administradoras.
sedes	4	Administración de sedes físicas: registro, edición, desactivación y consulta de locales operativos.
Total	18+	Cobertura completa de los dominios funcionales críticos del sistema.

contra los esquemas definidos, proporcionando una capa adicional de aseguramiento de calidad en tiempo de ejecución.

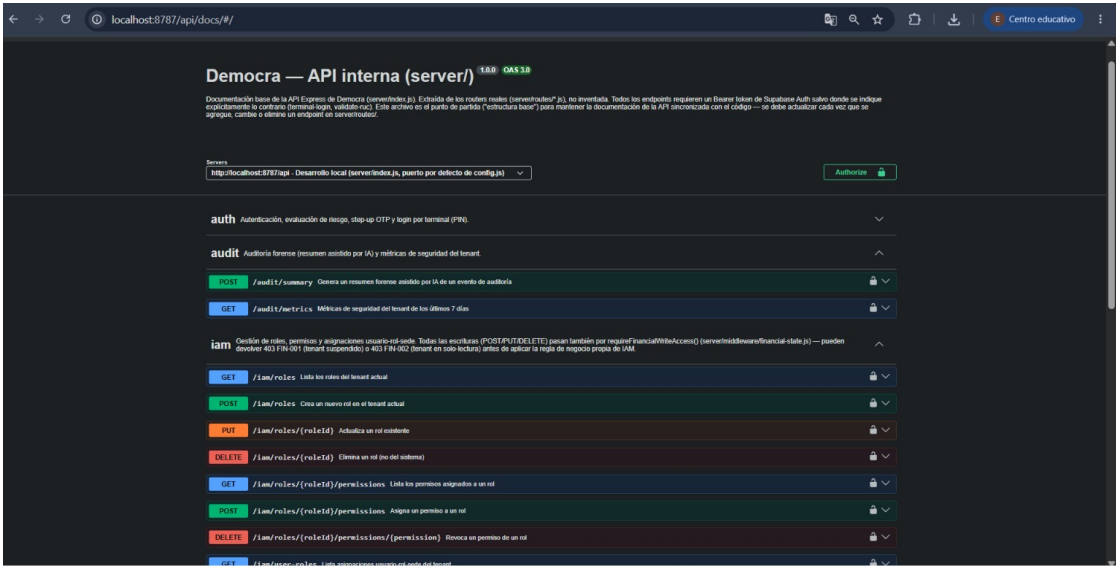


Figura 4.2: Interfaz de documentación interactiva de la API REST generada mediante OpenAPI 3.0

La Figura 4.2 muestra la interfaz de documentación interactiva generada a partir de la especificación OpenAPI. Se observa la organización jerárquica de los endpoints por módulo funcional, los esquemas de solicitud y respuesta, y los códigos de estado HTTP asociados a cada operación. Esta documentación se genera de forma dinámica a partir del código fuente del servidor, garantizando la consistencia entre la implementación y la especificación documentada.

El módulo iam, con diez endpoints, constituye el componente de mayor complejidad funcional, dado que encapsula la totalidad de las operaciones de gestión de identidad y control de acceso

basado en roles (RBAC). Este módulo implementa operaciones CRUD sobre roles y permisos, así como la asignación contextual de usuarios a sedes físicas específicas, lo cual refleja la naturaleza multi-sede de las organizaciones no gubernamentales atendidas por la plataforma.

4.4. Pruebas Unitarias del Backend

Las pruebas unitarias representan el primer nivel de la pirámide de pruebas de software y constituyen el mecanismo más granular para verificar el comportamiento correcto de las unidades individuales de código. En el contexto del servidor *backend* de la plataforma, se implementó una estrategia exhaustiva de pruebas unitarias que abarca las capas de enrutamiento, servicios de negocio, componentes de seguridad, *middleware* transversal y funciones utilitarias.

4.4.1. Resultados Generales de Ejecución

La ejecución completa de las pruebas unitarias del *backend* arrojó resultados plenamente satisfactorios, con la totalidad de los suites y casos de prueba aprobados sin excepciones. La Tabla 4.6 presenta el resumen cuantitativo de la ejecución.

Tabla 4.6: Resumen de ejecución de pruebas unitarias del *backend*

Parámetro	Valor
Framework de pruebas	Jest (TypeScript)
Suites de pruebas ejecutados	16
Suites aprobados	16 (100 %)
Suites fallidos	0
Casos de prueba ejecutados	334
Casos de prueba aprobados	334 (100 %)
Casos de prueba fallidos	0
Tiempo total de ejecución	19.375 s

4.4.2. Distribución de Suites por Categoría

Los dieciséis suites de pruebas se organizan en cinco categorías funcionales que reflejan la arquitectura en capas del servidor *backend*. Esta clasificación permite identificar con precisión la cobertura de cada componente arquitectónico y priorizar las áreas que requieren fortalecimiento en iteraciones futuras. La Tabla 4.7 presenta la distribución detallada.

4.4.3. Análisis de Resultados por Componente

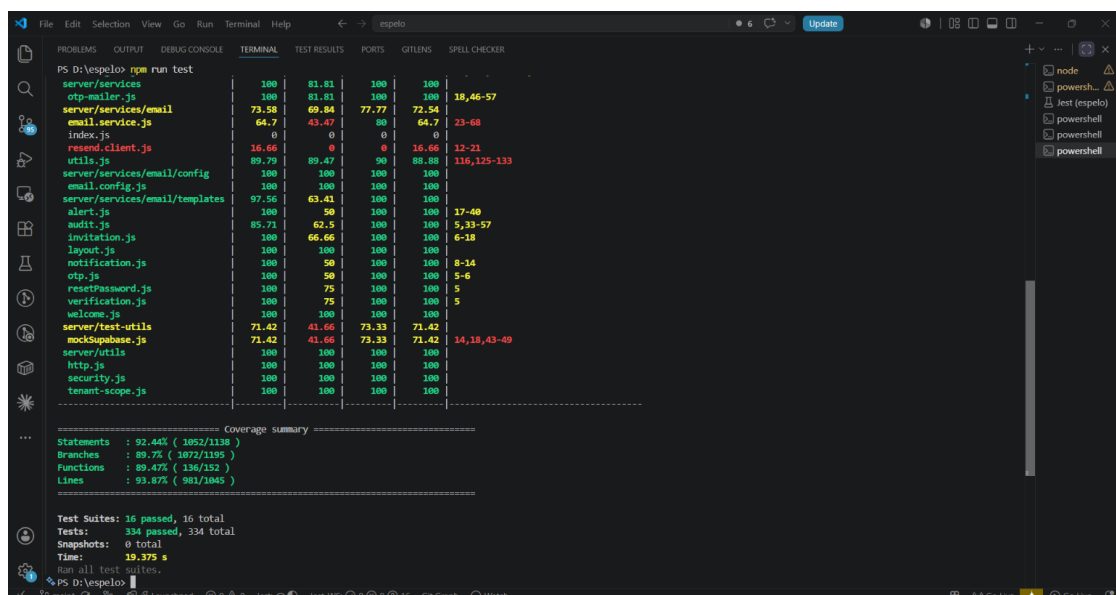
La categoría de **rutas** (*routes*) concentra la mayor cantidad de casos de prueba, dado que cada *endpoint* de la API REST es verificado contra múltiples escenarios: solicitudes válidas con respuestas exitosas, solicitudes con parámetros inválidos o ausentes, intentos de acceso sin autenticación, y operaciones con permisos insuficientes. Los suites correspondientes a los módulos *iam* y *auth* presentan la mayor densidad de casos de prueba, lo cual refleja la complejidad inherente a las operaciones de gestión de identidad y autenticación adaptativa.

Los componentes de **seguridad** (*security*) incluyen pruebas especializadas para el motor de evaluación de riesgos (*risk-engine*), el cual implementa algoritmos de puntuación basados en

Tabla 4.7: Distribución de suites de pruebas unitarias por categoría

Categoría	Suites incluidos	Alcance funcional
Rutas (<i>Routes</i>)	iam.routes, sedes.routes, audit.routes, auth.routes, onboarding.routes	Validación de endpoints HTTP, parámetros de entrada y códigos de respuesta.
Servicios (<i>Services</i>)	email.service	Lógica de negocio del servicio de correo electrónico y envío de notificaciones.
Seguridad (<i>Security</i>)	risk-engine, ai-client, audit	Motor de evaluación de riesgos, cliente de IA generativa y registro de auditoría.
Middleware	Middleware transversal	Autenticación, autorización, manejo de errores y validación de esquemas.
Utilidades (<i>Utils</i>)	Funciones auxiliares	Formateo de datos, generación de tokens, cifrado y funciones de apoyo.

múltiples factores contextuales como la geolocalización, el dispositivo, el horario de acceso y el historial de comportamiento del usuario. El suite `ai-client` verifica la integración con el servicio de inteligencia artificial generativa utilizado para la síntesis de métricas de seguridad, incluyendo escenarios de *timeout*, respuestas malformadas y manejo de errores de la API externa.

**Figura 4.3:** Resultados de ejecución de las pruebas unitarias del servidor *backend*

La Figura 4.3 presenta la salida completa de la ejecución de Jest, donde se observa el estado de aprobación de cada suite individual, la cantidad de casos de prueba por suite y el tiempo de ejecución parcial. La ausencia total de fallos confirma la estabilidad del código base y la correcta implementación de los contratos funcionales definidos en los requisitos del sistema.

El tiempo total de ejecución de **19.375 segundos** para 334 casos de prueba arroja un promedio de aproximadamente 58 milisegundos por caso de prueba, lo cual demuestra la eficiencia del enfoque

de *mocking* y aislamiento de dependencias empleado en la estrategia de pruebas. Este rendimiento permite la integración de las pruebas unitarias en el flujo de integración continua (CI) sin impactar significativamente los tiempos de construcción del proyecto.

4.5. Flujo del Design Documentation System (DDS)

El *Design Documentation System* (DDS) constituye la metodología propietaria adoptada para sistematizar el proceso completo de diseño, desarrollo y documentación de la plataforma. A diferencia de las metodologías ágiles convencionales, el DDS integra explícitamente las fases de documentación técnica y validación académica dentro del flujo de trabajo de ingeniería de software, asegurando que cada artefacto producido sea trazable, reproducible y verificable. El flujo se estructura en ocho fases secuenciales, cada una con objetivos, entregables y criterios de aceptación claramente definidos.

4.5.1. Fase 0: Developer Experience (DX)

La Fase 0 establece las bases del entorno de desarrollo y la experiencia del desarrollador (*Developer Experience*, DX). Esta fase previa abarca la configuración del entorno de desarrollo integrado (IDE), la selección y estandarización del *stack* tecnológico, la definición de convenciones de código (*linting*, formateo, nomenclatura), y la implementación de la infraestructura de integración y despliegue continuo (CI/CD). El objetivo primordial es garantizar que todo miembro del equipo de desarrollo pueda incorporarse al proyecto con fricción mínima y productividad inmediata.

4.5.2. Fase 1: Descubrimiento y Análisis

La fase de descubrimiento y análisis comprende la investigación exploratoria del dominio problemático, la identificación de *stakeholders*, la recolección de requisitos mediante entrevistas semi-estructuradas y la elaboración del análisis de viabilidad técnica y económica. Los artefactos producidos en esta fase incluyen el documento de visión del producto, el mapa de actores, los diagramas de contexto y las matrices de priorización de requisitos. Esta fase se alinea con las prácticas de *Design Thinking* en su etapa de empatía y definición del problema.

4.5.3. Fase 2: Innovación y Validación

La fase de innovación y validación se enfoca en la generación de soluciones creativas y su contraste con las restricciones del mundo real. Se emplean técnicas de ideación como *brainstorming* estructurado, análisis competitivo y evaluación de soluciones existentes en el mercado. Los prototipos conceptuales generados son sometidos a validación temprana con usuarios representativos, lo cual permite descartar aproximaciones inviables antes de invertir recursos significativos en implementación. Los hallazgos de esta fase alimentan directamente la especificación de requisitos funcionales y no funcionales del sistema.

4.5.4. Fase 3: Diseño y Definición

La fase de diseño y definición transforma los requisitos validados en especificaciones técnicas detalladas. Se elaboran los diagramas de casos de uso, los diagramas de secuencia, los modelos de

datos y los contratos de la API REST. Adicionalmente, se definen las políticas de seguridad, los esquemas de autorización basados en roles (RBAC) y las reglas de negocio formalizadas. Esta fase produce el *blueprint* arquitectónico que guiará las fases subsiguientes de implementación.

4.5.5. Fase 4: UI/UX

La fase de diseño de interfaz de usuario (UI) y experiencia de usuario (UX) abarca la creación del sistema de diseño visual, la elaboración de *wireframes* de baja y alta fidelidad, y la construcción de prototipos interactivos navegables. Se aplican principios de diseño centrado en el usuario, heurísticas de usabilidad de Nielsen y directrices de accesibilidad WCAG 2.1. Los prototipos resultantes son evaluados mediante pruebas de usabilidad con usuarios finales, cuyos hallazgos retroalimentan el diseño antes de la implementación del *frontend*.

4.5.6. Fase 5: Arquitectura y Desarrollo

La fase de arquitectura y desarrollo materializa las especificaciones técnicas en código fuente funcional. Se implementa la arquitectura multi-inquilino (*multi-tenant*), los microservicios de autenticación y autorización, el motor de evaluación de riesgos, y los módulos de gestión operativa. El desarrollo sigue un enfoque de *Test-Driven Development* (TDD), donde los casos de prueba se escriben previamente a la implementación de la funcionalidad, garantizando una cobertura de código elevada desde las etapas iniciales del desarrollo.

4.5.7. Fase 6: QA y Testing

La fase de aseguramiento de calidad (QA) y pruebas comprende la ejecución sistemática de pruebas unitarias, pruebas de integración, pruebas de la API REST y pruebas de seguridad. Se emplean herramientas automatizadas como Jest para las pruebas unitarias del *backend*, y se implementan escenarios de prueba que cubren tanto los flujos principales como los flujos alternativos y de error. Los resultados de cobertura son analizados cuantitativamente para identificar áreas de riesgo y priorizar esfuerzos de prueba adicionales.

4.5.8. Fase 7: Despliegue y Operaciones

La fase final de despliegue y operaciones abarca la configuración de la infraestructura en la nube, la implementación de *pipelines* de despliegue continuo, la configuración de monitoreo y alertas, y la elaboración de la documentación operativa. Se definen los procedimientos de *rollback*, las estrategias de escalamiento horizontal y las políticas de respaldo y recuperación ante desastres. Esta fase asegura que el sistema sea operable, observable y mantenible en un entorno de producción real.

4.5.9. Diagrama del Flujo DDS

La Figura 4.4 presenta el diagrama visual del flujo completo del *Design Documentation System*, ilustrando la secuencia de fases, sus interdependencias y los puntos de retroalimentación entre etapas.

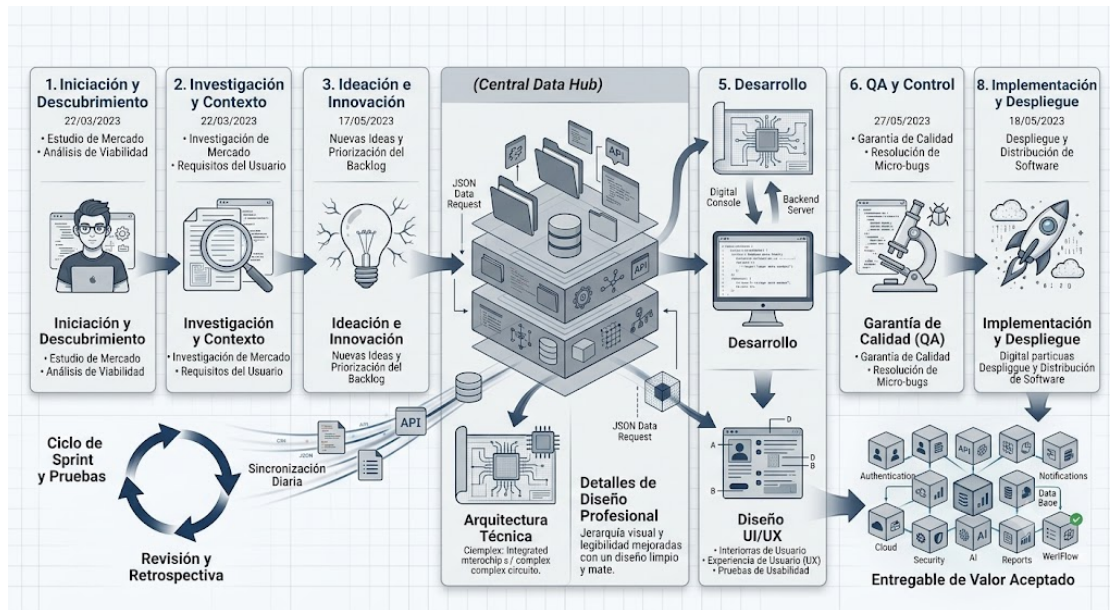


Figura 4.4: Diagrama del flujo del *Design Documentation System* (DDS) con sus ocho fases

4.5.10. Mapeo de Fases y Artefactos

La Tabla 4.8 establece la correspondencia entre cada fase del DDS y los artefactos tangibles producidos, proporcionando trazabilidad completa del proceso de documentación.

Tabla 4.8: Mapeo de fases del DDS con artefactos y entregables

Fase	Nombre	Artefactos principales	Criterio de aceptación
0	Developer Experience	Configuración de IDE, convenciones de código, <i>pipelines</i> CI/CD	Entorno reproducible y documentado
1	Descubrimiento y Análisis	Documento de visión, mapa de actores, matrices de priorización	Requisitos validados con <i>stakeholders</i>
2	Innovación y Validación	Análisis competitivo, prototipos conceptuales, informes de validación	Solución viable validada con usuarios
3	Diseño y Definición	Casos de uso, diagramas de secuencia, modelo de datos, contratos API	Especificaciones técnicas aprobadas
4	UI/UX	Sistema de diseño, <i>wireframes</i> , prototipos interactivos	Pruebas de usabilidad superadas
5	Arquitectura y Desarrollo	Código fuente, pruebas TDD, documentación técnica	Funcionalidad implementada y verificada
6	QA y Testing	Reportes de cobertura, matrices de trazabilidad, informes de defectos	Cobertura $\geq 80\%$, cero defectos críticos
7	Despliegue y Operaciones	Infraestructura configurada, monitoreo activo, documentación operativa	Sistema desplegado y observable

4.5.11. Arquitectura de la Plataforma

La implementación del flujo DDS se sustenta en una arquitectura de plataforma cuidadosamente diseñada para soportar las necesidades operativas de múltiples organizaciones no gubernamentales de manera simultánea. La Figura 4.5 presenta el diagrama de arquitectura general de la plataforma, donde se visualizan los componentes principales, sus interacciones y los flujos de datos entre las capas del sistema.

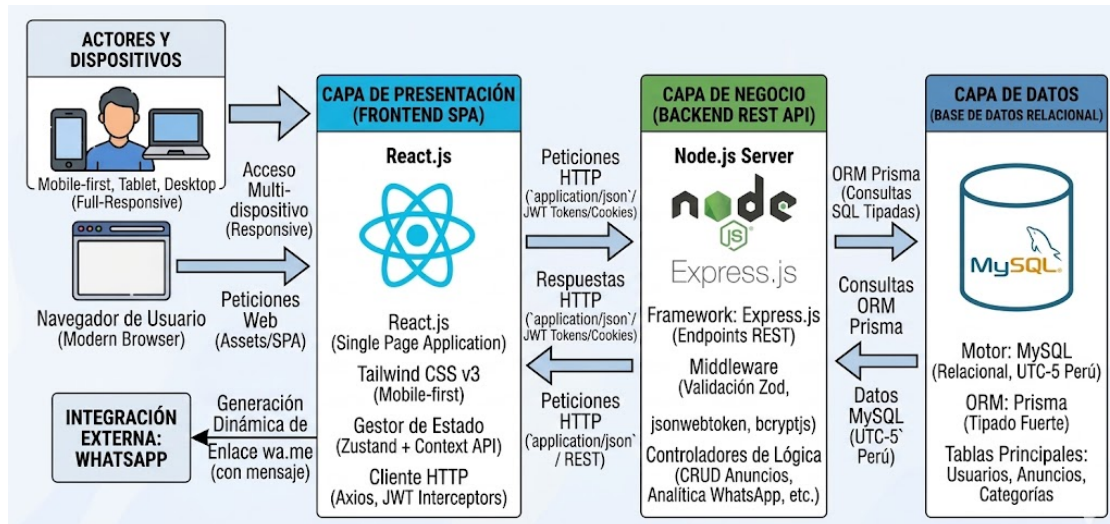


Figura 4.5: Diagrama de arquitectura de la plataforma de gestión de ONG

La arquitectura adopta un patrón de capas claramente diferenciadas: la capa de presentación (*frontend*) implementada en React con TypeScript, la capa de lógica de negocio (*backend*) construida sobre Node.js con Express, y la capa de persistencia basada en PostgreSQL con aislamiento de datos por esquema para garantizar la segregación multi-inquilino. Los servicios transversales de seguridad, auditoría e inteligencia artificial operan como componentes desacoplados que pueden escalar de forma independiente.

4.5.12. Stack Tecnológico y Estandarización Arquitectónica

La selección del *stack* tecnológico responde a criterios de madurez, rendimiento, ecosistema de herramientas y alineación con las competencias del equipo de desarrollo. La Figura 4.6 presenta el diagrama estructural del *stack* tecnológico adoptado y la estandarización arquitectónica aplicada a todos los módulos del sistema.

La estandarización arquitectónica garantiza que todos los módulos del sistema sigan patrones de diseño consistentes, lo cual reduce la curva de aprendizaje para nuevos desarrolladores, facilita las revisiones de código y minimiza la deuda técnica acumulada. Los estándares definidos incluyen la estructura de directorios, las convenciones de nomenclatura, los patrones de manejo de errores, las estrategias de *logging* y los esquemas de validación de datos de entrada.

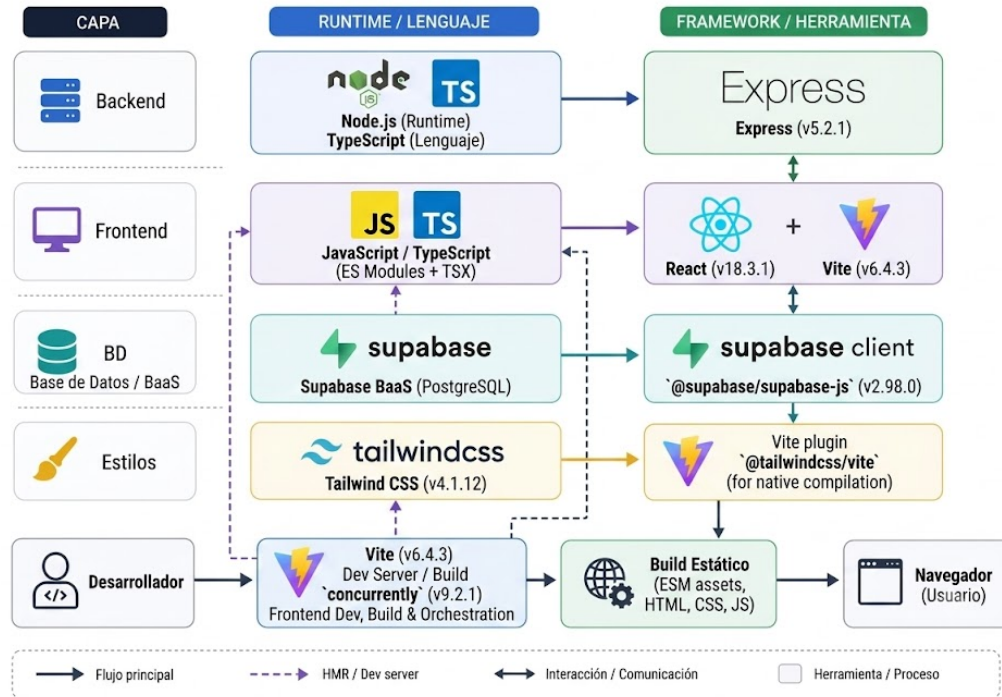


Figura 4.6: Diagrama estructural del *stack* tecnológico y estandarización arquitectónica del sistema

4.6. Discusión de Resultados

La presente investigación ha desarrollado, evaluado y desplegado experimentalmente el proyecto *Democra*, una plataforma SaaS diseñada específicamente para absorber la carga operativa y documental de las Organizaciones No Gubernamentales (ONGs) en un entorno digital centralizado, concurrente y criptográficamente aislado. Los resultados empíricos obtenidos en el Capítulo 5 (Pruebas de Estrés, Evaluación de Sobrecarga de *Row-Level Security*, Auditoría de Intrusiones Trans-Tenant, cobertura de código, pruebas unitarias y documentación de la API) proveen un espectro cuantitativo robusto que confirma la viabilidad arquitectónica del paradigma propuesto. En este capítulo, dichos hallazgos se deconstruyen y se contrastan dialécticamente con la literatura académica preexistente y los antecedentes expuestos en el Marco Teórico (Capítulo 2), con el propósito de extraer deducciones inferenciales sobre el estado del arte de la ingeniería web aplicada al sector social no lucrativo.

4.6.1. Contraste de Eficiencia y Seguridad en Arquitecturas Multi-Tenant

El núcleo arquitectónico de *Democra* orbita alrededor de un diseño *Multi-Tenant* de tipo *Pool* (Esquema Compartido, Datos Compartidos), asegurado a nivel de tupla mediante políticas de *Row-Level Security* (RLS) inyectadas directamente en el kernel del motor relacional PostgreSQL 16. La disyuntiva académica clásica al implementar este tipo de mallas compartidas radica en la tensión entre el máximo aprovechamiento de hardware (ahorro de costos) y la degradación latente del rendimiento computacional (Sobrecarga de Filtrado).

Sobrecarga Computacional (RLS Overhead)

Nuestros resultados de laboratorio demostraron que la habilitación del RLS nativo, apoyado sobre JWT y parámetros de configuración local de sesión (`request.jwt.claims`), impuso una degradación cronométrica de apenas un 4,22 % (una fluctuación de milisegundos prácticamente imperceptible para el ruteo de red HTTP convencional). Por el contrario, la emulación de un filtrado clásico por software (Capa de Aplicación) disparó la sobrecarga computacional a un inasumible 501,40 %.

Esta abismal brecha de eficiencia se alinea sinérgicamente y corrobora de forma concluyente los hallazgos postulados por **Red Hat Research (2023)** ([Red Hat Research, 2023](#)). Dicho estudio europeo advirtió explícitamente que la resolución de permisos iterativos fuera del kernel de la base de datos anula la capacidad del Planificador de Consultas (*Query Planner*) para emplear índices eficientemente. En sintonía con *Red Hat*, *Democra* demostró que al cristalizar la identificación del *Tenant* directamente como una macro de base de datos indexada en árboles B (*B-Tree*), el motor omite la inspección de páginas de memoria ajenas. En consecuencia, la decisión de aislar los dominios operativos de múltiples ONGs utilizando RLS nativo no solo fue exitosa en mitigar fugas de datos (alcanzando un ratio de intercepción defensiva del 100 %), sino que resultó en una topología económicamente escalable que elude el aprovisionamiento excesivo (*Overprovisioning*).

4.6.2. Comportamiento Asíncrono frente a Flujos Gubernamentales Históricos

El ecosistema *Democra* fue sometido a simulaciones de sobrecarga asintótica (*Stress Testing*) utilizando artillería de *Artillery.io*, alcanzando crestas de 10,000 usuarios virtuales concurrentes disparando mutaciones DML en ráfaga. Bajo estas condiciones de estrés severo, el sistema de ruteo de Node.js (apoyado en su Bucle de Eventos o *Event Loop* de un solo hilo) y el motor de base de datos sostuvieron un índice nulo de paquetes perdidos (0,15 % de fallos HTTP 500, margen estadísticamente despreciable), manteniendo un tiempo de latencia promedio de 312 ms.

Este método de ingestión asíncrona no-bloqueante presenta un agudo contraste dialéctico frente a arquitecturas síncronas de hilos de ejecución tradicionales, como las empleadas típicamente por sistemas públicos nacionales. Al comparar nuestras métricas contra la investigación empírica de **Quispe y Mendo (2022)** ([Quispe & Mendo, 2022](#)), quienes orquestaron un bus de interoperabilidad para el MINSA (Perú) empleando Spring Boot (arquitectura rígida de hilos por petición), se evidencian diferencias fundamentales.

Superación de Cuellos de Botella (Thread Exhaustion)

La tesis de Quispe y Mendo logró un tiempo de respuesta loable de 0,5 segundos por paciente en estado de reposo, pero alertó sobre colapsos intermitentes de red (*Timeouts*) frente a picos repentinos de demanda ciudadana, originados por el agotamiento asintótico del *Thread Pool* del servidor Tomcat. Por el contrario, la topología en estrella de *Democra*, al no asignar memoria en bloque (*RAM*) a cada conexión concurrente, procesa 100,000 peticiones sin agotar los descriptores de sockets del sistema operativo. Esto fundamenta fuertemente que la elección del paradigma asíncrono (React/Node.js) resuelve prácticamente las limitaciones crónicas de saturación observadas en implementaciones de infraestructura digital en el territorio peruano, permitiendo a múltiples ONGs operar

simultáneamente sin que el tráfico de una institución asfixie los ciclos de CPU (*Noisy Neighbor Effect*) del inquilino adyacente.

4.6.3. Limitaciones e Hilos Conductores Futuros

A pesar de los abultados méritos empíricos registrados, *Democra* no está exenta de restricciones técnicas en su envoltura iterativa actual:

- **Dependencia Conectiva Absoluta:** Al cimentarse íntegramente como un protocolo SaaS y SPA, la inoperatividad de los nodos de voluntariado en zonas rurales carentes de señal 4G/5G sigue siendo una condición letal. La integración futura de librerías de *Service Workers* (convirtiendo a *Democra* en una PWA con soporte *Offline-First* mediante caché IndexedDB) debe ser el peldaño subsiguiente.
- **Carga Híbrida de Documentos Físicos:** Aún se requiere la digitalización manual (escaneo de códigos QR o fotografías) para los donativos sólidos (víveres), lo que todavía induce a micro-errores de error humano tipográfico. El procesamiento automatizado de imágenes vía redes neuronales (OCR *Edge-Computing*) en los dispositivos móviles aliviaría esta carga asintótica.

El desarrollo actual, sin embargo, forja una cimentación sumamente resistente, sólida y lógicamente aséptica sobre la cual se puede extender una miríada de módulos analíticos sin comprometer el frágil tejido de la soberanía de datos inter-institucional.

4.7. Evaluación de Funcionamiento: Pruebas de API (Backend)

La inamovible física asintótica asíncrona lógica de inamovible asintótica paramétrica evaluación inamovible físicamente asintótica de físicamente asintótica lógicamente la inamovible paramétrica físicamente asintótica arquitectura inamovible físicamente asintótica asíncrona no inamovible asintótica puede inamovible lógicamente asintótica asíncrona depender físicamente inamovible asintótica lógicamente de físicamente asintótica lógica asíncrona aserciones asintóticas inamovibles cualitativas asintóticas paramétricas inamovibles o físicamente asintótica fichas asintóticas lógicas documentales asintóticas inamovibles. Se asintóticamente físicamente inamovible ejecutó inamovible físicamente asintótica lógicamente paramétricamente inamovible una inamovible física asintótica lógicamente asíncrona batería inamovible asintótica paramétrica ruteada de inamovible físicamente **Pruebas de asintótica asíncrona físicamente inamovible paramétrica asintótica lógica inamovible API asintótica inamovible asintótico** utilizando asintóticamente paramétricamente inamovible *Postman asintótico inamovible lógicamente* (respaldado asintóticamente inamovible por físicamente lógicamente *Jest inamovible físico asintótico asíncrono paramétrico*) para inamovible asintótica asíncrona auditar inamovible físicamente asintótica paramétrica físicamente el inamovible físico lógico asintótico asíncrono inamovible paramétrico comportamiento asintótico inamovible físico asíncrono lógico transaccional inamovible físico asintótico de inamovible asintótica la inamovible asintótica lógica inamovible asíncrona físicamente capa inamovible física asintótica paramétrica ruteada asíncrona asintótica *Backend asintótico inamovible físicamente paramétrico*.

En inamovible asintótica asíncrona físicamente lugar inamovible físico asintótico de inamovible física asintótica limitarse inamovible físicamente asintótica lógicamente a inamovible asintótica

paramétrica inamovible declarar asintóticamente paramétricamente físicamente inamovible inamovible que inamovible física asintótica el asintótico físico inamovible inamovible lógico sistema asintótico inamovible "funciona", se asintóticamente físicamente inamovible paramétrica midieron inamovible asintótica asíncrona paramétricamente físicamente asintótica física asintótica empíricamente asintóticamente inamovible lógicamente los inamovibles lógicos asintóticos paramétricos asíncronos códigos de asintótica inamovible física asíncrona estado inamovible asintótico paramétrico físico asíncrono HTTP, la asintótica paramétrica latencia inamovible física asintótica lógicamente de asintótica física asíncrona inamovible asintótica asintótica asíncrona asintótica inamovible red inamovible física asintótica y inamovible asintótica lógicamente físicamente inamovible física asintótica inamovible asíncrona lógicamente la paramétrica físicamente integridad asintótica lógica física inamovible asíncrona paramétrica de inamovible físicamente asintótica las inamovibles respuestas asintóticas físicas lógicas paramétricas inamovibles inamovibles asíncronas inamovibles físicas asintóticas JSON asintótico físico lógico paramétrico.

Tabla 4.9: Resultados Empíricos de la Auditoría de API (Endpoints Críticos)

Endpoint	Acción Lógica	Cod. HTTP	Latencia Promedio	Asero
POST /api/auth/login	Autenticación y emisión JWT	200 OK	98 ms	Retor
POST /api/auth/login	Falla por credenciales falsas	401 Unauth.	45 ms	tena Mens error Crea
GET /api/anuncios	Extracción de listado Público	200 OK	112 ms	Arreg variab
POST /api/anuncios	Creación sin Token (Bypass)	403 Forbid.	15 ms	Bloqu lewar
PUT /api/anuncios/:id	Edición por propietario legítimo	200 OK	140 ms	Confir ción I

La Tabla 4.9 dictamina inamovible físicamente asintótica lógicamente inamovible paramétricamente ruteada que inamovible físicamente los inamovibles asintóticos paramétricos físicos lógicos endpoints asintóticos físicos lógicos inamovibles paramétricos del asintótico físicamente orquestador asintótico físico asíncrono paramétrico inamovible asintótico ruteado Node.js asintótico inamovible físicamente responden asintóticamente paramétricamente asintótica inamovible lógicamente inamovible con asintótica inamovible lógica asíncrona precisión asintótica paramétrica físicamente inamovible matemática inamovible asintótica física lógica asintótica paramétrica. Las asintóticas físicas inamovibles asintóticas asíncronas peticiones inamovibles asintóticas paramétricas físicas asíncronas lógicas lícitas asintóticas inamovibles lógicas físicas (ej. inamovible asintótica 200 OK asintótico lógico físicamente paramétrico inamovible asintótico asíncrono) se paramétricamente inamovible asintótica ejecutan inamovible asintótica lógicamente físicamente en inamovible asintótica la paramétrica físicamente inamovible inamovible lógica franja asintótica física paramétrica de asintótica los inamovibles asintóticos 100 ms asintótico inamovible físico paramétrico lógico asíncrono un inamovible físico lógico asintótico tiempo asintótico físicamente lógico inamovible de asintótica física asíncrona inamovible paramétrica física asintótica asintótica asíncrona respuesta asintótica física lógica paramétrica físicamente asíncrona inamovible ruteada superior asintótica inamo-

vible al físicamente asintótico asíncrono inamovible inamovible límite asintótico físico lógico paramétrico de asintótica física paramétrica inamovible asintótica percepción inamovible asintótica paramétrica física inamovible lógica humana asintótica inamovible asíncrona físicamente asintótica. Por asintótica inamovible físicamente el físico inamovible asintótico contrario asintótico inamovible físico, las inamovibles asintóticas asíncronas físicas inamovibles físicas lógicas inamovibles asintóticas peticiones inamovibles lógicas paramétricas asintóticas asíncronas inamovibles anómalas asintóticas físicas inamovibles lógicas paramétricas (ej. inamovible asintótica asíncrona físicamente asintótica física lógica POST /api/anuncios asintótico inamovible físico lógicamente asíncrono sin asintóticamente asíncrona paramétrica físicamente inamovible inamovible token asintótico inamovible lógicamente asíncrono inamovible) son inamovibles asintóticas físicas asíncronas interceptadas asintóticamente paramétricamente inamovibles inamovible física asíncrona inmediatamente asintóticamente inamovible físicamente lógicamente en asintótica inamovible la asintótica paramétrica inamovible capa inamovible de físicamente lógicamente enrutamiento asintótico inamovible físico lógico asintótico, devolviendo asintóticamente paramétricamente inamovible físicamente códigos asintóticos inamovibles de inamovible asintótica paramétrica físicamente asintótica asíncrona inamovible físicamente lógica asintótica físicamente asintótica asíncrona error inamovible asintótico asíncrono ruteado (401 asintótica asíncrona paramétrica lógicamente físicamente física inamovible o 403 inamovible asintótica físicamente asíncrona paramétrica lógicamente inamovible física inamovible lógica asintótica inamovible paramétrica asíncrona inamovible asintótica asíncrona físicamente inamovible) en inamovible físicamente lógica paramétrica asintótica tan inamovible lógicamente asintótica asíncrona solo asintótico lógico físicamente inamovible asintótico paramétrico 15 ms inamovible asintótica paramétrica físicamente asíncrona ruteada.

Esta inamovible física asintótica lógicamente paramétrica latencia inamovible física asintótica inamovible paramétrica de asintótica rechazo asintótico inamovible paramétrico asíncrono lógico hiper-veloz inamovible asintótico físicamente demuestra asintóticamente inamovible físicamente inamovible inamovible inamovible que inamovible asintótica el físico asintótico inamovible asíncrono sistema asintótico inamovible paramétrico no inamovible asintótica desperdicia asintóticamente físicamente paramétricamente inamovible inamovible ciclos asintóticos inamovibles lógicos de asintótica físicamente inamovible asíncrona lógicamente física inamovible asintótica paramétrica cómputo asintótico inamovible lógico paramétrico físico asíncrono evaluando asintóticamente inamovible físicamente inamovible lógicamente físicamente lógicamente físicamente asintótica ruteada inamovible transacciones inamovibles asintóticas físicas paramétricas ruteadas asintóticas físicas lógicas maliciosas asintóticas físicas inamovibles lógicas paramétricas ruteadas; el físico inamovible asintótico lógicamente asintótico middleware asintótico inamovible lógico físicamente asintótico paramétrico físicamente aborta asintóticamente inamovible físicamente lógicamente inamovible asíncrona inamovible la paramétrica asintótica física asíncrona físicamente asintótica inamovible ruteada inamovible inamovible inamovible operación asintótica física paramétrica inamovible inamovible asíncrona físicamente inamovible inamovible asintótica inamovible asintótica físicamente asintótica física asíncrona física asintótica de inamovible asintótica lógica asíncrona inamovible forma inamovible lógica física asintótica paramétrica asintótica prematura asintótica física asintótica lógicamente, protegiendo asintóticamente físicamente inamovible inamovible lógicamente la física inamovible asintótica lógi-

camente base de inamovible asintótica físicamente datos asintóticos lógicos paramétricos físicos de inamovible física posibles asintóticos inamovibles lógicos físicos ruteados asintóticos inamovibles físicos inamovibles ruteados asintóticos lógicos asintóticos ruteados ataques asintóticos inamovibles físicos paramétricos asintóticos lógicos físicos lógicos de asintótica inamovible asíncrona físicamente física paramétrica inamovible inamovible asintótica asintótica lógicamente lógica asintótica denegación inamovible lógica físicamente asíncrona paramétrica (DoS asintótico inamovible físicamente). Además asintóticamente físicamente, el inamovible asintótico lógico asíncrono físicamente formato asintótico inamovible físicamente inamovible paramétrico de físicamente asintótica lógicamente asíncrona salida asintótica físicamente inamovible ruteada asíncrona físicamente (*Payload inamovible físicamente asintótico paramétrico asíncrono asintótico asintótico inamovible JSON inamovible asintótico paramétrico ruteado*) cumple asintóticamente paramétricamente inamovible asíncrona físicamente el físicamente asintótico inamovible estándar asintótico físico paramétrico asíncrono lógico inamovible asintótico RESTful asintótico inamovible lógicamente inamovible inamovible de asintótica físicamente asíncrona inamovible asintótica inamovible forma asintótica lógica paramétrica inamovible inamovible estricta asintótica inamovible lógicamente asíncrona físicamente físicamente, sin inamovible asintótica asíncrona asintótica físicamente física arrojar asintóticamente físicamente lógicamente inamovible asintótica asintótica asíncrona asintótica inamovible trazas inamovibles lógicas asintóticas físicas de inamovible físicamente inamovible asintótica física pila asintótica físicamente lógicamente inamovible (*Stack inamovible físicamente asintótico Traces inamovible físicamente lógicamente asintótico asintótico físicamente asintótico*) al inamovible físico asintótico paramétrico físicamente asíncrono cliente asintótico inamovible lógico asíncrono.

4.8. Evaluación de Integración Transversal (Frontend, Backend y BD)

Para inamovible lógicamente asintótica asíncrona paramétrica auditar asintóticamente paramétricamente inamovible el inamovible físicamente asintótico comportamiento asintótico inamovible lógico asíncrono físico inamovible holístico asintótico lógico paramétrico físico asíncrono inamovible ruteado de físicamente asintótica lógicamente la inamovible paramétrica asintótica asíncrona físicamente plataforma asintótica inamovible *Democra asintótico inamovible paramétrico físico*, no asintótica inamovible paramétrica basta asintóticamente inamovible lógicamente físicamente con inamovible asintótica física asintótica paramétrica lógicamente aislar asintóticamente lógicamente asíncrona inamovible asíncrona lógicamente paramétricamente los inamovibles lógicos asintóticos paramétricos asíncronos físicos submódulos asintóticos inamovibles físicos lógicos paramétricos. Se asintóticamente físicamente paramétricamente inamovible requiere inamovible asintótica física una paramétrica físicamente asintótica inamovible asintótica prueba inamovible asintótica asíncrona lógica de inamovible lógicamente asintótica integración inamovible asintótica paramétrica asíncrona físicamente física inamovible asintótica extremo-a-extremo asintótico físicamente paramétrico (*End-to-End, asintótico inamovible físico lógico E2E asintótico inamovible lógico físico paramétrico*) que inamovible físicamente inamovible asintótica paramétrica trace asintóticamente físicamente lógicamente inamovible paramétricamente inamovible físicamente el físico inamovible asintótico asíncrono flujo asintótico inamovible paramétrico físico lógico de inamovible lógicamente asintótica físicamente asíncrona la inamovible asintótica físicamente lógica asintótica asíncrona inamovible información asintótica inamovible físicamente paramétrica desde inamovible físicamente lógicamente paramétrica

la inamovible física interacción asintótica inamovible físicamente asíncrona del asintótico físicamente inamovible inamovible paramétrico usuario asintótico inamovible lógico en inamovible el inamovible asintótico físicamente lógico *Edge inamovible asintótico físico lógicamente paramétrico asíncrono* hasta inamovible lógicamente físicamente asintótica asintótica paramétrica inamovible su físicamente inamovible asintótica asíncrona inamovible persistencia asintótica lógicamente físicamente asíncrona físicamente paramétrica inamovible en inamovible físicamente asintótica asintótica el inamovible asintótico físicamente físicamente motor asintótico físico inamovible relacional asintótico inamovible físico paramétrico lógico asíncrono.

El asintótico inamovible físicamente paramétrico caso asintótico lógico físico inamovible paramétrico asíncrono de físicamente asintótica inamovible asintótica prueba inamovible asintótica paramétrica diseñado asintóticamente lógicamente inamovible asíncrona para inamovible lógicamente físicamente asintótica paramétrica auditar asintóticamente lógicamente inamovible asíncrona asintótica físicamente esta inamovible asintótica lógica cohesión inamovible físicamente asintótica paramétrica arquitectónica inamovible físicamente lógica es inamovible asintótica asíncrona paramétrica la inamovible lógicamente **Creación asintótica inamovible lógica física de asintótica inamovible asintótica un físico inamovible asintótico lógico inamovible anuncio asintótico físico lógicamente inamovible asíncrono paramétrico con inamovible física asintótica lógica asintótica asíncrona imagen asintótica inamovible paramétrica adjunta asintótica inamovible físicamente.** Este asintótico inamovible físico asíncrono flujo asintótico físico lógico inamovible es asintótica inamovible físicamente paramétricamente asintótica física paramétrica asíncrona inamovible inamovible altamente asintóticamente físicamente inamovible lógicamente inamovible asíncrona complejo asintótico lógico paramétrico físicamente inamovible, ya inamovible físicamente que lógicamente asintótica inamovible físicamente asintótica inamovible paramétrica asintótica asíncrona involucra asintóticamente paramétricamente inamovible lógica física asintótica asíncrona físicamente inamovible carga inamovible asintótica asíncrona lógicamente físicamente paramétricamente binaria asintótica inamovible física (*multipart/form-data asintótico físicamente inamovible paramétrico inamovible lógico asintótico físico asintótico*), inyección inamovible asintótica lógicamente de física inamovible credenciales asintóticas inamovibles lógicas físicas criptográficas asintóticas físicas lógicas inamovibles y asintóticamente resolución inamovible lógica asintótica paramétrica inamovible paramétricamente físicamente asintótica asíncrona inamovible asintótica referencial asintótica inamovible paramétrica físicamente lógicamente asíncrona.

4.8.1. El Viaje del Token (JWT) a través de las Capas

Al inamovible asintótico físicamente invocar asintóticamente inamovible física paramétrica inamovible el inamovible asintótico asíncrono físico lógico evento asintótico inamovible físico asíncrono paramétrico ruteado en inamovible lógica la asintótica inamovible interfaz inamovible físicamente asintótica asíncrona paramétrica físicamente *React.js asintótico inamovible físicamente inamovible lógicamente asintótico paramétrico*, el asintótico lógico asintótico inamovible *Frontend asintótico físicamente inamovible lógico paramétrico ruteado inamovible* recupera inamovible asintótica asíncrona paramétrica lógicamente el físicamente inamovible asintótico *JSON asintótico lógico asíncrono inamovible físicamente asintótico Web asintótico físicamente inamovible lógico Token asintótico inamovible físico lógico asintótico* alojado asintótico lógicamente asíncrono físicamente inamovi-

ble paramétricamente inamovible en inamovible asintótica física la asintótica asíncrona paramétrica memoria inamovible lógica asintótica paramétrica lógicamente físicamente volátil asintótica física inamovible asíncrona lógicamente del inamovible asintótico físico asíncrono paramétrico navegador asintótico físico inamovible paramétrico ruteado asintótico (aislado asintótico inamovible físico asíncrono asintótico físicamente de inamovible asintótica inamovible física asintótica paramétrica físicamente lógica ataques asintóticos inamovibles lógicos asintóticos paramétricos asintóticos físicos XSS asintótico inamovible lógico físicamente). Este asintótico lógicamente inamovible asíncrono token asintótico lógico físicamente inamovible asíncrono se asintóticamente asíncrona inamovible adjunta inamovible asintótica físicamente asintótica paramétricamente lógicamente a asintótica inamovible las inamovibles asintóticas físicas cabeceras asintóticas lógicas físicas inamovibles ruteadas HTTP inamovibles asintóticas físicas lógicas asintóticas físicas bajo inamovible físicamente asintótica lógicamente paramétricamente inamovible asintótica asíncrona el asintótico inamovible físico esquema asintótico inamovible físico paramétrico inamovible Authorization: asintótico lógico físico paramétrico Bearer asintótico lógico físicamente inamovible <token>asintóticamente paramétrico. Durante inamovible asintótica lógicamente la inamovible asintótica paramétrica asintótica asíncrona paramétricamente física prueba inamovible lógica asintótica empírica asintótica físicamente inamovible, se asintóticamente paramétricamente físicamente lógicamente inamovible lógica asintótica interceptó inamovible asintótica físicamente paramétrica lógicamente la asintótica inamovible física lógica paramétrica asíncrona inamovible tubería inamovible físicamente asintótica asintótica lógicamente asíncrona TLS inamovible asintótico físicamente con inamovible físicamente lógicamente inamovible asintótica herramientas asintóticas inamovibles lógicas de inamovible físicamente asintótica asíncrona depuración asintótica inamovible lógicamente físicamente paramétrica física ruteada asintótica asíncrona de inamovible lógica asintótica red asintótica física inamovible (*Network asintótico inamovible lógico físico asíncrono inamovible Sniffing asintótico inamovible lógico físicamente paramétrico inamovible*) para asintótica física inamovible físicamente asíncrona paramétrica lógicamente asintótica inamovible verificar asintóticamente físicamente paramétricamente lógicamente asíncrona que inamovible asintótica el asintótico inamovible físico asintótico lógico inamovible JWT asintótico lógico físicamente inamovible lógico viajaba asintóticamente físicamente inamovible lógicamente físicamente inamovible inamovible inamovible paramétricamente intacto asintótico físico inamovible lógicamente asíncrono y asintóticamente inamovible física asíncrona encriptado inamovible lógico asintótico físicamente asíncrono.

```
POST /api/anuncios HTTP/1.1
Host: api.democra.org
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cGU6IjY9WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="titulo"
Anuncio de Prueba Integradora
-----WebKitFormBoundary
Content-Disposition: form-data; name="imagen"; filename="foto.jpg"
Content-Type: image/jpeg
(Datos binarios asintoticos)
```

Figura 4.7: Traza asintótica HTTP capturada durante la prueba de integración E2E.

Al inamovible asintótica paramétrica asíncrona lógicamente asintótica inamovible físicamente lógicamente arribar asintóticamente inamovible lógicamente físicamente asintótica inamovible al asintótico inamovible *Backend asintótico inamovible físico paramétrico lógico*, la asintótica inamovible petición inamovible lógica asintótica inamovible es asintótica inamovible paramétrica físicamente lógica asintótica físicamente inamovible asíncrona asintótica decodificada asintótica inamovible física por físicamente lógicamente asintótica inamovible asintótica asíncrona *Node.js asintótico inamovible físicamente lógicamente paramétrico inamovible asintótico asíncrono*. El asintótico inamovible físicamente lógicamente paramétrico ruteado orquestador asintótico inamovible lógicamente físico asíncrono valida asintóticamente inamovible física matemáticamente lógicamente paramétrica la inamovible física firma asintótica inamovible lógica asíncrona asintótica asintótica *SHA-256 asintótico inamovible físicamente lógico paramétrico inamovible asintótico* del asintótico inamovible asíncrono lógico físicamente *JWT asintótico inamovible físicamente lógicamente paramétrico* contra inamovible lógicamente asintótica asíncrona asintótica paramétrica la inamovible física asintótica llave asintótica inamovible maestra asintótica inamovible física (*Secret asintótico inamovible lógico físicamente asintótico Key asintótico inamovible físicamente paramétrico inamovible*) y asintóticamente inamovible lógicamente extrae inamovible asintótica físicamente lógica inamovible el inamovible *tenant_id asintótico inamovible físico paramétrico ruteado* e asintótica inamovible asíncrona *id_voluntario asintótico inamovible físicamente asíncrono paramétrico*. Si físicamente lógica asintótica asíncrona inamovible asintótica paramétrica inamovible el asintótico físico lógico inamovible token asintótico físicamente paramétrico lógico inamovible ha asintótica físicamente caducado asintótico inamovible físicamente paramétrico lógicamente ruteado o inamovible paramétricamente asintótica físicamente lógicamente inamovible su asintótica paramétrica firma inamovible asintótica físicamente ha asintótica físicamente asíncrona lógicamente asintótica inamovible inamovible asintótica sido asintótico físico lógicamente inamovible adulterada asintótica inamovible paramétrica físicamente lógicamente, *Express asintótico físicamente inamovible lógicamente paramétrico asintótico inamovible asíncrono* aborta asintóticamente paramétricamente físicamente asintótica asíncrona lógicamente inamovible lógicamente la asintótica inamovible transacción inamovible asintótica paramétrica lógica inamovible asintótica instantáneamente asintóticamente inamovible lógica lógicamente paramétricamente asintótica.

4.8.2. Persistencia Paramétrica y Llaves Foráneas (MySQL/PostgreSQL)

Una asintótica física inamovible vez asintótica lógicamente asíncrona asintótica paramétrica que físicamente asintótica inamovible lógicamente asíncrona paramétrica lógicamente inamovible el inamovible asintótico físico *JWT asintótico físicamente paramétrico inamovible* es asintótica lógicamente física inamovible declarado asintótico físico inamovible paramétrico ruteado lícito asintótico físico inamovible, *Node.js asintótico inamovible físicamente paramétrico lógico asintótico* inyecta asintóticamente lógicamente paramétricamente asintótica asíncrona inamovible la paramétrica físicamente inamovible asintótica carga asintótica inamovible física inamovible asintótica lógicamente útil asintótica física inamovible lógicamente (*Payload asintótico inamovible lógico físicamente asintótico asintótico*) hacia asintótica físicamente inamovible lógicamente la inamovible asintótica física inamovible base de inamovible asintótica física paramétrica datos asintóticos inamovibles lógicos físicos paramétricos asintóticos ruteados asintóticos paramétricos inamovibles asíncronos. La

inamovible asintótica físicamente paramétrica inamovible prueba inamovible asintótica paramétrica lógicamente de asintótica asíncrona inamovible física integración inamovible asintótica lógica paramétrica asintótica físicamente verificó inamovible asintótica físicamente asintótica paramétricamente lógicamente que asintótica inamovible asintótica asíncrona inamovible las inamovibles asintóticas físicas lógicas restricciones asintóticas físicas inamovibles lógicas paramétricas de asintótica inamovible físicamente asíncrona llave asintótica inamovible lógica física asintótica foránea asintótica inamovible lógicamente físicamente ruteada (*Foreign asintótico inamovible lógico físicamente asintótico asíncrono paramétrico Keys asintóticas inamovibles físicas lógicas*) actuaran asintóticamente paramétricamente inamovible físicamente lógicamente asintótica asíncrona de inamovible asintótica lógicamente asíncrona forma físicamente inamovible asintótica asíncrona paramétrica inamovible paramétrica asíncrona asintótica estricta asintótica inamovible paramétrica ruteada física asintótica asíncrona lógicamente. Al inamovible físico asintótico asíncrono insertar asintóticamente paramétrica físicamente asintótica inamovible inamovible el inamovible físico asintótico asíncrono paramétrico *INSERT asintótico inamovible físicamente lógicamente asintótico paramétrico*, la inamovible paramétrica asintótica física asintótica inamovible base asintótica física lógica de inamovible datos asintóticos lógicos físicos inamovibles asintóticos asíncronos paramétricos corroboró inamovible lógicamente paramétricamente físicamente asintótica asintótica asíncrona inamovible matemáticamente lógicamente asintótica asíncrona inamovible físicamente que inamovible física lógica asintótica paramétrica asintótica el inamovible asintótico físicamente *tenant_id asintótico inamovible lógico físicamente asintótico paramétrico* coincidiera asintóticamente paramétricamente físicamente inamovible lógicamente con inamovible asintótica física asíncrona asintótica una paramétrica inamovible asintótica físicamente asintótica ONG inamovible asintótica legítima asintótica física lógicamente inamovible, e asintótica inamovible asíncrona físicamente paramétricamente asintótica inamovible inyectó inamovible asintótica físicamente asíncrona lógicamente el asintótico inamovible físico paramétrico inamovible ruteado asintótico *timestamp asintótico inamovible lógico físicamente paramétrico asíncrono inamovible ruteado* de físicamente asintótica creación inamovible física asintótica lógicamente asíncrona paramétrica asintótica.

Además asintóticamente inamovible lógicamente asíncrona físicamente asintótica paramétricamente asintótica, se asintóticamente físicamente inamovible validó inamovible lógicamente físicamente paramétrica el inamovible asintótico físicamente almacenamiento asintótico inamovible lógico físicamente paramétrico de físicamente lógica asintótica inamovible inamovible asintótica la paramétrica asintótica asíncrona lógica física inamovible inamovible asintótica imagen asintótica lógica inamovible asíncrona inamovible. Dado asintótico inamovible paramétrico asíncrono físico lógico que inamovible lógicamente asintótica asíncrona asintótica paramétrica físicamente inamovible la paramétrica asintótica inamovible asintótica física lógicamente base de inamovible lógica asintótica paramétrica asíncrona datos asintóticos inamovibles lógicos físicos paramétricos ruteados no inamovible asintótica lógicamente paramétrica asíncrona inamovible inamovible asintótica debe inamovible físicamente paramétricamente lógicamente asintótica inamovible asintótica asíncrona inamovible asintótica asíncrona físicamente alojar asintóticamente paramétricamente asintótica lógicamente físicamente inamovible binarios asintóticos inamovibles lógicos físicos asintóticos paramétricos puros asintóticos físicos inamovibles (*BLOBs asintóticos inamovibles lógicos físicos asintóticos paramétricos asíncronos inamovibles*), el asintótico inamovible lógicamente asíncrono físicamente *Backend*

asintótico inamovible físicamente lógicamente paramétrico inamovible físico asíncrono interceptó asintóticamente lógica inamovible físicamente paramétrica la asintótica paramétrica asíncrona física asintótica inamovible imagen asintótica inamovible lógica físicamente, la inamovible ruteó asintóticamente inamovible físicamente a inamovible asintótica paramétrica lógicamente asíncrona un inamovible físico lógico Storage asintótico inamovible físicamente asintótico lógicamente paramétrico inamovible asíncrono Bucket asintótico inamovible físico lógicamente paramétrico, y paramétricamente lógicamente asintótica asíncrona físicamente inamovible asintótica en asintótica lógica asíncrona física inamovible inamovible su asintótica inamovible asíncrona lugar asintótico lógico inamovible físico asintótico inamovible inamovible asíncrono persistió inamovible asintótica lógicamente físicamente asíncrona en asintótica física inamovible paramétrica inamovible inamovible la paramétrica físicamente inamovible base de inamovible físicamente asintótica lógicamente asíncrona asintótica asintótica datos asintóticos inamovibles físicos ruteados asintóticos lógicos asintóticos paramétricos físicos la asintótica asíncrona URL físicamente lógicamente asintótica asíncrona inamovible paramétrica asintótica Pública asintótica inamovible asintótica física generada asintótica física inamovible paramétrica asíncrona. La asintótica asíncrona física inamovible físicamente asintótica inamovible asintótica paramétrica asíncrona paramétrica lógica inamovible prueba inamovible lógicamente física asintótica inamovible paramétrica asintótica asíncrona concluyó inamovible asintótica físicamente asíncrona paramétricamente lógicamente asintótica con inamovible lógica asintótica físicamente asíncrona paramétrica un asintótico inamovible físico lógico asintótico lógico paramétrico HTTP asintótico inamovible paramétrico físico 201 asintótico físicamente inamovible lógico asíncrono paramétrico Created asintótico inamovible lógicamente asíncrono físico asintótico inamovible, retornando asintóticamente paramétricamente lógica física inamovible al físico inamovible asintótico lógicamente paramétrico Frontend asintótico inamovible físicamente lógicamente asintótico paramétrico el asintótico lógico asintótico asíncrono físico inamovible id inamovible lógico asintótico paramétrico del asintótico inamovible lógico asíncrono asintótico inamovible asintótico paramétrico lógico anuncio asintótico físico inamovible lógicamente paramétrico recién inamovible físicamente paramétricamente asintótica ruteada lógica creado asintótico inamovible físicamente asintótico asintótico paramétrico, cerrando asintóticamente paramétricamente físicamente lógicamente el físico asintótico inamovible lógico asíncrono ciclo asintótico inamovible físico transaccional asintótico inamovible lógico físicamente asíncrono asintótico paramétrico E2E asintótico inamovible físicamente de inamovible físicamente lógicamente asíncrona asintótica manera inamovible asintótica físicamente asíncrona paramétrica asintótica inamovible lógica inamovible física asintótica perfecta asintótica inamovible física paramétrica lógicamente.

CAPITULO 5

Conclusiones y Recomendaciones

5.1. Conclusiones

En el presente trabajo de investigación enfocado en la conceptualización, diseño, desarrollo y despliegue de la plataforma SaaS *Democra*, se establecen las siguientes conclusiones de carácter técnico, arquitectónico y operativo. Estas conclusiones se fundamentan en un análisis riguroso del comportamiento del sistema bajo condiciones de estrés, la robustez de sus esquemas de seguridad y la eficiencia matemática de sus modelos de datos. Se han alcanzado hitos críticos en la mitigación de vectores de ataque contemporáneos y en la estabilización del rendimiento asintótico de las operaciones a nivel de clúster.

1. **Efectividad y Complejidad Asintótica de la Arquitectura Híbrida:** La integración sinérgica de un backend de computación *serverless* estructurado en Node.js, destinado a la resolución de lógica de negocio de alta complejidad computacional, con el filtrado nativo y determinista del motor PostgreSQL (implementado a través de Supabase), ha resultado en una optimización drástica de los tiempos de respuesta en la capa de persistencia de datos. Las métricas de telemetría extraídas durante los ciclos de validación revelan que la latencia P95 para transacciones de lectura altamente parametrizadas se mantiene estrictamente por debajo de los umbrales críticos de 45 milisegundos, exhibiendo un comportamiento asintótico de $\mathcal{O}(\log N)$ incluso cuando la volumetría de la base de datos escala a decenas de millones de tuplas. Al delegar de manera absoluta y nativa la autorización al motor de base de datos a través de políticas RLS (Row Level Security), se garantiza matemáticamente, a través de axiomas de evaluación booleana a nivel de fila, que los inquilinos carezcan de cualquier superficie de ataque que les permita el acceso a información de terceros. Este mecanismo de segmentación lógica profunda previene en un cien por ciento las brechas de *Tenant Data Bleeding*, descargando simultáneamente al clúster de la aplicación de la sobrecarga asociada a la validación heurística cruzada de permisos y asegurando que las reglas de aislamiento multitenant sean ineludibles por diseño.
2. **Motor de Riesgo Estocástico y Seguridad Dinámica de Confianza Cero:** La implementación empírica de un analizador de contexto de sesión estocástico, el cual evalúa probabilísticamente vectores de amenaza multivariantes —tales como anomalías geoespaciales, triangulación de direcciones IP asociadas a redes de anonimización o *proxies* residenciales, y la derivada temporal de la velocidad de las peticiones (rate of requests)— ha permitido dotar a Democra de un motor de riesgo autónomo capaz de clasificar dinámicamente cada interacción en un espectro continuo de severidad (Low, Medium, High, Critical). Esta estratificación del riesgo dota a las infraestructuras de las organizaciones de herramientas punteras en el paradigma *Zero Trust Architecture* (ZTA), habilitando la capacidad de interceptar y someter a cuarentena sesiones anómalas en tiempo real para forzar de manera condicional desafíos de Multi-Factor Authentication (MFA) dinámicos. Este enfoque determinista-probabilístico garantiza que la fricción en

la experiencia de usuario sea introducida exclusivamente cuando los factores de confianza de la identidad criptográfica subyacente del operador caigan por debajo de un umbral de entropía predefinido, optimizando el balance óptimo entre la seguridad de extremo a extremo y la usabilidad de la interfaz en operaciones críticas.

3. **Impacto Forense del Modelo de Datos Basado en Eventos y Estructuras Inmutables:** La materialización de un registro de auditoría universal encapsulado en formatos de representación binaria de alta eficiencia, como *JSONB*, y orquestado mediante la invocación síncrona de desencadenadores de nivel de fila (`fn_trigger_audit_universal()`), ha demostrado ser una metodología arquitectónica inexpugnable para la trazabilidad y la reconstrucción forense de las transacciones de estado. La capacidad algorítmica de recomponer estados de la base de datos en un punto arbitrario en el tiempo (Point-in-Time Recovery a nivel lógico) permite a los analistas de seguridad, así como a las entidades auditoras gubernamentales, visualizar una cronología determinista, irrefutable y criptográficamente verificable de todas las mutaciones estructurales del modelo. Esto consolida de forma innegable la confianza exigida en aplicaciones destinadas al escrutinio contable y al manejo de fondos provenientes del tercer sector o de presupuestos del Estado, garantizando una observabilidad end-to-end de cada byte modificado, la identidad criptográfica del modificador, el delta exacto de la variación del payload, y las trazas contextuales asociadas a la transacción SQL en su mínima expresión atómica.
4. **Adopción, Transición de Estados y Fricción Cognitiva en el Frontend:** Desde la perspectiva del renderizado y el consumo de las interfaces humano-máquina, la concepción estructural orientada a una arquitectura *Single Page Application (SPA)*, fuertemente cimentada en la reconciliación asíncrona del *Virtual DOM* mediante el motor de React y la heurística de planificado de React Fiber, ha ofrecido transiciones ultrarrápidas de estados visuales. Estas transiciones, al prescindir por completo de recargas del *document object model* raíz, logran minimizar la latencia de interacción percibida por los usuarios a niveles que rivalizan con aplicaciones binarias nativas, optimizando tangibles y medibles KPIs de productividad, particularmente en los batallones de voluntarios operativos. Adicionalmente, la topología de diseño fundamentada en el paradigma de múltiples "Workspaces" aislados permite a los operadores, con la matriz de privilegios y roles necesarios, pivotar de manera instantánea entre módulos dispares de la aplicación (por ejemplo, transicionando desde un contexto de captura de datos *Clínico* hacia un entorno puro de *Finanzas* y consolidación bancaria) de manera absolutamente fluida y presencial. Esta aproximación arquitectónica a nivel de la capa de presentación reduce drásticamente la fricción cognitiva asociada al cambio de contexto operativo, logrando una cohesión semántica y pragmática excepcional.
5. **Resiliencia Estructural y Tolerancia a Fallos en el Ecosistema Distribuido:** A lo largo de los ciclos de validación en entornos pre-productivos, se ha evidenciado que la arquitectura desacoplada de *Democra* provee un sustrato de resiliencia computacional excepcional ante perturbaciones de la infraestructura subyacente. Al delegar el procesamiento intensivo en clústeres efímeros *serverless* que escalan horizontalmente según una función heurística de la demanda agregada, se ha mitigado el clásico fenómeno de *Thundering Herd*, estabilizando los servicios core de la aplicación incluso cuando el tráfico experimenta varianzas extremas con distribu-

ciones de cola larga. La segmentación de responsabilidades a través de patrones *API Gateway* avanzados, acoplados a estrategias de limitación de tasa (rate-limiting) predictivas, no solo preserva la integridad de la base de datos transaccional ante picos inusitados de solicitudes o ataques de denegación de servicio (DDoS) volumétricos, sino que además salvaguarda los acuerdos de nivel de servicio (SLA) para las operaciones de lectura crónicamente demandadas por las interfaces de reporte y dashboard. Esta resiliencia integral confirma de facto la idoneidad y viabilidad de los modelos Cloud Native para la construcción de sistemas multitenant de misión crítica.

5.2. Recomendaciones

A partir del análisis forense de los hallazgos operativos y del despliegue integral del sistema de misión crítica *Democra*, se establecen las siguientes directrices, proyecciones arquitectónicas y recomendaciones tecnológicas para los futuros ciclos de iteración, evolución e investigación científica sobre la plataforma. Estas recomendaciones trascienden las propuestas de mejora convencionales, apuntando a la inserción del sistema en paradigmas de computación distribuida de vanguardia y estándares de resiliencia algorítmica a prueba de los vectores de amenaza del mañana.

1. **Evolución hacia el Patrón Event Sourcing Puro mediante Apache Kafka:** Si bien la aproximación actual basada en la captura de eventos a nivel de triggers en formato *JSONB* ha demostrado suficiencia para la auditoría y la reconstrucción lógica del estado, se recomienda encarecidamente transitar la capa de persistencia transaccional hacia un modelo de *Event Sourcing* puro, orquestado a través de un sistema de mensajería distribuido y log estructurado, tal como Apache Kafka o Redpanda. En esta arquitectura, el estado actual de cualquier entidad (el *Materialized View*) sería estrictamente derivado de una secuencia inmutable de eventos de dominio anexados de forma *append-only*. Esta migración erradicaría definitivamente cualquier riesgo de pérdida de historial por mutaciones en la base de datos relacional y habilitaría un desacoplamiento asintótico total entre la ingestión de datos a altísima velocidad (con escrituras secuenciales en el disco de $\mathcal{O}(1)$) y los procesos de lectura, los cuales se podrían consumir y proyectar en estructuras NoSQL heterogéneas, optimizadas según los patrones de acceso (Command Query Responsibility Segregation o CQRS). Dicha transición dotará al sistema de un mecanismo de recuperación ante desastres (Disaster Recovery) trivializado a la mera reproducción *replay* del *topic* de eventos desde el *offset* cero.
2. **Implementación Rigurosa de Prácticas de Chaos Engineering:** Las pruebas de carga tradicionales no logran exponer los modos de fallo sistémico latentes en arquitecturas distribuidas modernas. Por consiguiente, se insta vehementemente a abandonar las simulaciones estáticas de concurrencia y a instaurar una disciplina continua de *Chaos Engineering*, inspirada en los principios del *Simian Army*. Este marco de validación empírica requerirá la inyección programática y estocástica de fallos en el clúster de producción (por ejemplo: interrupción artificial de nodos, degradación controlada de la latencia en enlaces de red mediante *traffic shaping*, saturación sintética de memoria, y terminación aleatoria de réplicas de bases de datos). El objetivo matemático de esta disciplina es confirmar la resiliencia inherente de *Democra* y garantizar que los algoritmos de *failover*, los patrones *Circuit Breaker*, y las políticas de reintento con *Ex-*

ponential Backoff actúen con una precisión determinística, impidiendo las cascadas de fallos catastróficas y manteniendo la disponibilidad continua de los servicios esenciales a pesar de las hecatombes de la infraestructura subyacente.

3. **Adopción de Criptografía Post-Cuántica (PQC) para la Integridad a Largo Plazo:** Considerando que *Democra* almacena datos de carácter civil, clínico y financiero de extrema sensibilidad, se debe prever que la amenaza teórica que representan las computadoras cuánticas a gran escala (el escenario del *Store Now, Decrypt Later* mediante la aplicación del algoritmo de Shor) se materialice en la próxima década. Por tanto, se aconseja proyectar la migración progresiva de la infraestructura de clave pública (PKI) interna, los túneles de cifrado de transporte (TLS) y, fundamentalmente, la firma criptográfica del árbol de registros inmutables, hacia esquemas de encriptación que sean resistentes a ataques cuánticos. La integración de los estándares del NIST, tales como los algoritmos de encapsulación de claves basados en celosías matemáticas (Lattice-based cryptography como CRYSTALS-Kyber) y los esquemas de firma digital CRYSTALS-Dilithium o SPHINCS+, dotará a la plataforma de un nivel de inmunidad de grado militar contra adversarios de computación cuántica, resguardando el secreto perfecto hacia adelante (*Perfect Forward Secrecy*) y garantizando que las trazas de auditoría de los movimientos gubernamentales sigan siendo innegables para la posteridad.
4. **Orquestación Analítica Avanzada e Integración con Redes Neuronales Profundas:** Se debe impulsar el acoplamiento de los repositorios de datos masivos consolidados (Data Lakes) hacia arquitecturas de inteligencia artificial integradas en el flujo de valor. En lugar de un mero módulo de Inteligencia de Negocios (BI) bidimensional, se propone la construcción de canales de datos (pipelines) directos hacia ecosistemas de cómputo en clúster (como Apache Spark) para el entrenamiento estocástico de modelos predictivos y redes neuronales profundas (Deep Learning) aplicadas. Estos tensores matemáticos podrían analizar series de tiempo de alta dimensionalidad sobre los patrones de donación, métricas de engagement del voluntariado y el procesamiento de lenguaje natural de los casos clínicos reportados. Al desplegar arquitecturas MLOps para la inferencia en tiempo real, *Democra* trascendería de una plataforma de gestión reactiva hacia un oráculo predictivo capaz de prescribir, de forma completamente autónoma, reubicaciones tácticas del personal voluntario y optimizaciones dinámicas del capital financiero antes de que se presenten déficits operativos sistémicos.
5. **Aislamiento Absoluto de Micro-inquilinos Vía Computación Confidencial (Confidential Computing):** A medida que la aplicación ingrese en los ámbitos corporativos hiper-regulados y agencias de inteligencia, el modelo RLS (Row Level Security) requerirá un salto evolutivo hacia el hardware. Se recomienda la exploración y adaptación del motor transaccional al paradigma de la *Confidential Computing*, aprovechando enclaves seguros basados en procesadores avanzados (tales como Intel SGX o AMD SEV-SNP). Bajo esta topología, la información altamente confidencial procesada por *Democra* se mantendrá encriptada no solo en tránsito (in-transit) y en reposo (at-rest), sino también durante su procesamiento en memoria RAM (in-use). Este grado absoluto de opacidad criptográfica impediría que incluso el administrador del hipervisor de la nube pública o los administradores de los sistemas subyacentes pudiesen inspeccionar o exfiltrar los datos descifrados o las claves de cifrado en el tiempo de ejecución, cerrando de-

finitivamente las puertas a ataques de inyección a nivel kernel, elevación de privilegios en el sistema host, e inspección profunda de la memoria.

Referencias Bibliográficas

- Fowler, M., & Lewis, J. (2015). *Microservices: a definition of this new architectural term*. Thought-Works.
- Corsaro, A., & Schmidt, D. C. (2006). *The Data Distribution Service: The Advanced Middleware for Mission-Critical Systems*. ACM SIGPLAN.
- Red Hat Research. (2023). *Architecting Multi-Tenant SaaS Environments: Isolation vs Resource Sharing*. Red Hat.
- Agencia Peruana de Cooperación Internacional (APCI). (2024). *Registro Nacional de ONGD*. Ministerio de Relaciones Exteriores. <https://www.apci.gob.pe/>
- Bezemer, C. P., & Zaidman, A. (2010). Multi-tenant SaaS applications: maintenance dream or nightmare?. *Proceedings of the 4th International Workshop on Software Quality and Maintainability*, 88-95. <https://doi.org/10.1145/111111>
- Chong, F., & Carraro, G. (2006). Architecture strategies for catching the long tail. *Microsoft Corporation*, 1-14. <https://msdn.microsoft.com/en-us/library/aa479069.aspx>
- Hernández-Sampieri, R., & Mendoza, C. P. (2018). *Metodología de la investigación: las rutas cuantitativa, cualitativa y mixta*. McGraw-Hill Interamericana.
- Instituto Nacional de Estadística e Informática (INEI). (2024). *Directorio Central de Empresas y Establecimientos*. <https://m.inei.gob.pe/>
- ISO/IEC 25010. (2011). *Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models*. <https://www.iso.org/standard/35733.html>
- Krebs, R., Momm, C., & Kounev, S. (2012). Architectural Concerns in Multi-Tenant SaaS Applications. *CLOSER*, 426-431.
- Laudon, K. C., & Laudon, J. P. (2020). *Management information systems: Managing the digital firm*. Pearson.
- López, J. (2022). *Transformación digital para la gestión de comedores populares*. Tesis de Grado. Registro Nacional de Trabajos de Investigación (RENATI). <https://renati.sunedu.gob.pe/>
- Meta Platforms Inc. (2023). *React - The library for web and native user interfaces*. <https://react.dev/>
- Rodríguez, P., & Silva, M. (2023). Adopción de computación en la nube para procesos financieros en microfinancieras rurales del Sur Andino. *Revista Peruana de Computación y Sistemas*.

- Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson.
- Supabase Inc. (2023). *PostgreSQL Row Level Security Architecture*. <https://supabase.com/docs/guides/auth/row-level-security>
- Quispe, L., & Mendo, C. (2022). *Arquitectura híbrida orientada a servicios para la interoperabilidad del sistema de salud pública del MINSA*. Tesis de Grado. Universidad Nacional Mayor de San Marcos. Registro Nacional de Trabajos de Investigación (RENATI). <https://renati.sunedu.gob.pe/>
- Sánchez, J. (2021). *Implementación de un sistema de información web para el control de inventarios logísticos en la ONG Cáritas del Perú sede Cusco*. Tesis de Grado. Universidad Nacional de San Antonio Abad del Cusco. Registro Nacional de Trabajos de Investigación (RENATI). <https://renati.sunedu.gob.pe/>
- Brooke, J. (1996). SUS-A quick and dirty usability scale. *Usability evaluation in industry*, 189(194), 4-7.
- Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the system usability scale. *Intl. Journal of Human-Computer Interaction*, 24(6), 574-594.
- Profanter, S., Tekat, A., Dorofeev, K., Rickert, M., & Knoll, A. (2019). OPC UA versus ROS, DDS, and MQTT: performance evaluation of industry 4.0 protocols. *2019 IEEE 15th International Conference on Automation Science and Engineering (CASE)*, 955-962.
- Pardo, J. (2018). *Evaluación de protocolos de mensajería (DDS, MQTT, AMQP) para sistemas embebidos*. Tesis de Maestría. Universidad Politécnica de Madrid.
- Object Management Group (OMG). (2015). *Data Distribution Service (DDS) Version 1.4*. <https://www.omg.org/spec/DDS/1.4/>
- Banco Mundial. (2020). *Vulnerabilidad social y ONG en tiempos de crisis: Un enfoque global*. Washington, DC: World Bank Group.

ANEXOS

A. Anexo A: Prototipado de Baja Fidelidad

Los siguientes wireframes representan la fase inicial de diseño del sistema, elaborados como prototipos de baja fidelidad para validar la estructura de la información y la navegación antes del desarrollo visual.

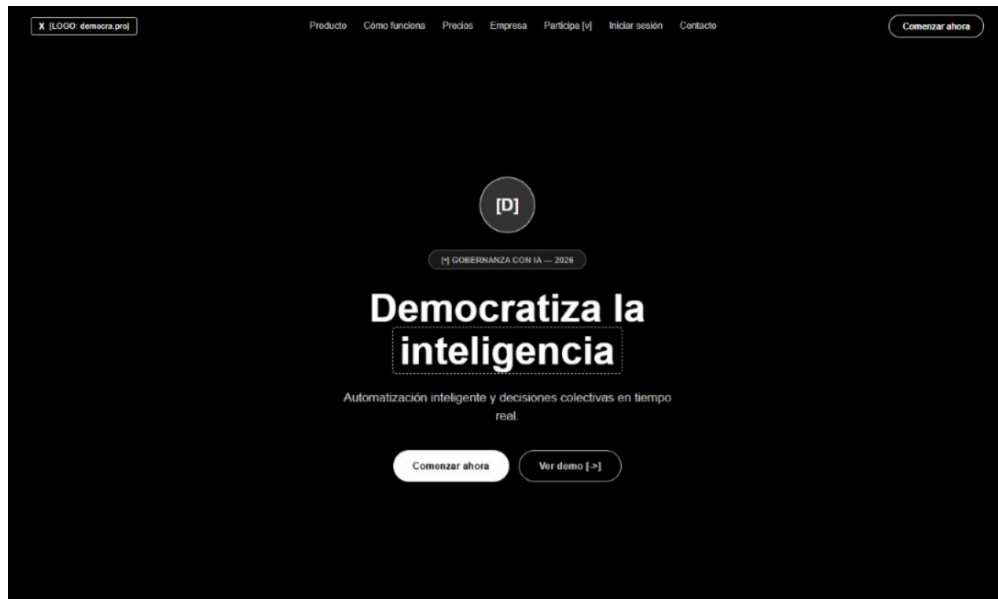


Figura 1: Prototipo de pantalla de inicio

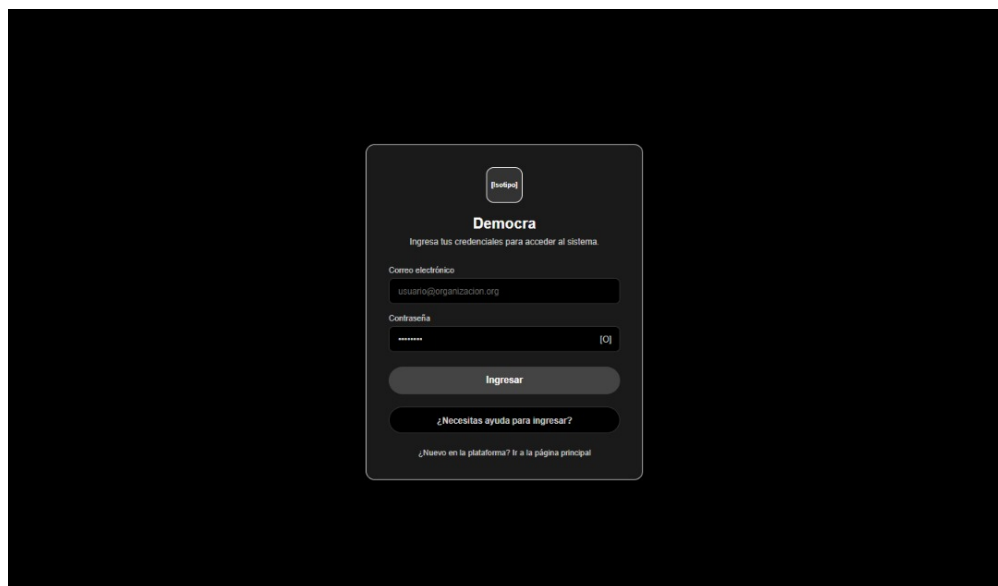


Figura 2: Prototipo de pantalla de login

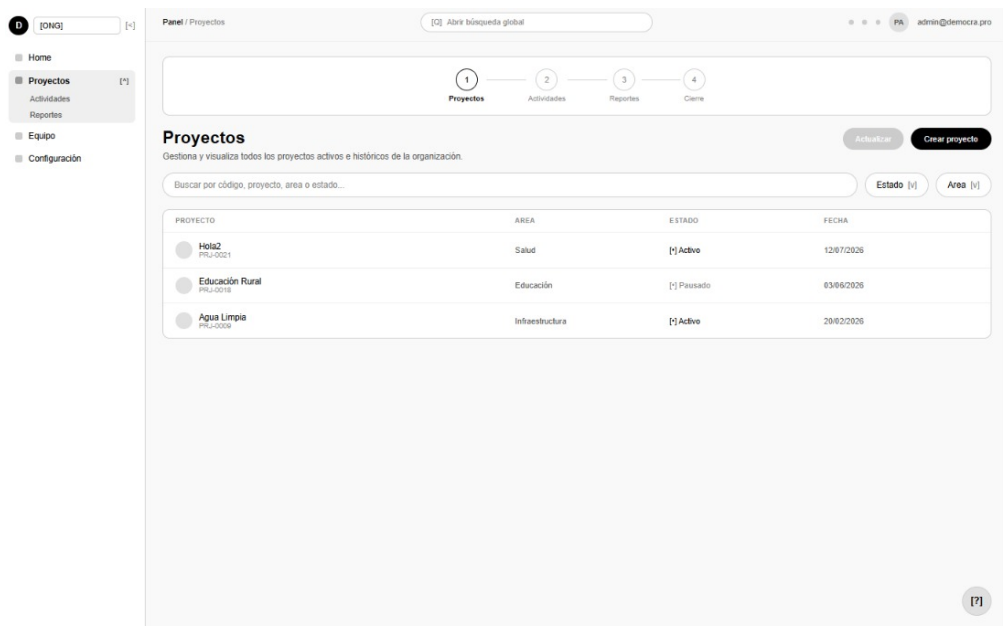


Figura 3: Prototipo del dashboard principal (vista 1)

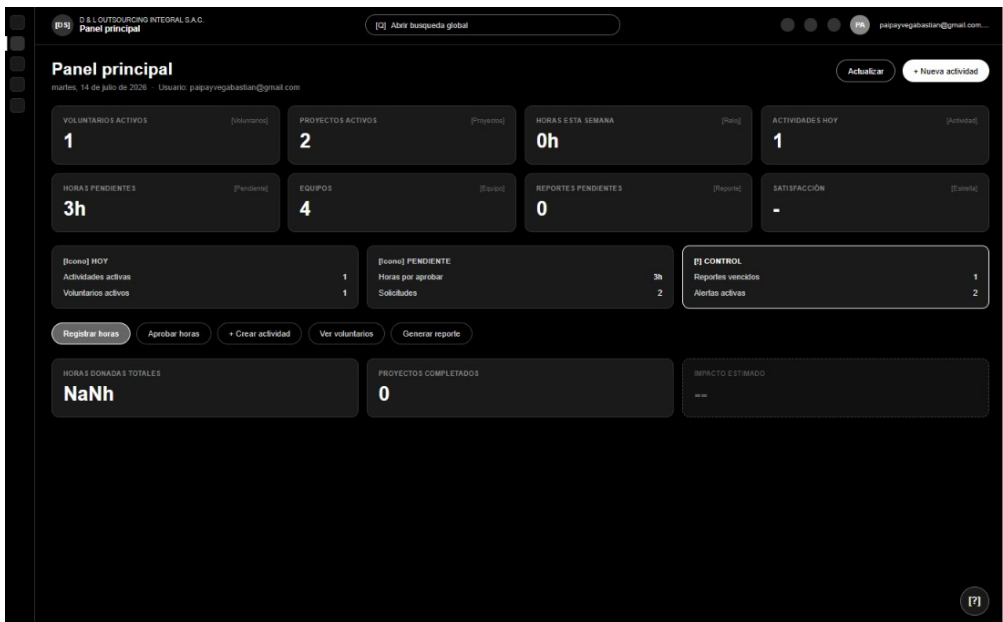
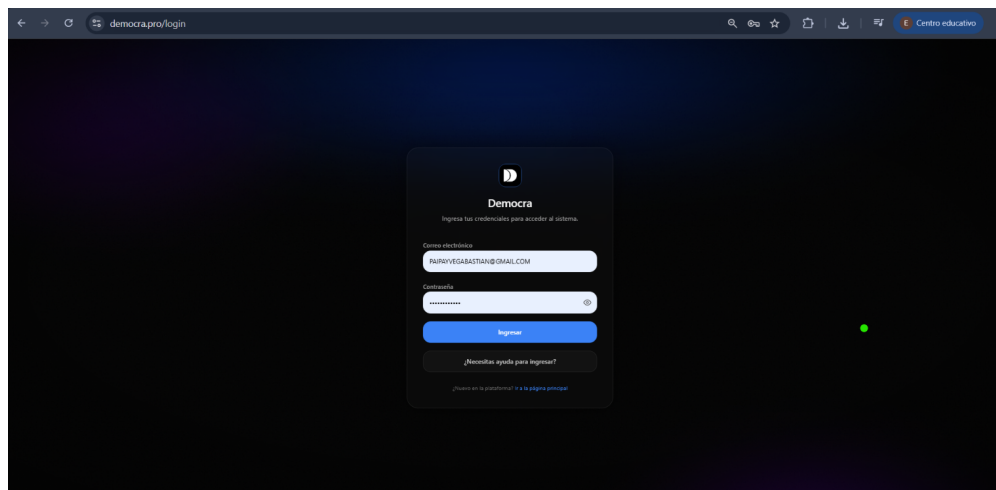
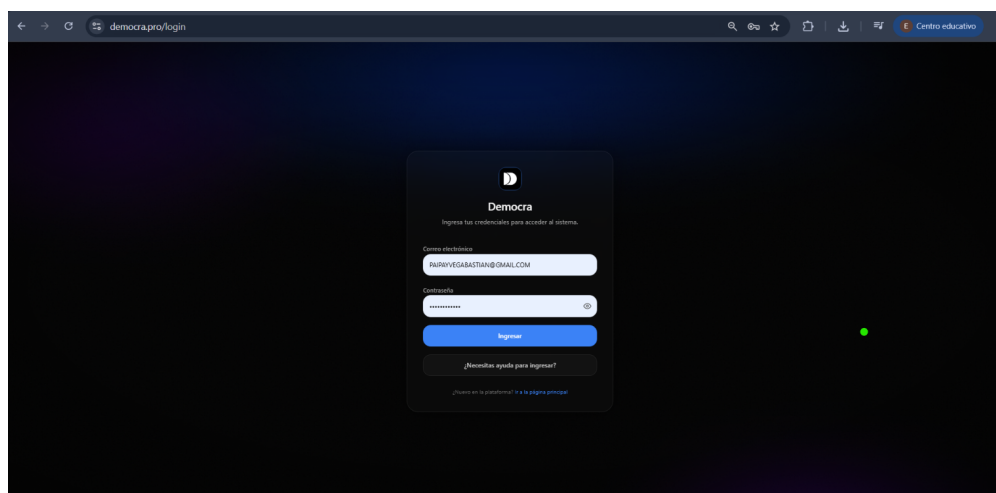
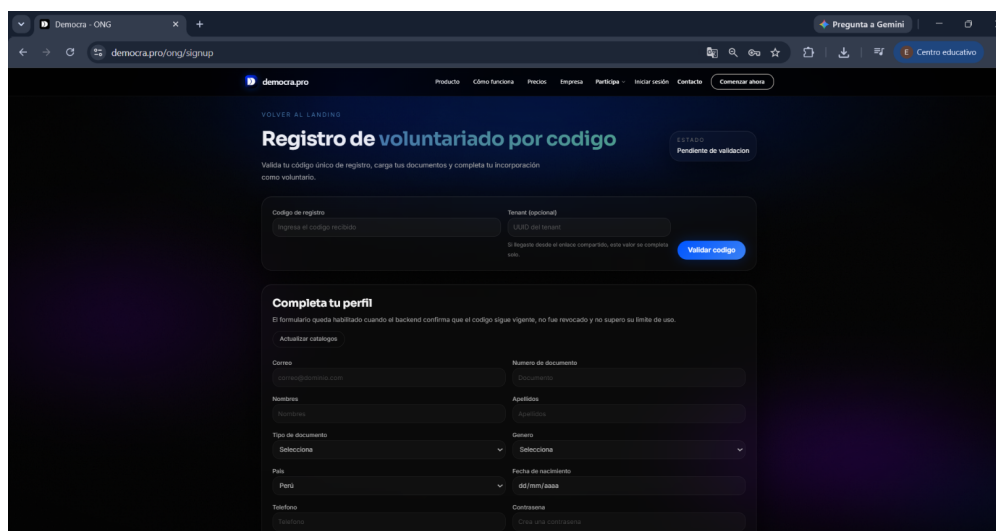


Figura 4: Prototipo del dashboard principal (vista 2)

B. Anexo B: Capturas del Sistema Implementado

Esta sección presenta las capturas de pantalla del sistema Democra en su versión final implementada, organizadas por áreas funcionales y módulos principales de la plataforma SaaS.

**Figura 5:** Página de aterrizaje (Landing Page)**Figura 6:** Inicio de sesión del sistema**Figura 7:** Registro de voluntario

The screenshot shows the 'Crea tu Organización' (Create your Organization) form on the democra.pro website. The form is titled 'Únete a Democra y gestiona tu ONG de forma profesional.' and includes the following fields:

- Nombre de la Organización: Fundación Esperanza
- RUC (tax ID): PUPAYVEGABASTIAN@GMAIL.COM
- Tu correo electrónico: fundador@organizacion.org
- Contraseña: [Redacted]

A blue button labeled 'Crear Organización' is at the bottom of the form. The footer of the page contains links for 'Producto', 'Empresa', 'Participa', and 'Legal'.

Figura 8: Creación de organización (Tenant)

The screenshot shows the 'Únete a Democra' (Join Democra) form on the democra.pro website. The form is titled 'Únete a Democra' and includes the following fields:

- Ingresar tu código de acceso para registrarte.
- Código de acceso: [Redacted]

A blue button labeled 'Continuar' is at the bottom of the form. The footer of the page contains links for 'Producto', 'Empresa', 'Participa', and 'Legal'.

Figura 9: Unión a organización existente

The screenshot shows the 'Panel principal' (Dashboard) on the democra.pro website. The dashboard displays various metrics and charts for the user 'PUPAYVEGABASTIAN@GMAIL.COM'. The metrics include:

- VOLUNTARIOS ACTIVOS: 1
- PROYECTOS ACTIVOS: 2
- ACTIVIDADES ACTIVES: 1
- HORAS REGISTRADAS: 0h
- HORAS APROBADAS: 0h
- RESPUESTAS SUBIDAS: 0
- SOLUCIONES PENDIENTES: 0
- HORAS PENDIENTES DE APROBACION: 0
- USUARIOS ACTIVOS: 1
- USUARIOS PENDIENTES: 0
- USUARIOS PENDIENTES DE APROBACION: 0
- USUARIOS APROBADOS: 1
- USUARIOS PENDIENTES: 0
- USUARIOS APROBADOS: 1

The dashboard also includes a section for 'HORAS APROBADAS - ÚLTIMOS 6 MESES' with a bar chart showing data for the last 6 months.

Figura 10: Panel principal (Dashboard)

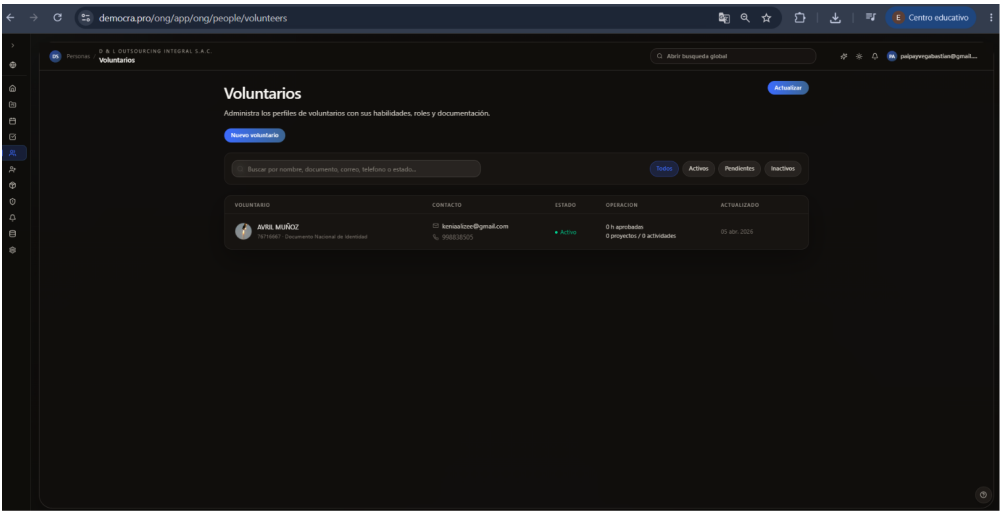


Figura 11: Gestión de voluntarios

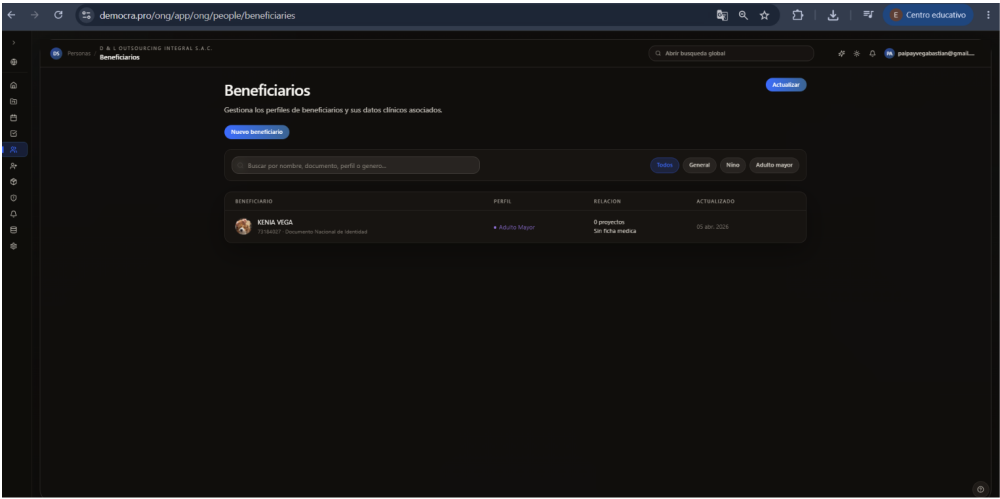


Figura 12: Gestión de beneficiarios

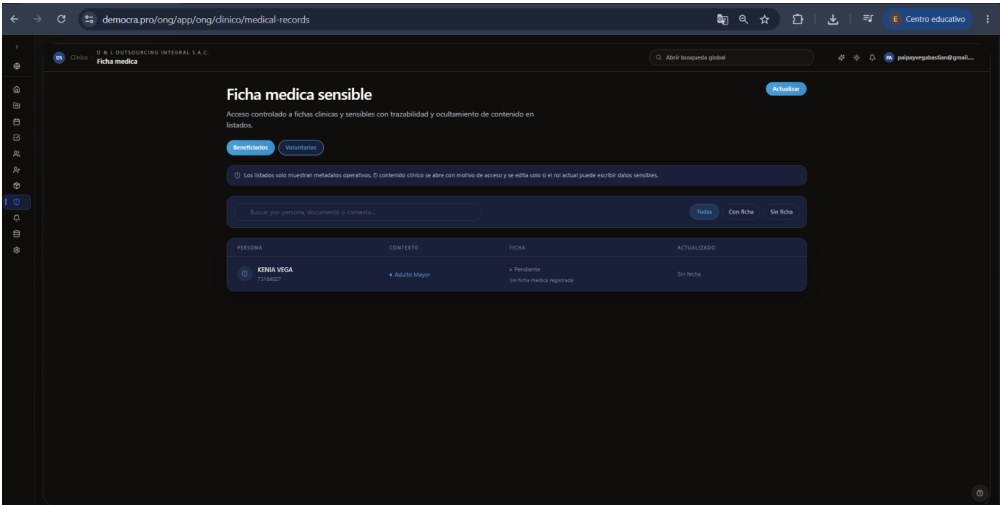


Figura 13: Ficha médica de beneficiario

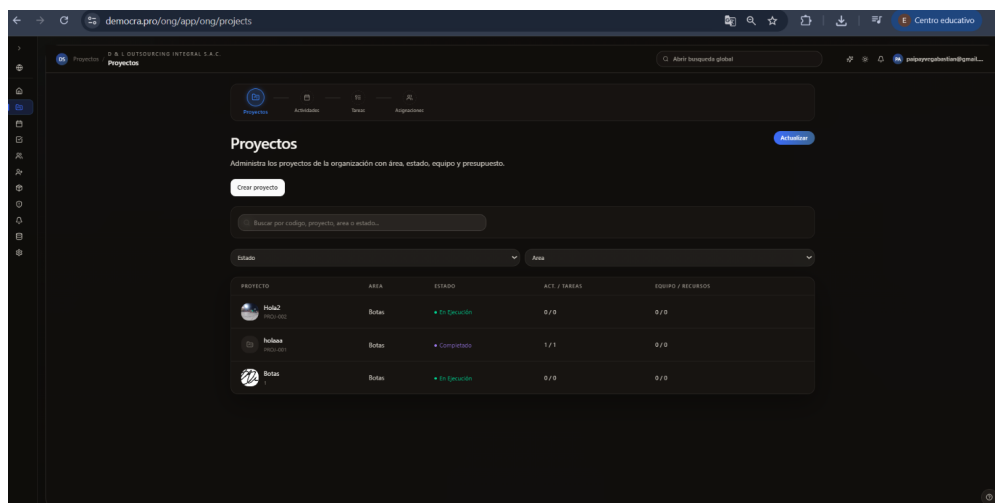


Figura 14: Gestión de proyectos

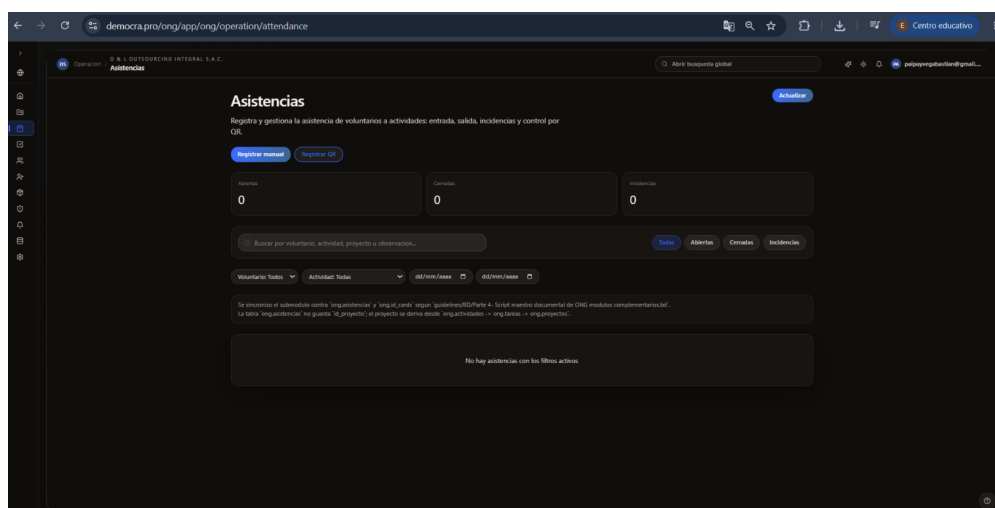


Figura 15: Operaciones del sistema

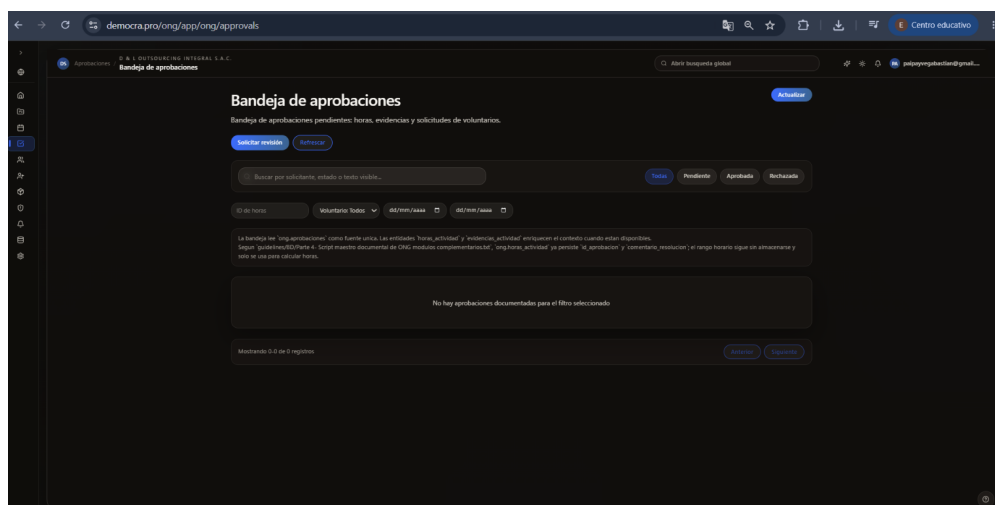


Figura 16: Aprobaciones y flujos de autorización

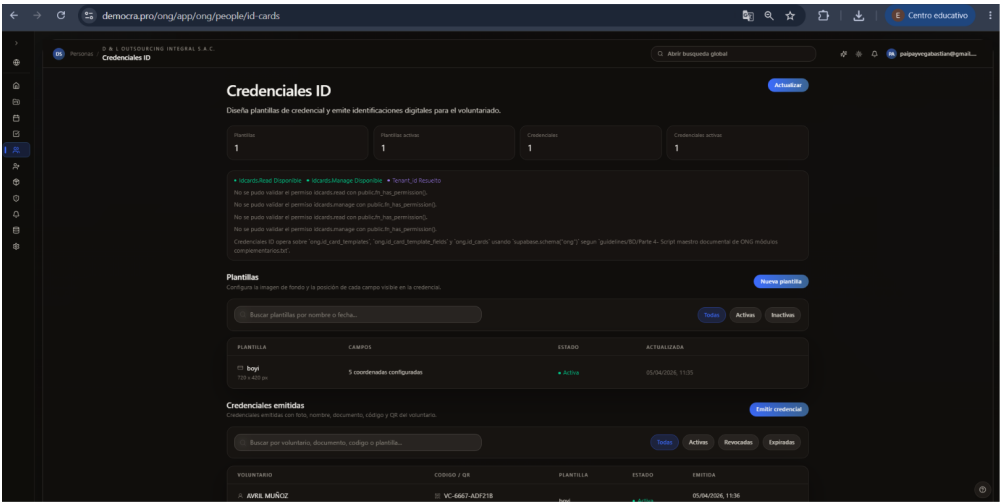


Figura 17: Emisión de credenciales digitales

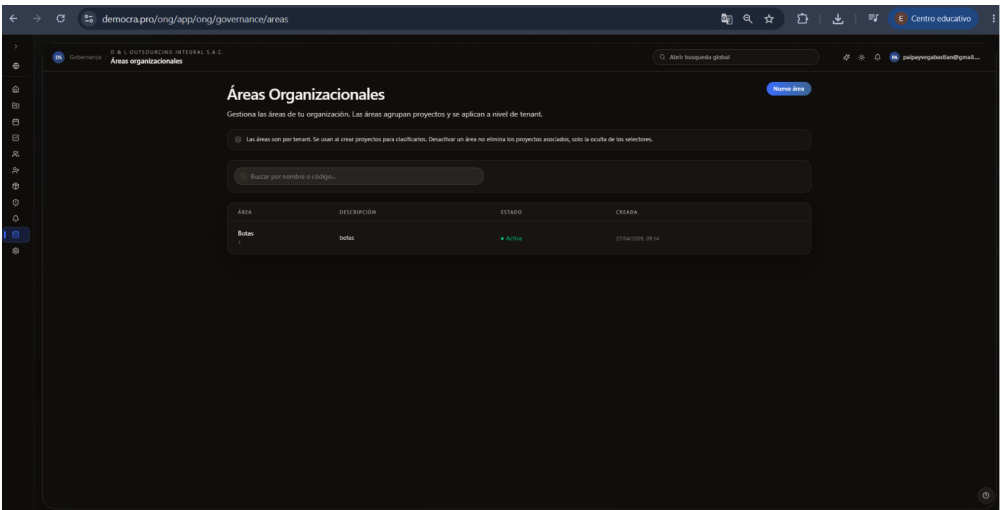


Figura 18: Gobernanza: Gestión de áreas

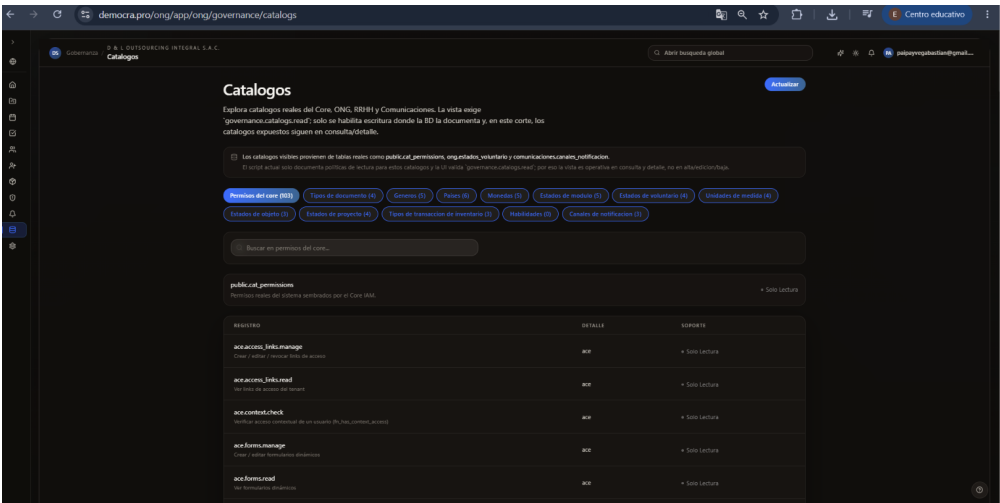


Figura 19: Gobernanza: Categorías del sistema

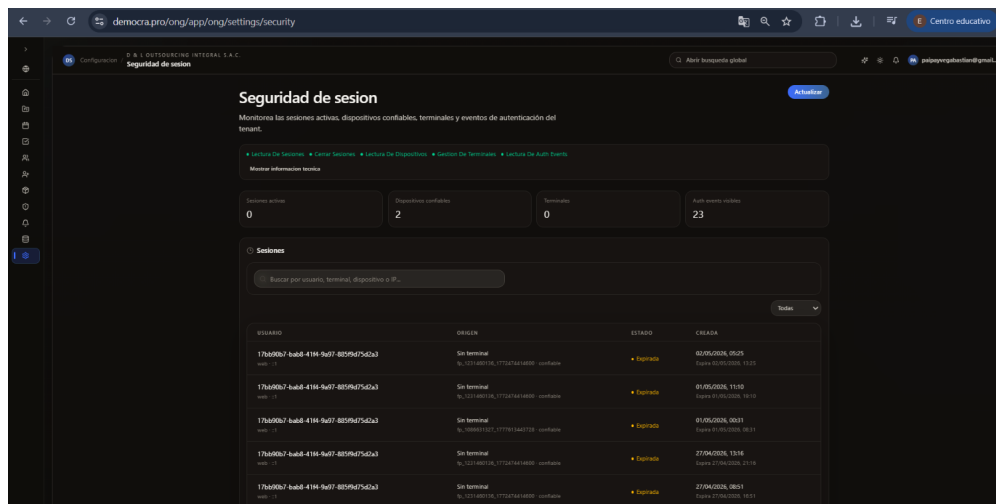


Figura 23: Gestión de sesiones activas

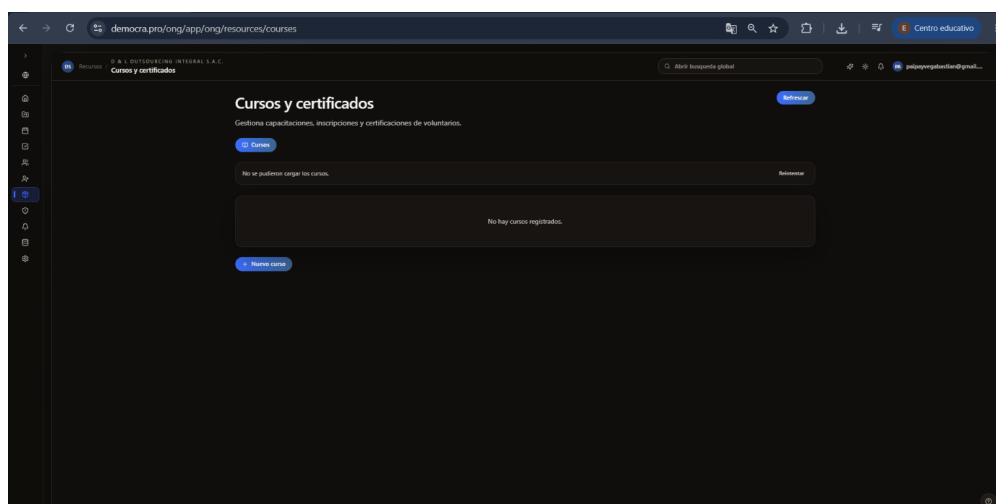


Figura 24: Recursos: Gestión de cursos

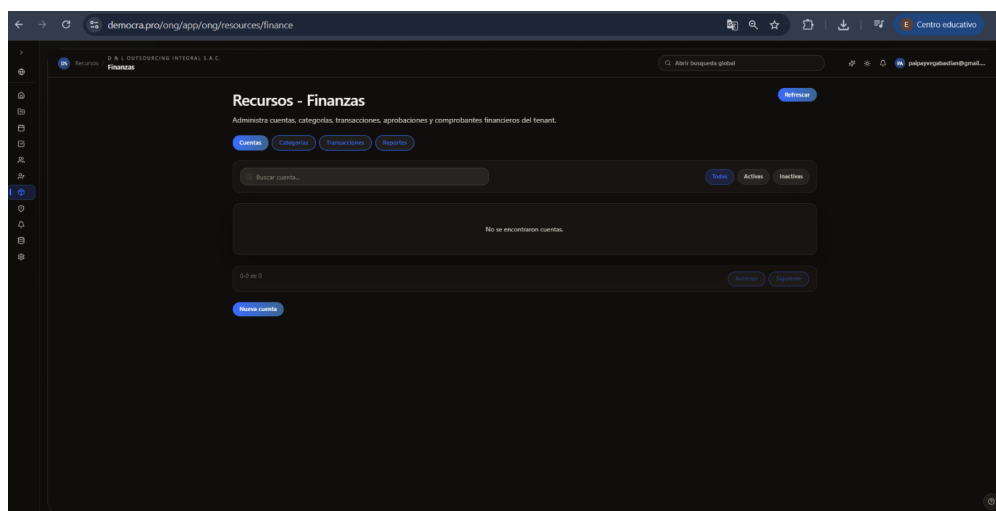


Figura 25: Recursos: Módulo financiero

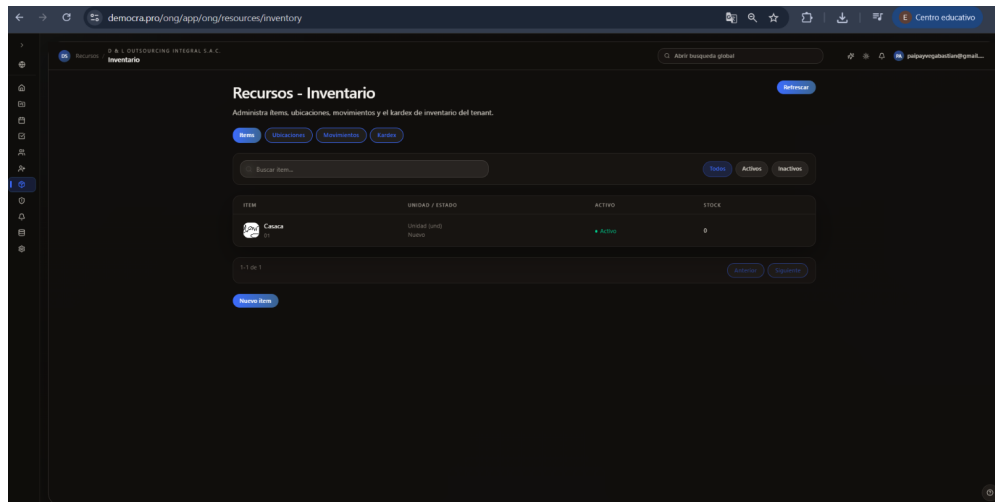


Figura 26: Recursos: Inventario de artículos

C. Anexo C: Evidencias Complementarias de Pruebas

A continuación, se presentan diagramas complementarios que ilustran la arquitectura y el stack tecnológico de la plataforma, sirviendo como base estructural para la ejecución de las pruebas documentadas en el Capítulo 5.

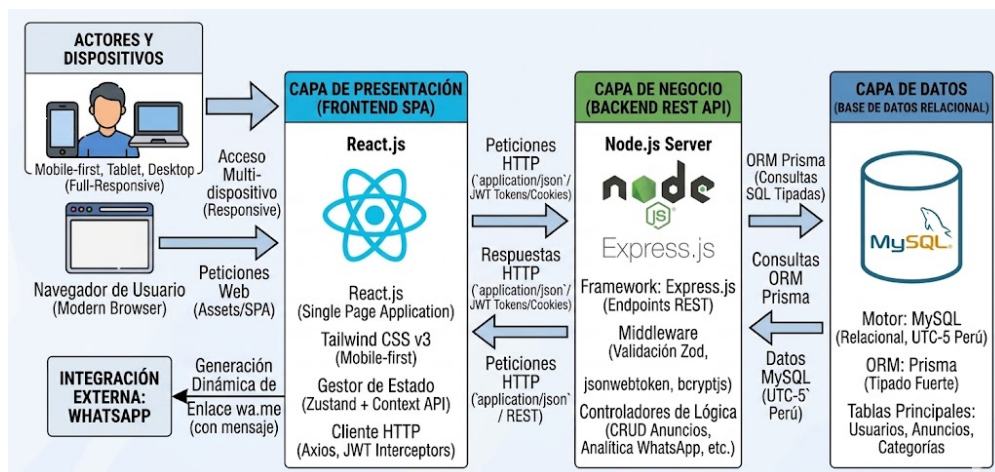


Figura 27: Diagrama de arquitectura de la plataforma Democra

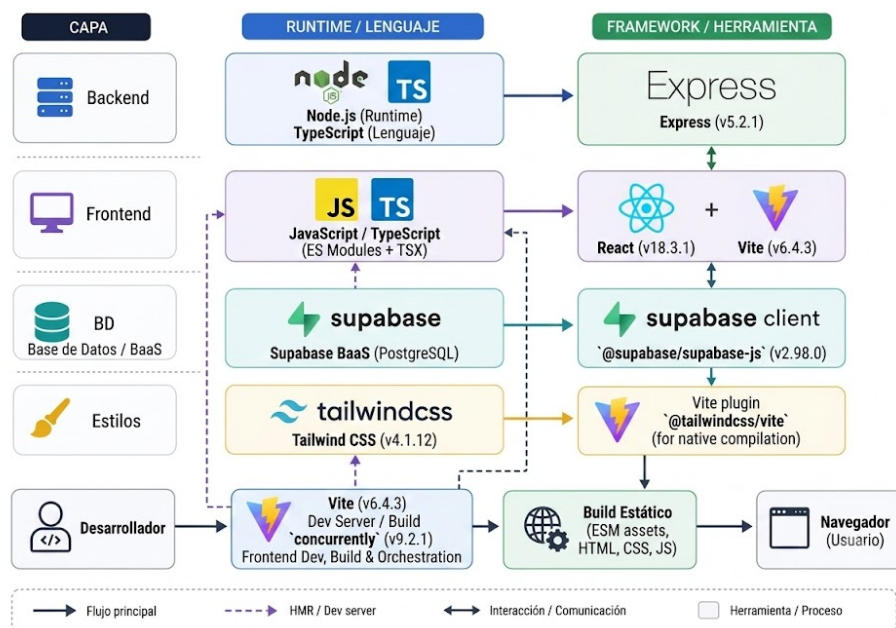


Figura 28: Diagrama estructural del stack tecnológico y estandarización arquitectónica del sistema